

APPENDIX 2 / RUST BACKEND MASTERY

付録 — 出てくるコードとコマンドの一覧

第0巻〜付録に出てくる 1936 個のコードの部品とコマンドを、機能ごとに、意味と初めて出てくる所つきで

付録: 出てくるコードとコマンドの一覧（機能ごと）

この一覧について

この本（第0巻〜第14巻）のコードに出てくる、**キーワード・マクロ・関数・メソッド・型・コマンド・オプション・書式・記号・設定ファイル・SQL・HTML・CSS など 1936 個**を、**63 の分類**に分けて、1 つずつ説明した一覧です。

- **本文との関係:** どの部品も、本文の中で**初めて出てくるコードのすぐ下**に、「**▶ ここで初めて出てくるコード**」の箱で同じ説明を載せています。この一覧は、それを機能ごとに並べ直した物です。
- **使い方:** 「この `.collect()` は何だっけ？」と思ったら、下の「分類の一覧」から分類を選んで探してください。HTML 版（`Rust\html`）なら、`Ctrl` + `F` で、コードの名前で探すのがいちばん速いです。
- **「初めて出てくる所」の列:** その部品が最初に出てくる巻と UNIT です。そこで、実際にどう使われているかを見られます。
- **読者が自分で書く名前**（`Note`・`load_user` など、この本のコードで自分で作る型や関数の名前）は、作った所の本文で説明しているので、この一覧には入れていません。ただし、作るより前の巻に先に出てくる物（`Flash`・`Db` など）は、「RustNote で作る物」の表に、意味と作る巻を載せています。
- **文字や記号の書き方も入っています:** `println!` の文字の中の書式（`{candidate:<22}` の `:<22` など）・`\n` のような特別な文字・正規表現・日時の書式（`%Y-%m-%d`）・記号（`&`・`..`・`_ =>`・`'a`）・数の書き方（`1_000`・`0xff`）・`Cargo.toml` や `fly.toml`・Dockerfile・GitHub Actions の設定・HTML の属性・コマンドのオプション（`cargo run --release` の `--release`）も、1 つずつ説明しています。
- **やりたいことから探す時は:** 次の付録 3「逆引き」で、「ファイルを読みたい」「幅をそろえて表示したい」のような、やりたいことからコードを探せます。本文に出てこない、合わせて覚えておくとよいコードも載せています。
- 同じ名前でも、使う所で意味が違う物（`.join()` が、場所をつなぐ意味と、文字列をつなぐ意味など）は、説明の中で両方を書いています。

> **ヒント:** 1 つの部品を覚えたら、その分類の表の、近くにある物も読んでみてください。例えば `Vec（並び）` の表には、作る（`vec!`）・足す（`.push()`）・取り出す（`.pop()`）・並べかえる（`.sort()`）が並んでいます。一緒に覚えると、使える場面がぐっと増えます。

分類の一覧

分類	数	分類	数
Rust の道具	17	モジュール	13
cargo のコマンド	86	所有権と借用	11
設定ファイル（TOML）	77	スマートポインタ	20

分類	数	分類	数
Rust の部品（クレート）	18	イテレーター	30
PowerShell	162	ファイルとフォルダ	36
コマンド	12	入出力	13
WSL・bash	43	プロセス	18
Git	26	設定	2
GitHub Actions	25	時間	18
Docker	37	標準ライブラリ	12
Fly.io	12	スレッドと共有	33
Node.js	15	非同期	68
設定ファイル（JSON）	10	ネットワーク	11
Python の道具	2	HTTP	25
コメント	4	HTTP クライアント	25
画面に出す	19	クッキー	9
変数	6	認証	25
数	47	ログ	24
文字	9	テスト	37
文字列	53	性能	18
文字列の中の特別な書き方	10	データベース	35
正規表現	19	SQL	97
Vec（並び）	30	JSON	10
HashMap（表）	17	Topcoat	84
範囲	2	RustNote で作る物	49
条件	10	HTML	54
くり返し	8	CSS	57
場合分け	9	JavaScript・TypeScript	91
Option	13	TypeScript	2

分類	数	分類	数
エラー処理	38	Playwright	26
関数	8	Python	94
型とメソッド	45		

Rust の道具

コード	意味と使い方	初めて出てくる所
`rustc`	Rust のコンパイラー（翻訳する係）。人が書いた `.rs` のファイルを、パソコンが直接動かせる実行ファイル（`.exe`）に翻訳します。ふだんは直接打たず、`cargo` が中で呼んでくれます。`rustc --version` で版を確かめられます。	第0巻 はじめに: 手を動かす準備
`rustc --version`	`rustc --version` は、入っている Rust のコンパイラーの版（`rustc 1.99.1 …`）を表示します。入れた直後に、ちゃんと入ったかを確かめる時に使います。	第0巻 はじめに: 手を動かす準備
`cargo`	Rust のプロジェクトをまとめて世話する道具。`cargo new`（作る）・`cargo run`（翻訳して動かす）・`cargo build`（翻訳だけ）・`cargo test`（テスト）・`cargo add`（部品を足す）のように、後ろに「何をするか」を付けて使います。`cargo --version` で版を確かめられます。	第0巻 はじめに: 手を動かす準備
`rustc --explain`	`rustc --explain E0384` は、エラーの番号（`error[E0384]` の部分）の、くわしい説明と例を表示します（英語）。	第0巻 UNIT 09
`rustup show`	入っている Rust の版と、今使っている版を表示します。`rustup show active-toolchain` は、今このフォルダで使われる版だけを表示します。	第1巻 UNIT 01
`rustup`	Rust そのもの（`rustc`・`cargo`）を入れる・新しくする・版を切りかえる道具。`rustup --version` で版を確かめられます。	第1巻 UNIT 04
`rustup --version`	`rustup --version` は、Rust を入れる道具 `rustup` の版を表示します。	第1巻 UNIT 04
`rustup component`	Rust の部品（`rustfmt`・`clippy` など）を見る・足すコマンド。`rustup component list --installed` で、入っている部品の一覧を出します。	第1巻 UNIT 04
`rustup component list --installed`	`--installed` を付けると、入っている部品（`rustfmt`・`clippy` など）だけを一覧にします。付けないと、入れられる部品が全部出ます。	第1巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`rustup update`	入っている Rust を、新しい版にするコマンド。`rustup update stable` で、安定版を最新にします。	第1巻 UNIT 04
`rustup toolchain`	入っている Rust の版（ツールチェーン）を見る・足す・消すコマンド。`rustup toolchain list` で一覧を出します。	第1巻 UNIT 04
`rustc -vV`	`rustc -vV` は、版をくわしく表示します。`host:` の行は、このパソコンがどの種類（`x86_64-pc-windows-msvc` など）かを表します。	第1巻 UNIT 05
`rustup target`	ほかの種類のパソコン（CPU・OS）向けに翻訳するための部品を足すコマンド。`rustup target add x86_64-pc-windows-msvc` で、x64 の Windows 用の実行ファイルを作れるようになります。	第1巻 UNIT 05
`rustup target list --installed`	`--installed` を付けると、入っている「ほかの種類のパソコン向けの部品」だけを一覧にします。	第1巻 UNIT 05
`rustup toolchain install --profile`	`--profile minimal` は、Rust の版を入れる時に、いちばん少ない部品（`rustc`・`cargo` など）だけを入れる指定です。説明書などを入れないので、速く入ります。	第1巻 UNIT 06
`rustup component add --toolchain`	`--toolchain 1.99.0` は、どの版の Rust に部品を足すかを指定します。付けないと、今使っている版に足します。	第1巻 UNIT 06
`rustup toolchain install --component`	`--component rustfmt,clippy` は、版を入れる時に、一緒に入れる部品を指定します。	第9巻 この巻で身に付けること

cargo のコマンド

コード	意味と使い方	初めて出てくる所
`cargo --version`	`cargo --version` は、Cargo の版を表示します。	第0巻 はじめに: 手を動かす準備
`cargo new`	新しい Rust のプロジェクト（プログラムをまとめるフォルダ）を作ります。`cargo new practice` で、`practice` フォルダの中に `Cargo.toml`（設定）と `src\main.rs`（プログラムを書くファイル）ができます。名前には英小文字・数字・`-`・`_` を使います。	第0巻 はじめに: 手を動かす準備

コード	意味と使い方	初めて出てくる所
`cargo run`	プログラムを翻訳（コンパイル）して、すぐ動かします。`Cargo.toml` があるフォルダで打ちます。プログラムが変わっていなければ翻訳を省くので、2 回目からは速くなります。`cargo run -- 値` のように `--` の後ろに書いた値は、プログラムへの引数（外から渡す値）になります。	第0巻 はじめに: 手を動かす準備
`cargo build`	プログラムを翻訳（コンパイル）して実行ファイルを作るだけで、動かしはしません。でき上がった物は `target\debug\` に入ります。`cargo build --release` だと、速く動く本番用の物が `target\release\` にできます。	第0巻 UNIT 01
`cargo run -q`	`-q`（quiet＝静かに）を付けると、Cargo 自身の「Compiling …」「Running …」の表示を出さず、プログラムが出した文だけを見せます。	第0巻 UNIT 03
`cargo run --`	`cargo run -- Rust "Web アプリ"` の `--` は、「ここから後ろは、Cargo ではなく、動かすプログラムに渡す」という区切りです。後ろの値は、プログラムの中で `std::env::args()` で受け取れます。	第0巻 UNIT 03
`cargo run --release`	`--release` を付けると、時間をかけて最適化した、速いプログラムを作ってから動かします。できた物は `target\release` に置かれます。付けない時（debug）は、速く翻訳できる代わりに、プログラムは遅くなります。速さを測る時は、必ず `--release` を付けます。	第0巻 UNIT 04
`cargo clippy`	動くけれど直した方がよい書き方を、理由と直し方の例つきで教えてくれる道具。`cargo clippy` と打つと、`warning:` の形で知らせてくれます。	第0巻 UNIT 09
`cargo test`	`#[test]` の付いた関数（テスト）を全部見つけて動かし、結果（`ok`・`FAILED`）を表示します。`cargo test 名前の一部` で、名前に合うテストだけを動かせます。	第0巻 UNIT 10
`cargo clippy --all-targets`	`--all-targets` は、本体だけでなく、テスト・例（`examples`）・ベンチマークのコードも全部調べる指定です。	第1巻 UNIT 03
`cargo clippy --locked`	`--locked` は、`Cargo.lock` の版のまま調べ、`Cargo.lock` を書きかえない指定です。	第1巻 UNIT 03
`cargo clippy --`	`cargo clippy -- -D warnings` の `--` は、「ここから後ろは、Cargo ではなく、調べる係（clippy）に渡す」という区切りです。	第1巻 UNIT 03
`cargo clippy -- -D`	`-D warnings` は、「警告（warning）を、エラーとして扱う」という指定です。警告が 1 つでもあると失敗するので、CI で「警告ゼロ」を守らせる時に使います。	第1巻 UNIT 03
`cargo build --target`	`cargo build --target x86_64-pc-windows-msvc` は、どの種類のパソコン（CPU と OS）向けに翻訳するかを指定します。できた物は `target\x86_64-pc-windows-msvc\…` に置かれます。	第1巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`cargo new --lib`	`cargo new --lib quality` は、実行するプログラムではなく、ほかから使う部品（ライブラリ）のプロジェクトを作ります。`src\main.rs` の代わりに `src\lib.rs` ができます。	第1巻 UNIT 06
`cargo check`	コードに間違いがないかだけを調べて、実行ファイルは作らないコマンド。`cargo build` より速いので、書きながら何度も打ちます。	第1巻 UNIT 06
`cargo fmt`	コードの書き方（字下げ・空白・改行）を、決まった形に自動で整えるコマンド。`cargo fmt --check` は、整っているかを調べるだけで、ファイルは書きかえません。	第1巻 UNIT 06
`cargo fmt --check`	`cargo fmt --check` は、ファイルを書きかえずに、「整っていない所があるか」だけを調べます。あれば違いを表示して失敗（終了コード 1）するので、CI（自動の確かめ）で使います。	第1巻 UNIT 06
`cargo clippy -q`	`-q` を付けると、clippy の表示を少なくして、警告と結果だけを見せます。	第1巻 UNIT 06
`cargo test -q`	`-q` を付けると、テスト 1 つずつの名前を出さず、`.` の並びと結果だけを短く表示します。	第1巻 UNIT 06
`cargo +1.99.0`	`+版` を付けると、その版の Rust で cargo を動かします。`cargo +1.99.0 fmt --check` は、Rust 1.99.0 の `rustfmt` で調べます（その版が `rustup` で入っている必要があります）。	第1巻 UNIT 06
`cargo +版`	`cargo +1.99.0 fmt --check` のように、`cargo` のすぐ後ろに `+版` を書くと、その 1 回だけ、指定した版の Rust で動かします（`rustup` で入れてある版に限ります）。	第1巻 UNIT 06
`cargo install`	Rust で作られた道具（コマンド）を、パソコンに入れるコマンド。`cargo install topcoat-cli --version 0.11.0 --locked` は、Topcoat の道具 `topcoat` を、決まった版で入れます。	第1巻 UNIT 08
`cargo install --version`	`cargo install topcoat-cli --version 0.11.0` の `--version` は、入れる道具の版を指定します。付けないと、いちばん新しい版を入れます。	第1巻 UNIT 08
`cargo install --locked`	`--locked` は、道具を作る時に、道具に付いている `Cargo.lock` の版のまま作る指定です。部品の新しい版のせいで作れない、ということを防げます。	第1巻 UNIT 08
`cargo +1.90.0`	`+1.90.0` で、Rust 1.90.0 を使って cargo を動かします。古い版で動くかを確かめる時に使います（その版が入っていないと、エラーになります）。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`cargo fmt --version`	`cargo fmt --version` は、コードを整える道具（rustfmt）の版を表示します。入っていない時はエラーになるので、入っているかの確かめにも使えます。	第1巻 UNIT 10
`cargo clippy --version`	`cargo clippy --version` は、コードの気になる所を教える道具（clippy）の版を表示します。	第1巻 UNIT 10
`cargo run --bin`	`cargo run --bin u01_stack_heap` は、`src\bin` の中の `u01_stack_heap.rs` を動かします。1 つのプロジェクトに、プログラムをいくつも置いた時に、どれを動かすかを選びます。	第3巻 UNIT 01
`cargo add`	部品（クレート）を、プロジェクトに足すコマンド。`cargo add thiserror anyhow serde_json` で、3 つの部品を `Cargo.toml` の `[dependencies]` に書き足します。`--features 機能` で、部品の追加の機能も選べます。部品は、世界中の人が公開している場所（crates.io）から自動で取ってきます。	第4巻 この巻で身に付けること
`cargo add --features`	`cargo add serde --features derive` の `--features` は、部品を足す時に、その部品の機能（ここでは `derive`）も ON にして、`Cargo.toml` に書き込みます。	第4巻 この巻で身に付けること
`cargo run -p`	`cargo run -p app` の `-p`（package）は、ワークスペース（いくつものパッケージをまとめた物）の中の、どのパッケージを動かすかを選びます。	第4巻 UNIT 06
`cargo add --optional`	`--optional` は、「いつもは使わず、機能（feature）を ON にした時だけ使う」部品として足す指定です。	第4巻 UNIT 07
`cargo run --features`	`cargo run --features json` は、`Cargo.toml` の `[features]` で決めた `json` という機能を ON にして動かします。	第4巻 UNIT 07
`cargo run --no-default-features`	`--no-default-features` は、`[features]` の `default` で ON にしてある機能を、全部 OFF にして動かします。	第4巻 UNIT 07
`cargo tree`	プロジェクトが使っている部品（クレート）と、その部品がさらに使っている部品を、木の形で表示するコマンド。`--depth 1` で、直接使っている物だけにできます。	第4巻 UNIT 07
`cargo tree --features`	`cargo tree --features json` は、`json` の機能を ON にした時に使われる部品の木を表示します。	第4巻 UNIT 07
`cargo build --locked`	`--locked` は、`Cargo.lock` に書いてある部品の版のまま作り、`Cargo.lock` を書きかえない指定です。版がずれていたらエラーで止まるので、いつ作っても同じ物ができます。本番用やテストの時に付けます。	第4巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`cargo add --no-default-features`	`--no-default-features` は、部品の「既定で ON の機能」を OFF にして足す指定です。必要な機能だけを `--features` で選ぶ時に使います。	第4巻 UNIT 07
`cargo doc`	コードに書いた説明（`///` のコメント）から、説明書（ドキュメント）の Web ページを作るコマンド。`--no-deps` で自分のコードだけ、`--open` でできたページをブラウザーで開きます。	第4巻 UNIT 10
`cargo doc --no-deps`	`--no-deps` は、自分のクレートの説明書だけを作り、使っている部品の説明書は作らない指定です。速く作れます。	第4巻 UNIT 10
`cargo test --doc`	`cargo test --doc` は、ドキュメントコメント（`///`）の中に書いた、例のコードだけをテストとして動かします。説明の例が、本当に動くかを確かめられます。	第4巻 UNIT 10
`cargo clippy --bin`	`cargo clippy --bin 名前` は、`src\bin` の中の 1 つだけを調べます。	第5巻 UNIT 01
`cargo build --bin`	`cargo build --bin e03_href` は、`src\bin` の中の 1 つだけを翻訳します。エラーの見本を、わざと翻訳して確かめる時などに使います。	第6巻 UNIT 03
`cargo run -- --setup`	`cargo run -q --bin sql -- --setup 準備.sql 実行.sql` の `--setup` は、練習用の道具 `sql`（第7巻で作る）のオプションです。`--setup` で渡した SQL は、結果を表示せずに先に流します（表やデータの準備用）。	第7巻 UNIT 01
`cargo run -- --db`	練習用の道具 `sql`（第7巻で作る）の `--db ファイル` は、使うデータベースのファイルを指定します。付けないと、メモリの中だけのデータベース（終わると消える）を使います。	第7巻 UNIT 06
`cargo test --test`	`cargo test --test u01_three_checks` は、`tests` フォルダの中の `u01_three_checks.rs` のテストだけを動かします。	第8巻 UNIT 01
`cargo test --`	`cargo test -- --nocapture` の `--` は、「ここから後ろは、Cargo ではなく、テストを動かすプログラムに渡す」という区切りです。	第8巻 UNIT 01
`cargo test -- --nocapture`	`--nocapture` は、テストの中の `println!` の表示を、そのまま画面に出す指定です。ふだんは、成功したテストの表示は隠されます。	第8巻 UNIT 01
`cargo test -- --test-threads`	`--test-threads=1` は、テストを 1 つずつ順番に動かす指定です。ふだんは、いくつものテストを同時に動かします。同じファイルやポートを使うテストが、ぶつからないようにする時に使います。	第8巻 UNIT 01
`cargo build --example`	`cargo build --example 名前` は、`examples` フォルダの中の 1 つだけを翻訳します（動かしません）。	第8巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`cargo run --example`	`cargo run --example u01_idor` は、`examples` フォルダの中の `u01_idor.rs` を動かします。例やエラーの見本を、本体とは別に置く時に使います。	第8巻 UNIT 01
`cargo test -- --list`	`--list` は、テストを動かさずに、テストの名前の一覧だけを表示します。	第9巻 UNIT 01
`cargo tree --depth`	`--depth 1` は、部品の木を何段目まで表示するかの指定です。`1` なら、自分が直接使っている部品だけです。	第9巻 UNIT 05
`cargo tree --charset`	`--charset ascii` は、木の線を、`|--` のような英数字の記号で書く指定です。文字化けしやすい画面でも読めます。	第9巻 UNIT 05
`cargo search`	公開されている部品（クレート）を、名前で探すコマンド。`cargo search topcoat --limit 1` で、`topcoat` のいちばん新しい版が分かります。	第9巻 UNIT 05
`cargo search --limit`	`cargo search topcoat --limit 1` の `--limit` は、crates.io を探した結果を何件表示するかの指定です。いちばん新しい版を確かめる時に使います。	第9巻 UNIT 05
`cargo update`	使っている部品を、`Cargo.toml` で許す範囲の新しい版にして、`Cargo.lock` を書きかえるコマンド。`-p 名前` で、その部品だけにできます。	第9巻 UNIT 05
`cargo update -p`	`cargo update -p topcoat` の `-p` は、`Cargo.lock` の中の、指定した部品の版だけを新しくする指定です。付けないと、全部の部品を新しくします。	第9巻 UNIT 05
`cargo update --dry-run`	`--dry-run` は、本当には書きかえずに、「新しくすると、どの部品がどの版になるか」だけを表示します。	第9巻 UNIT 05
`cargo test --lib`	`cargo test --lib` は、`src\lib.rs` とその中のモジュールに書いたテストだけを動かします。	第11巻 UNIT 01
`cargo add --dev`	`cargo add --dev scraper` は、テストの時だけ使う部品として、`[dev-dependencies]` に足します。本番の実行ファイルには入りません。	第11巻 UNIT 03
`cargo +nightly`	`+nightly` で、毎日作られている試し版（nightly）の Rust で cargo を動かします。ファズテスト（`cargo fuzz`）など、まだ安定版に無い機能を使う時に要ります。	第11巻 UNIT 04
`cargo mutants`	コードをわざと少しだけ変えて（`>` を `>=` にする など）、テストがそれに気付くかを確かめる道具（ミューテーションテスト）。気付かなければ、テストが足りない所です。	第11巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`cargo test -- --ignored`	`--ignored` は、`#[ignore]` を付けた（ふだんは飛ばす）テストだけを動かします。	第11巻 UNIT 06
`cargo test -- --include-ignored`	`--include-ignored` は、ふだんのテストと、`#[ignore]` を付けたテストの、両方を動かします。	第11巻 UNIT 06
`cargo bench`	速さを測るコード（ベンチマーク）を動かすコマンド。最適化した版（release）で動きます。	第11巻 UNIT 08
`cargo bench --bench`	`cargo bench --bench parse` は、`benches` フォルダの中の `parse.rs`（速さを測るコード）だけを動かします。	第11巻 UNIT 08
`cargo test --release`	`--release` を付けると、最適化した速いプログラムでテストを動かします。速さやメモリを測るテストで使います。	第11巻 UNIT 08
`cargo build --release`	`cargo build --release` は、時間をかけて最適化した、速い本番用の実行ファイルを作ります（動かしはしません）。できた物は `target\release` に置かれます。	第11巻 UNIT 09
`cargo test --locked`	`--locked` は、`Cargo.lock` の版のままテストし、`Cargo.lock` を書きかえない指定です。	第11巻 UNIT 10
`cargo audit`	使っている部品に、知られている弱点（セキュリティの問題）がないかを調べる道具。	第11巻 UNIT 10
`cargo tree -i`	`cargo tree -i rustls-webpki@0.102.8` の `-i`（invert＝逆さに）は、「その部品を、だれが使っているか」を、下からたどって表示します。古い版の部品が、どこから入ったかを調べる時に使います。	第11巻 UNIT 10
`cargo clean`	翻訳でできたファイル（`target` フォルダの中）を消すコマンド。`--release -p rustnote` で、release の RustNote の分だけを消します。次の翻訳は時間がかかります。	第12巻 UNIT 01
`cargo clean --release`	`cargo clean --release` は、`target\release`（本番用に作った物）だけを消します。	第12巻 UNIT 01
`cargo clean -p`	`cargo clean -p rustnote` は、指定したパッケージの、翻訳した物だけを消します。部品（依存クレート）の分は残るので、作り直しが速く済みます。	第12巻 UNIT 01
`cargo build --target-dir`	`--target-dir target\brepro` は、翻訳した物を置くフォルダを、いつもの `target` から変える指定です。ほかの組み立てと混ざらないように、分けて試す時に使います。	第12巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`cargo clean --target-dir`	`--target-dir` は、消す対象のフォルダ（`--target-dir` で作った物を置いたフォルダ）を指定します。	第12巻 UNIT 01
`cargo tree --locked`	`--locked` は、`Cargo.lock` の版のまま木を表示する指定です。	第13巻 UNIT 01
`cargo update --precise`	`cargo update -p tokio --precise 1.53.3` の `--precise` は、部品の版を、その版にぴったり合わせる指定です。	第13巻 UNIT 01
`cargo tree -p`	`-p tokio` は、指定した部品を根にした木だけを表示します。	第13巻 UNIT 01
`cargo tree -e`	`-e normal`（edges）は、どの種類のつながりを表示するかの指定です。`normal` なら、テスト用（dev）や組み立て用（build）の部品を除きます。	第13巻 UNIT 01
`cargo tree --edges`	`--edges normal` は `-e normal` と同じで、ふつうの部品のつながりだけを表示します。	第13巻 UNIT 01
`cargo check --locked`	`cargo check` は、実行ファイルを作らずに、エラーが無いかだけを速く調べます。`--locked` は、`Cargo.lock` の版のまま調べる指定です。	第13巻 UNIT 01
`cargo remove`	プロジェクトから部品（クレート）を外すコマンド。`Cargo.toml` の `[dependencies]` から、その行を消します。	第14巻 A. 道具の早見表
`cargo test --bin`	`cargo test --bin a18_tests` は、`src\bin` の中の `a18_tests.rs` に書いたテストだけを動かします。	第14巻 B. サンプル集

設定ファイル（TOML）

コード	意味と使い方	初めて出てくる所
`[package]`	`Cargo.toml` の `[package]` の下には、このプロジェクト（パッケージ）自身のこと（名前・版・Rust の版の決まり）を書きます。`[ ]` で囲んだ行は「ここから、この組の設定」という見出し（テーブル）です。	第0巻 はじめに: 手を動かす準備
`name =`	`name = "practice"` は、パッケージの名前です。`cargo run` で作られる実行ファイルの名前（`practice.exe`）にもなります。	第0巻 はじめに: 手を動かす準備

コード	意味と使い方	初めて出てくる所
`version =`	`version = "0.1.0"` は、このパッケージの版です。直したら数を上げます（「大きな変更.機能の追加.直し」の 3 つの数）。	第0巻 はじめに: 手を動かす準備
`edition =`	`edition = "2024"` は、「Rust の書き方の決まりの、どの年の版を使うか」です。2015・2018・2021・2024 があり、新しく作る時は `cargo new` が最新を書きます。	第0巻 はじめに: 手を動かす準備
`[dependencies]`	`[dependencies]` の下には、このプロジェクトが使う部品（クレート）を、`名前 = "版"` の形で 1 行ずつ書きます。`cargo add 名前` を打つと、ここに書き足されます。	第0巻 はじめに: 手を動かす準備
`#`	TOML（`Cargo.toml` などの設定ファイルの書き方）では、`#` から行の終わりまでがコメントです。	第1巻 UNIT 01
`[toolchain]`	`rust-toolchain.toml` の `[toolchain]` は、「このフォルダの中で使う Rust の版」を決める見出しです。このファイルがあるフォルダで `cargo` を打つと、書いた版が自動で使われます。	第1巻 UNIT 01
`channel =`	`channel = "1.99.1"` は、使う Rust の版です。`"stable"`（いちばん新しい安定版）とも書けますが、版を書いておくと、だれのパソコンでも同じ版になります。	第1巻 UNIT 01
`components =`	`components = ["rustfmt", "clippy"]` は、その版と一緒に入れる部品です。	第1巻 UNIT 01
`[ … ]`	`["rustfmt", "clippy"]` のように `[ ]` の中に `,` で区切って並べると、いくつもの値の並び（配列）になります。	第1巻 UNIT 01
`"1.0"`	部品の版の `"1.0.229"` は、「1.0.229 以上で、2.0.0 より前」を表します（新しい直しは自動で使われてよい）。`"=1.0.229"` のように `=` を付けると、「ぴったりこの版だけ」になります。実際に使う版は、`Cargo.lock` に記録されます。	第1巻 UNIT 08
`{ キー = 値 }`	`{ version = "1.0", features = ["derive"] }` のように `{ }` で囲むと、1 行の中に、いくつもの設定をまとめて書けます（インラインテーブル）。	第1巻 UNIT 08
`version =`	部品の `{ version = "1.51.1", … }` の `version` は、使う部品の版です。	第1巻 UNIT 08
`features =`	部品の `features = ["macros", "rt-multi-thread"]` は、その部品の、どの機能（feature）を ON にして使うかです。部品は、使わない機能を OFF にして小さくできるようになっています。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`default-features =`	`default-features = false` は、部品の「既定で ON の機能」を全部 OFF にする指定です。そのうえで、要る機能だけを `features = […]` で選びます。	第1巻 UNIT 08
`[profile.dev]`	`[profile.dev]` は、ふだんの `cargo build`・`cargo run`（debug）の時の、作り方の設定です。	第1巻 UNIT 08
`debug =`	`debug = "line-tables-only"` は、実行ファイルに入れる「調べるための情報」を、エラーの行番号が分かる分だけにする設定です。翻訳が速くなり、`target` フォルダも小さくなります。	第1巻 UNIT 08
`[profile.dev.package."*"]`	`[profile.dev.package."*"]` は、「自分のコード以外の、全部の部品」の作り方の設定です。部品だけ最適化（`opt-level = 2` など）すると、自分のコードの翻訳は速いまま、動きを速くできます。	第1巻 UNIT 08
`rust-version =`	`rust-version = "1.99"` は、「このパッケージを作るのに要る、いちばん古い Rust の版」です。古い Rust で作ろうとすると、分かりやすいエラーで止まります。	第2巻 UNIT 10
`[workspace]`	`[workspace]` は、いくつものパッケージ（`app`・`domain` など）を、1 つのまとまり（ワークスペース）にする見出しです。`Cargo.lock` と `target` フォルダを共有します。	第4巻 UNIT 06
`resolver =`	`resolver = "3"` は、部品の版の選び方の決まりの版です。ワークスペースでは、edition 2024 に合う `"3"` を書きます。	第4巻 UNIT 06
`members =`	`members = ["app", "domain"]` は、ワークスペースに入れるパッケージ（フォルダ）の一覧です。	第4巻 UNIT 06
`[workspace.package]`	`[workspace.package]` には、ワークスペースの全部のパッケージで共通の設定（`edition` など）を書きます。各パッケージは `edition.workspace = true` で受け取ります。	第4巻 UNIT 06
`[workspace.dependencies]`	`[workspace.dependencies]` には、ワークスペースで使う部品と版をまとめて書きます。各パッケージは `serde = { workspace = true }` で受け取ります。	第4巻 UNIT 06
`edition.workspace =`	`edition.workspace = true` は、「edition は、ワークスペースの `[workspace.package]` に書いた物を使う」という指定です。	第4巻 UNIT 06
`rust-version.workspace =`	`rust-version.workspace = true` は、「rust-version は、ワークスペースに書いた物を使う」という指定です。	第4巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`workspace =`	`serde = { workspace = true }` は、「部品の版は、ワークスペースの `[workspace.dependencies]` に書いた物を使う」という指定です。いくつものパッケージで、版をそろえられます。	第4巻 UNIT 06
`path =`	`domain = { path = "../domain" }` の `path` は、部品を crates.io からではなく、このパソコンのフォルダから使う指定です。ワークスペースの中の、自分で作った部品を使う時に書きます。	第4巻 UNIT 06
`optional =`	`optional = true` は、「この部品は、いつもは使わず、自分の機能（feature）を ON にした時だけ使う」という指定です。	第4巻 UNIT 07
`[features]`	`[features]` には、このパッケージの「ON・OFF できる機能」を、`機能の名前 = [ON にした時に一緒に使う物]` の形で書きます。コードでは `#[cfg(feature = "名前")]` で、その機能の時だけの部分を書き分けます。	第4巻 UNIT 07
`default =`	`[features]` の `default = ["markdown"]` は、「何も指定しない時に ON になる機能」の一覧です。`--no-default-features` で OFF にできます。	第4巻 UNIT 07
`機能の名前 =`	`[features]` の `json = ["dep:serde_json"]` のような行は、`json` という機能を作り、ON にした時に一緒に使う部品（`dep:` を付けた名前）を並べます。`csv = []` のように空なら、部品は増えず、コードの書き分けだけに使います。	第4巻 UNIT 07
`csv =`	`[features]` に書く `csv = []` は、`csv` という機能を足す書き方です。`[]` が空なので、ON にしても部品は増えません。コードの `#[cfg(feature = "csv")]` の部分だけが ON になります。	第4巻 UNIT 07
`[[assets]]`	`topcoat asset bundle` が作る目録（`manifest`）の `[[assets]]` は、束ねたアセット（CSS など）1 つ分の記録です。	第6巻 UNIT 08
`id =`	アセットの `id` は、アセットを見分けるための番号です（中身から計算されます）。	第6巻 UNIT 08
`file =`	アセットの `file` は、束ねた後のファイル名です。名前に中身の指紋（`7becc8b6…`）が入るので、中身が変わると名前も変わり、ブラウザーが古い物を使い続けることがありません。	第6巻 UNIT 08
`hash =`	`hash` は、ファイルの中身から計算した指紋（長い 16 進数）です。	第6巻 UNIT 08
`content_type =`	`content_type = "text/css"` は、そのファイルの種類です。ブラウザーに渡す時の `Content-Type` になります。	第6巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`description =`	`description = "…"` は、パッケージの短い説明です。	第9巻 UNIT 05
`publish =`	`publish = false` は、「このパッケージは crates.io（みんなで部品を共有する所）に公開しない」という印です。間違って公開するのを防ぎます。	第9巻 UNIT 05
`default-run =`	`default-run = "rustnote"` は、実行ファイルがいくつもある時に、`cargo run` だけで動かす物を決めます。	第9巻 UNIT 05
`[dev-dependencies]`	`[dev-dependencies]` の下には、テストの時だけ使う部品を書きます。本番の実行ファイルには入りません。	第9巻 UNIT 05
`[[http_service.checks]]`	`[[http_service.checks]]` は、Fly.io がアプリの元気さを確かめる方法（ヘルスチェック）を 1 つ登録する見出しです。	第9巻 UNIT 10
`grace_period =`	`grace_period = "10s"` は、アプリが起動してから、確かめを始めるまで待つ時間です。	第9巻 UNIT 10
`interval =`	`interval = "30s"` は、確かめる間かくです（30 秒ごと）。	第9巻 UNIT 10
`method =`	`method = "GET"` は、確かめに送る要求の種類です。	第9巻 UNIT 10
`path =`	`path = "/health"` は、確かめに開く URL の場所です。アプリに、その URL が無いと、ずっと「元気でない」と判定されます。	第9巻 UNIT 10
`timeout =`	`timeout = "5s"` は、答えを待つ時間です。それより遅いと、失敗と数えます。	第9巻 UNIT 10
`[[bench]]`	`[[bench]]` は、速さを測るプログラム（ベンチマーク）を 1 つ登録する見出しです。`[[ ]]` と 2 重にした見出しは、何回も書いて、同じ種類の物を並べられます。	第11巻 UNIT 08
`harness =`	`harness = false` は、Rust の標準のベンチマークの係を使わず、`criterion` などの部品が自分で測る、という指定です。	第11巻 UNIT 08
`[advisories]`	`deny.toml`（`cargo deny` の設定）の `[advisories]` は、「弱点が知られている部品」の確かめ方の設定です。	第11巻 UNIT 10
`ignore =`	`ignore = ["RUSTSEC-…"]` は、知られている弱点のうち、「今は直せないので、理由を書いて見逃す」物の一覧です。見直す日を、コメントに書いておきます。	第11巻 UNIT 10
`[profile.release]`	`[profile.release]` は、`--release` を付けて作る時（本番用）の、作り方の設定です。	第12巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`overflow-checks =`	`overflow-checks = true` は、本番用でも、数があふれた時（`u8` に 256 を入れた時など）に止まって知らせる設定です。ふだん本番用では、確かめずに、あふれた数がそのまま回り込みます。	第12巻 UNIT 01
`app =`	`fly.toml` の `app = "rustnote-example"` は、Fly.io の上のアプリの名前です。	第12巻 UNIT 04
`primary_region =`	`primary_region = "nrt"` は、アプリを動かす主な場所（地域）です。`nrt` は東京（成田）です。	第12巻 UNIT 04
`[build]`	`fly.toml` の `[build]` は、イメージの作り方の見出しです。	第12巻 UNIT 04
`dockerfile =`	`dockerfile = "Dockerfile"` は、イメージを作る時に使う Dockerfile のファイル名です。	第12巻 UNIT 04
`[env]`	`fly.toml` の `[env]` の下には、アプリに渡す環境変数を `名前 = "値"` で書きます。秘密の値（パスワードなど）は、ここではなく `fly secrets set` で渡します（`fly.toml` は Git に入れて、みんなに見えるからです）。	第12巻 UNIT 04
`HOST =`	`[env]` の `HOST = "0.0.0.0"` は、「どのアドレスで待ち受けるか」を渡します。`0.0.0.0` は「全部」で、コンテナの外から届くようにするために要ります。	第12巻 UNIT 04
`PORT =`	`[env]` の `PORT = "8080"` は、アプリが待ち受けるポート番号です。`[http_service]` の `internal_port` と同じ数にします。	第12巻 UNIT 04
`[http_service]`	`fly.toml` の `[http_service]` は、外からの HTTP（Web）の受け方の見出しです。	第12巻 UNIT 04
`internal_port =`	`internal_port = 8080` は、Fly.io が、外から来た要求を、アプリのどのポートに渡すかです。アプリが待ち受けるポート（`PORT`）と同じでないと、つながりません。	第12巻 UNIT 04
`force_https =`	`force_https = true` は、`http://` で来た人を、暗号化した `https://` へ移す設定です。	第12巻 UNIT 04
`auto_stop_machines =`	`auto_stop_machines = "stop"` は、だれも使っていない時に、アプリの機械（マシン）を自動で止める設定です。お金を節約できます。	第12巻 UNIT 04
`auto_start_machines =`	`auto_start_machines = true` は、止まっている時に要求が来たら、自動で起こす設定です。	第12巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`min_machines_running =`	`min_machines_running = 0` は、いつも動かしておく機械の最低の数です。0 なら、だれも使っていない時は全部止まります（最初の 1 回の表示が少し遅くなります）。	第12巻 UNIT 04
`[[vm]]`	`fly.toml` の `[[vm]]` は、アプリを動かす機械（マシン）の大きさの見出しです。	第12巻 UNIT 04
`memory =`	`memory = "512mb"` は、機械のメモリの大きさです。	第12巻 UNIT 04
`cpu_kind =`	`cpu_kind = "shared"` は、CPU をほかの人と分け合う、安い種類にする設定です。	第12巻 UNIT 04
`cpus =`	`cpus = 1` は、CPU の数です。	第12巻 UNIT 04
`[deploy]`	`fly.toml` の `[deploy]` は、公開（deploy）のしかたの見出しです。	第12巻 UNIT 07
`release_command =`	`release_command = "/app/rustnote --migrate-only"` は、新しい版を動かす前に、1 回だけ動かすコマンドです。ここでは、データベースの表を新しい形にします（マイグレーション）。失敗したら、新しい版に切りかえません。	第12巻 UNIT 07
`[[package]]`	`Cargo.lock` の `[[package]]` は、使っている部品 1 つ分の記録（名前・版・どこから取ったか・指紋）です。`Cargo.lock` は Cargo が自動で書くので、手では書きかえません。	第13巻 UNIT 01
`dependencies =`	`Cargo.lock` の `dependencies = […]` は、その部品が、さらに使っている部品の一覧です。	第13巻 UNIT 01
`source =`	`Cargo.lock` の `source = "registry+https://github.com/rust-lang/crates.io-index"` は、「その部品を crates.io から取った」という記録です。	第13巻 UNIT 01
`checksum =`	`Cargo.lock` の `checksum` は、部品の中身の指紋です。取ってきた部品が、記録の時と 1 文字でもちがえば、エラーで止まります（すりかえを防ぎます）。	第13巻 UNIT 01

Rust の部品（クレート）

コード	意味と使い方	初めて出てくる所
`tokio`	`tokio` は、Rust で非同期（待ち時間の間に、ほかの仕事を進める）のプログラムを動かすための部品です。たくさんの人の要求を、同時にさばく Web のサーバーの土台になります。	第1巻 UNIT 08
`topcoat`	`topcoat` は、Rust の Web アプリを作るための部品（フレームワーク）です。画面（`view!`）・URL ごとの処理（`#[page]`）・セッションなどを用意しています。	第1巻 UNIT 08
`anyhow`	`anyhow` は、いろいろな種類のエラーを、1 つの型（`anyhow::Error`）でまとめて扱い、説明を書き足せる（`.context(…)`）部品です。アプリの一番上の、エラーを表示する所で使います。	第4巻 この巻で身に付けること
`serde`	`serde` は、Rust の値を、JSON などの形に変えたり（シリアライズ）、JSON から Rust の値を作ったり（デシリアライズ）するための部品です。`features = ["derive"]` で、`#[derive(Serialize, Deserialize)]` が使えるようになります。	第4巻 この巻で身に付けること
`serde_json`	`serde_json` は、`serde` と組み合わせて、JSON を読み書きする部品です。	第4巻 この巻で身に付けること
`thiserror`	`thiserror` は、自分のエラーの型を、`#[derive(Error)]` と `#[error("…")]` だけで作れるようにする部品です。	第4巻 この巻で身に付けること
`argon2`	`argon2` は、パスワードを、元に戻せない形（ハッシュ）にして保存するための部品です。わざと時間とメモリを使う計算なので、盗まれても、元のパスワードを当てにくくなります。	第5巻 UNIT 08
`reqwest`	`reqwest` は、Rust のプログラムから、ほかのサーバーへ HTTP の要求を送る（ブラウザーのように Web にアクセスする）部品です。	第5巻 UNIT 08
`tokio-util`	`tokio-util` は、`tokio` で使う便利な道具（止める合図の `CancellationToken` など）を集めた部品です。	第5巻 UNIT 08
`libsql`	`libsql` は、Turso（SQLite の仲間のデータベース）や、このパソコンの SQLite のファイルに、Rust からつなぐための部品です。	第7巻 この巻で身に付けること
`base64`	`base64` は、バイトの並びを、英数字と `+`・`/` だけの文字（Base64）に変えたり、戻したりする部品です。	第8巻 UNIT 09
`hmac`	`hmac` は、秘密の鍵を使って、データに「改ざんされていない印」（HMAC）を付ける計算の部品です。	第8巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`rand`	`rand` は、でたらめな数（乱数）を作る部品です。	第8巻 UNIT 09
`sha1`	`sha1` は、SHA-1 という古い指紋の計算をする部品です。安全のためではなく、昔の決まり（ほかの仕組みとの比べっこ）に合わせるために使います。	第8巻 UNIT 09
`sha2`	`sha2` は、SHA-256 などの、データの指紋（ハッシュ）を計算する部品です。	第8巻 UNIT 09
`async-trait`	`async-trait` は、トレイトの中に `async fn` を書き、それを `dyn`（中身の型を後で決める）で使えるようにする部品です。	第9巻 UNIT 05
`dotenvy`	`dotenvy` は、`.env` のファイルに書いた `名前=値` を読み、環境変数にする部品です。開発の時の設定に使います。	第9巻 UNIT 05
`tempfile`	`tempfile` は、テスト用の一時的なファイルやフォルダを作り、使い終わったら自動で消す部品です。	第9巻 UNIT 05

PowerShell

コード	意味と使い方	初めて出てくる所
`cd`	「change directory（フォルダを移る）」の略。`cd rust-study` で、今いるフォルダの中の `rust-study` へ移ります。`cd ..` は 1 つ上のフォルダへ、`cd ~` はホームフォルダ（`C:\Users\you`）へ戻ります。今いる場所は、行の頭の `PS C:\Users\you\rust-study>` の部分に表示されます。	第0巻 はじめに: 手を動かす準備
`mkdir`	「make directory（フォルダを作る）」の略。`mkdir rust-study` で、今いるフォルダの中に `rust-study` という新しいフォルダを作ります。もう同じ名前のフォルダがあると、エラーになります。	第0巻 はじめに: 手を動かす準備
`#`	`#` から行の終わりまでは「コメント」（人が読むためのメモ）で、PowerShell は無視します。コマンドの横に、何をするかを日本語で書いておけます。コピーしてそのまま貼り付けても、問題ありません。	第0巻 はじめに: 手を動かす準備
`.\target\debug\practice.exe`	`cargo build` が作った実行ファイルを、直接動かす書き方。PowerShell では、今いるフォルダのプログラムを動かす時、頭に `.\`（「ここの」という意味）を付ける決まりです。	第0巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`$LASTEXITCODE`	`$LASTEXITCODE` は、最後に動かしたプログラム（`cargo`・`git`・`psql` など、`.exe` のコマンド）の「終了コード」が入る、特別な変数です。`0` なら成功、`0` 以外なら失敗です。`if ($LASTEXITCODE -ne 0) { … }` で、失敗した時の処理を書けます。	第0巻 UNIT 01
`.\`	`.\target\debug\practice.exe` の `.\` は、「今いるフォルダの中の」という意味です。PowerShell は、安全のため、今いるフォルダのプログラムでも `.\` を付けないと動かしません。	第0巻 UNIT 01
`Get-Content`	ファイルの中身を画面に出すコマンド（短い呼び名は `cat`）。`Get-Content memo.txt` で中身を表示します。日本語のファイルは `-Encoding utf8` を付けると文字化けしません。`-Tail 10` で最後の 10 行だけを出せます。	第0巻 UNIT 01
`>`	`コマンド > out.txt` は、画面に出るはずの文を、`out.txt` というファイルに書き込みます（前の中身は消えます）。後ろに書き足したい時は `>>` を使います。	第0巻 UNIT 01
`.\practice.exe`	今いるフォルダにある `practice.exe` を動かす書き方。PowerShell では、安全のため、今いるフォルダのプログラムにも `.\`（「ここの」という意味）を付けて呼ぶ決まりです。	第0巻 UNIT 02
`Get-Location`	今いるフォルダ（カレントディレクトリ）を表示するコマンド。短い呼び名は `pwd` です。	第0巻 UNIT 03
`Set-Content`	ファイルに文字を書くコマンド。`Set-Content memo.txt "1 行目"` で、ファイルを作るか、中身を置きかえます。後ろに書き足す時は `Add-Content` を使います。	第0巻 UNIT 03
`Set-Content -Encoding`	`-Encoding utf8` で、ファイルを UTF-8 という文字コードで書きます。Windows PowerShell 5.1 は、指定しないと古い文字コード（Shift_JIS）で書くので、日本語を書く時は付けておくと安心です。	第0巻 UNIT 03
`ls`	フォルダの中の一覧を見るコマンド（正式な名前は `Get-ChildItem`）。`Mode` の列の `d` はフォルダ、`a` はファイルです。	第0巻 UNIT 03
`cat`	ファイルの中身を表示するコマンド（正式な名前は `Get-Content`）。	第0巻 UNIT 03
`Copy-Item`	ファイルやフォルダをコピーするコマンド（短い呼び名は `cp`）。`Copy-Item 元 先`。フォルダを中身ごとコピーする時は `-Recurse` を付けます。	第0巻 UNIT 03
`Remove-Item`	ファイルやフォルダを消すコマンド（短い呼び名は `rm`）。**ごみ箱に入らず、すぐ消えます。**フォルダを中身ごと消す時は `-Recurse` を付けます。消す前に、名前をよく確かめます。	第0巻 UNIT 03

コード	意味と使い方	初めて出てくる所
`$env:名前`	`$env:GREETING` のように `$env:` を付けた名前は「環境変数」（プログラムに外から渡す設定）です。`$env:GREETING = "おはよう"` で設定すると、その PowerShell の窓から動かすプログラム（`cargo run` など）が読めます。窓を閉じると消えます。	第0巻 UNIT 03
`.\main.exe`	今いるフォルダの `main.exe` を動かす書き方。`rustc main.rs` で翻訳すると、同じフォルダに `main.exe` ができます。	第0巻 UNIT 04
`Get-NetTCPConnection`	パソコンのネットワークの接続や、待ち受けているポート番号を一覧にするコマンド。「8080 番をだれが使っているか」を調べる時に使います。	第0巻 UNIT 06
`Get-NetTCPConnection -LocalPort`	`-LocalPort 8080` で、このパソコン側のポート番号が 8080 の物だけに絞ります。	第0巻 UNIT 06
`Get-NetTCPConnection -State`	`-State Listen` で、「待ち受けている」状態の物だけに絞ります（ほかに `Established` ＝つながっている、など）。	第0巻 UNIT 06
`Select-Object`	結果のうち、見たい列だけを選ぶコマンド。`… | Select-Object Name, Length` のように、`|`（パイプ）で前のコマンドの結果を受け取って使います。`-First 5` で最初の 5 つだけにもできます。	第0巻 UNIT 06
`Get-Process`	動いているプログラム（プロセス）の一覧を出すコマンド。`Get-Process -Id 1234` で、番号 1234 のプロセスの名前などが分かります。	第0巻 UNIT 06
`Get-Process -Id`	`-Id 番号` で、プロセスの番号（PID）を指定して 1 つだけ見ます。	第0巻 UNIT 06
`|`	`|`（パイプ）は、左のコマンドの結果を、右のコマンドに流して渡す記号です。`Get-Service | Where-Object { … } | Select-Object Name` のように、「取り出す → しぼる → 選ぶ」をつないで書けます。キーボードでは `Shift` ＋ `¥` で打ちます。	第0巻 UNIT 06
`.OwningProcess`	`Get-NetTCPConnection` の結果の `.OwningProcess` は、そのポートを使っているプログラムの番号（PID）です。`Get-Process -Id 番号` で、プログラムの名前が分かります。	第0巻 UNIT 06
`$PSVersionTable`	`$PSVersionTable` は、PowerShell の版などが入った、特別な変数です。`$PSVersionTable.PSVersion` で版（5.1 や 7.6 など）が分かります。	第1巻 UNIT 01
`.PSVersion`	`$PSVersionTable.PSVersion` は、PowerShell の版です。`.Major` で一番上の数（5 や 7）だけを取り出せます。	第1巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`Set-Content -Path`	`-Path 名前` で、書き込むファイルの名前（場所）を指定します。`Set-Content memo.txt "…"` のように名前を最初に書く時は、`-Path` を省けます。	第1巻 UNIT 01
`Set-Content -Value`	`-Value 中身` で、ファイルに書く中身を指定します。`@' … '@` で囲むと、何行もの文章をそのまま中身にできます（ヒア文字列）。	第1巻 UNIT 01
`Get-CimInstance`	Windows の中の情報（OS・CPU・サービス・ディスクなど）を問い合わせるコマンド。`Get-CimInstance Win32_OperatingSystem` は OS の情報、`Get-CimInstance Win32_Service` はサービスの一覧（自動で始まるかどうかも分かる）です。	第1巻 UNIT 01
`Select-Object -First`	`-First 1` で、結果の最初の 1 つだけを取り出します。`-Last 3` なら最後の 3 つです。	第1巻 UNIT 01
`switch`	値によって、することを分ける書き方。`switch ($x) { 'A' { … } 'B' { … } default { … } }`。合った所の `{ }` の中だけが動きます。結果を `$machine = switch (…) { … }` のように受け取ることもできます。	第1巻 UNIT 01
`default`	PowerShell の `switch` の中で、「どの場合にも合わなかった時」の処理を書く所。Rust の `match` の `_ =>` に当たります。	第1巻 UNIT 01
`Write-Output`	値を「出力」として次へ流す（最後なら画面に出す）コマンド。`Write-Output "こんにちは"`。文字列を 1 行に書くだけでも同じ働きをしますが、はっきり書く時に使います。	第1巻 UNIT 01
`if`	PowerShell の条件の書き方。`if (条件) { 合う時 } else { 合わない時 }`。条件は `( )` で囲みます。比べる時は `-eq`（等しい）・`-ne`（違う）・`-gt`（大きい）・`-match`（パターンに合う）のような書き方をします（`==` や `>` ではありません）。	第1巻 UNIT 01
`-match`	正規表現（文字の形の決まり）に合うかの比べ方。`$line -match '^\s*#'` は「空白のあと `#` で始まる行（コメント）か」。合った部分は `$Matches` で取り出せます。	第1巻 UNIT 01
`else`	`if (条件) { … } else { … }` の、条件に合わなかった時の所。`elseif (条件)` で、条件をつなげられます。	第1巻 UNIT 01
`-and`	「かつ」（両方とも `True` の時だけ `True`）。`$a -gt 0 -and $b -gt 0` のように、条件をつなぎます。	第1巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`@' … '@`	`@'` の次の行から、行の頭の `'@` までは、何行にもまたがる文字（ヒアストリング）です。中の文字は書いたとおりに扱われ、`$` も置きかわりません。スクリプトやファイルの中身を、そのまま書き込む時に使います。`'@` は、必ず行の頭に書きます。	第1巻 UNIT 01
`Set-ExecutionPolicy`	PowerShell のスクリプト（`.ps1` のファイル）を動かしてよいかの決まり（実行ポリシー）を変えるコマンド。`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` は、「自分のアカウントだけ、自分のパソコンで作ったスクリプトは動かしてよい」にします。会社のパソコンでは、決まりがある時は勝手に変えません。	第1巻 UNIT 01
`Set-ExecutionPolicy -Scope`	`-Scope CurrentUser` で、決まりを「今のユーザーだけ」に当てます（管理者の権限が要りません）。	第1巻 UNIT 01
`Get-ExecutionPolicy`	今の実行ポリシー（スクリプトを動かしてよいかの決まり）を見るコマンド。`Restricted` はスクリプトを動かせない、`RemoteSigned` は自分で作った物は動かせる、という意味です。	第1巻 UNIT 01
`Get-ExecutionPolicy -List`	`-List` で、決まりを「どの範囲（全員・自分・この窓など）に、何が決まっているか」の一覧で表示します。	第1巻 UNIT 01
`.\whereami.ps1`	今いるフォルダの `whereami.ps1`（PowerShell のスクリプト）を動かす書き方。今いるフォルダのファイルを動かす時は、頭に `.\`（「ここの」）を付ける決まりです。	第1巻 UNIT 01
`$名前`	`$os = …` のように `$` で始まる名前は「変数」（値に付けた名前）です。`=` の右の結果を覚えておき、あとで `$os` と書けば使えます。名前の大文字・小文字は区別しません。	第1巻 UNIT 01
`'…'`	`'…'`（シングルクォート）で囲んだ文字は、書いたとおりの文字で、中の `$` や `` ` `` は特別な意味になりません。正規表現（`'^\s*#'`）や、そのまま渡したい文字に使います。中に `'` を入れる時は `''` と 2 つ書きます。	第1巻 UNIT 01
`~\rust-study\whereami.ps1`	ホームフォルダ（`~` ＝ `C:\Users\you`）から数えた場所で、スクリプトを動かす書き方。どこにいても同じ書き方で動かせます。	第1巻 UNIT 01
`function`	PowerShell で、自分の関数を作る書き方。`function New-LfFile($Path, $Lines) { … }` と書くと、`New-LfFile` をコマンドのように呼べます。	第1巻 UNIT 02
`-join`	並びの中身を、区切り文字でつないで 1 つの文字列にする演算子。`$lines -join "`n" は「改行でつなぐ」、@("a","b") -join "," は "a,b"`。	第1巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`New-Item`	新しいファイルやフォルダを作るコマンド。`New-Item memo.txt` でファイル、`New-Item -ItemType Directory 名前` でフォルダを作ります。	第1巻 UNIT 02
`New-Item -Path`	`-Path 場所` で、作るファイルやフォルダの場所（名前）を指定します。	第1巻 UNIT 02
`New-Item -Value`	`-Value 中身` で、作るファイルに最初から書いておく中身を指定します。	第1巻 UNIT 02
`New-Item -Force`	`-Force` で、同じ名前のファイルがあっても上書きして作ります（無いと、エラーで止まります）。上書きするので、使う時は名前をよく確かめます。	第1巻 UNIT 02
`Out-Null`	コマンドの結果を、画面に出さずに捨てるコマンド。`… | Out-Null` のように後ろに付けると、表示が出なくなります。	第1巻 UNIT 02
`n</code> | "…" の中の `` n は改行です。PowerShell では、`\n` ではなく ` ``（バッククォート）を使います。`	第1巻 UNIT 02	
`[string[]]`	`[string[]]` は「文字列の並び（配列）」という型です。	第1巻 UNIT 02
`@( … )`	`@( … )` は「並び（配列）」を作る書き方です。`@("a", "b")` で 2 つの並び、`@()` で空の並び。中のコマンドの結果が 1 つでも、必ず並びとして扱えます。	第1巻 UNIT 02
`.\rustup-init.exe`	ダウンロードした Rust のインストーラー（`rustup-init.exe`）を、今いるフォルダから動かす書き方。	第1巻 UNIT 04
`Get-ChildItem`	フォルダの中の一覧を出すコマンド（短い呼び名は `ls`・`dir`）。`Get-ChildItem フォルダ` でそのフォルダの中身、`-Recurse` で中のフォルダまで全部を出します。	第1巻 UNIT 04
`ForEach-Object`	`|` で流れてきた物を、1 つずつ処理するコマンド。`{ }` の中の `$_` が「今の 1 つ」を表します。`… | ForEach-Object { $_.Name }` は、1 つずつ名前を取り出します。	第1巻 UNIT 04
`-f`	書式（形）を決めて文字列を作る演算子。`"{0,-10} {1}" -f $name, $hash` は、`{0}` に 1 つ目、`{1}` に 2 つ目を入れます（`,-10` は「10 文字分の幅で左に寄せる」）。	第1巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`Get-FileHash`	ファイルの中身から、ハッシュ（指紋のような長い英数字）を計算するコマンド。中身が 1 文字でも違えば、まったく違う値になります。ダウンロードしたファイルが壊れていないか確かめる時に使います。	第1巻 UNIT 04
`"{0}" -f 値`	`"{0} と {1}" -f $a, $b` の `-f` は、左の文字の `{0}`・`{1}` に、右に並べた値を、0 番目から順に入れる書き方です（書式の演算子）。`"全体 {0:N1} GB" -f 15.6` → `全体 15.6 GB`。	第1巻 UNIT 04
`{0,幅}`	`"{0,-10} {1}" -f …` の `,-10` は、幅の指定です。`-10` は「10 文字分の場所に左寄せ」、`10` なら右寄せです。名前の長さがちがっても、次の列の位置がそろいます。	第1巻 UNIT 04
`$_`	`$_` は、`|`（パイプ）で流れてきた物の「今の 1 つ」を表す、特別な変数です。`Where-Object { $_.Name -like 'a*' }` は、「流れてきた 1 つずつの `Name` が a で始まる物だけ」。`catch { … }` の中では、起きたエラーを表します。	第1巻 UNIT 04
`{ … }`	`Where-Object { … }` や `ForEach-Object { … }` の `{ }` は「スクリプトブロック」（あとで動かす、命令のかたまり）です。流れてきた 1 つずつに対して、中の命令が動き、`$_` がその 1 つを表します。	第1巻 UNIT 04
`.Name`	`$_.Name` は、流れてきた物の名前です（ファイルならファイル名、サービスならサービスの名前）。	第1巻 UNIT 04
`.FullName`	`Get-ChildItem` の結果の `.FullName` は、ファイルやフォルダの、`C:\` から始まる場所の全部です（`.Name` は名前だけ）。	第1巻 UNIT 04
`.Hash`	`Get-FileHash` の結果の `.Hash` は、ファイルの中身から計算した、指紋のような長い文字です。中身が 1 文字でもちがえば変わります。	第1巻 UNIT 04
`Get-ChildItem -Filter`	`-Filter *.exe` で、名前が合う物（ここでは `.exe` で終わる物）だけを一覧にします。`*` は「何文字でもよい」という意味です。	第1巻 UNIT 04
`.\target\debug\hello.exe`	`cargo build` が作った実行ファイルを、直接動かす書き方。	第1巻 UNIT 04
`"</code> | "…" の中に " そのものを入れる時は、`` " ` と書きます（ ""` と 2 つ重ねても同じです）。`	第1巻 UNIT 04	
`Get-ChildItem -Name`	`-Name` で、日付や大きさを出さずに、名前だけを一覧にします。	第1巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`Test-Path`	ファイルやフォルダがあるかを調べるコマンド。あれば `True`、無ければ `False` を返します。	第1巻 UNIT 05
`"…$変数…"`	`"…"`（ダブルクォート）で囲んだ文字の中の `$名前` は、変数の中身に置きかわります。`"こんにちは、$name さん"` → `こんにちは、太郎さん`。そのまま `$` を出したい時は、`'…'`（シングルクォート）で囲みます。	第1巻 UNIT 05
`&`	`& "C:\Program Files\…\vswhere.exe"` の先頭の `&` は、「この文字（や変数）が指すプログラムを動かす」という印です。場所に空白が入っていて `"` で囲んだ時や、変数に入れた場所を動かす時に必要です。	第1巻 UNIT 05
`foreach`	PowerShell のくり返し。`foreach ($path in $args) { … }` は、`$args`（スクリプトに渡された値の並び）から 1 つずつ `$path` に入れて、中を動かします。	第1巻 UNIT 05
`Resolve-Path`	相対の場所（`..\a.txt` など）を、`C:\` から始まる絶対の場所に変えるコマンド。`(Resolve-Path 場所).Path` で文字列として取り出せます。	第1巻 UNIT 05
`{0:X桁}`	`{1:X4}` の `X4` は、「16 進数の大文字で、4 けたになるまで前を 0 で埋める」指定です（`332` → `014C`）。	第1巻 UNIT 05
`$args`	`$args` は、スクリプトや関数に、名前を付けずに渡された引数が全部入った並びです。`.\x.ps1 a b` なら `$args[0]` が `a`、`$args[1]` が `b` です。	第1巻 UNIT 05
`[System.IO.File]`	`[System.IO.File]` は、ファイルを読み書きする関数が集まった .NET の型です。`[System.IO.File]::ReadAllBytes($path)` は、ファイルの中身を、バイトの並びのまま全部読みます。	第1巻 UNIT 05
`[BitConverter]`	`[BitConverter]::ToInt32($bytes, 0x3C)` は、バイトの並びの、指定した位置から 4 バイトを、整数に変えます。実行ファイルの中身を調べる時に使っています。	第1巻 UNIT 05
`[型]::名前`	`[System.IO.File]::ReadAllBytes($path)` のように、`[型の名前]::名前` と書くと、.NET（PowerShell の土台）の型に付いている関数や値を、直接使えます。	第1巻 UNIT 05
`.Path`	`(Resolve-Path $path).Path` は、相対の場所を、`C:\` から始まる場所の文字にした物です。	第1巻 UNIT 05
`.\target\x86_64-pc-windows-msvc\debug\hello.exe`	`--target x86_64-pc-windows-msvc` を付けて翻訳した時の実行ファイルを、直接動かす書き方。ほかの種類向けに翻訳した物は、`target\（種類の名前）\` の下にできます。	第1巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`Get-ChildItem -Directory`	`-Directory` で、フォルダだけを一覧にします（ファイルだけなら `-File`）。	第1巻 UNIT 05
`Get-Command`	コマンドがどこにあるか・何なのかを調べるコマンド。`(Get-Command rustc).Source` は、`rustc.exe` の場所を返します。入っているかを確かめる時に使います。	第1巻 UNIT 05
`</code> | 行の最後の `` （バッククォート。`Shift` + `@` で打ちます）は、「次の行へ続く」という印です。長いコマンドを、何行かに分けて書けます。 ` `` の後ろに空白があると続かないので、注意します。`	第1巻 UNIT 05	
`.Source`	`(Get-Command rustc).Source` は、そのコマンドの実行ファイルが置いてある場所です。どこの `rustc` が動くのかを調べられます。	第1巻 UNIT 05
`;`	`;` は、1 行にコマンドを 2 つ以上書く時の区切りです。`cargo fmt --check; $LASTEXITCODE` は、左を動かしてから右を動かします（左が失敗しても、右は動きます）。	第1巻 UNIT 06
`return`	関数やスクリプトを、そこで終える命令。`return $null` のように、返す値を書けます。	第1巻 UNIT 07
`.\target\debug\hello-topcoat.exe`	`cargo build` で作った Topcoat のアプリの実行ファイルを、直接動かす書き方。サーバーなので、止めるまで動き続けます（止める時は `Ctrl` + `C`）。	第1巻 UNIT 08
`Invoke-RestMethod`	URL にアクセスして、返ってきた JSON などを PowerShell で扱える形にして受け取るコマンド。API を試す時に使います（短い呼び名は `irm`）。	第1巻 UNIT 09
`Invoke-RestMethod -Method`	`-Method Post` で、送る方法（GET・POST など）を指定します。指定しないと GET です。	第1巻 UNIT 09
`Invoke-RestMethod -Uri`	`-Uri URL` で、アクセスする先の URL を指定します。	第1巻 UNIT 09
`Invoke-RestMethod -Body`	`-Body 中身` で、送る中身（JSON の文字列など）を指定します。	第1巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`Invoke-RestMethod -ContentType`	`-ContentType application/json` で、送る中身の種類（ここではJSON）をサーバーに伝えます。	第1巻 UNIT 09
`ConvertTo-Json`	PowerShell の値（オブジェクト）を、JSON の文字列に変えるコマンド。受け取った結果を JSON の形で見たい時にも使います。	第1巻 UNIT 09
`ConvertTo-Json -Depth`	`-Depth 5` で、中に入れ子になった物を何段目まで JSON にするかを決めます（既定は 2 段で、それより深い所は省かれます）。	第1巻 UNIT 09
`Where-Object`	`|` で流れてきた物のうち、条件に合う物だけを残すコマンド。`{ }` の中の `$_` が「今の 1 つ」です。`Where-Object { $_.Length -gt 1000 }` のように使います。	第1巻 UNIT 09
`-like`	`*`（何文字でも）と `?`（1 文字）を使った、ワイルドカードで合うかの比べ方。`$_ -like '127*'` は「127 で始まるか」。	第1巻 UNIT 09
`Get-Item`	ファイルやフォルダ 1 つの情報（名前・大きさ・日時）を取り出すコマンド。	第1巻 UNIT 09
`Get-Item -Stream`	`-Stream *` で、ファイルに付いている「隠れた別の中身」（代替データストリーム。ダウンロードした印などが入る）の一覧を出します。	第1巻 UNIT 09
`Format-Table`	結果を表の形に整えて表示するコマンド。	第1巻 UNIT 09
`Format-Table -AutoSize`	`-AutoSize` で、列の幅を中身に合わせて自動で決めます（長い値が `...` で切られにくくなります）。	第1巻 UNIT 09
`*`	`'127*'` や `*.log` の `*` は、「何文字でも、何でも合う」という印（ワイルドカード）です。`-like '127*'` は「127 で始まる」、`-Filter *.log` は「名前が .log で終わる物」。1 文字だけなら `?` です。	第1巻 UNIT 09
`pwsh`	PowerShell 7（新しい PowerShell）を動かすコマンド。Windows に最初から入っているのは Windows PowerShell 5.1（`powershell`）です。`pwsh -Command "…"` で、1 つの命令を PowerShell 7 で動かします。	第1巻 UNIT 10
`pwsh -Command`	`pwsh -Command "…"` は、新しい PowerShell（7）を起動して、後ろの文字を命令として動かします。	第1巻 UNIT 10
`Write-Host`	画面に文字を表示するコマンド。`Write-Output` と違い、次のコマンドへは流れず、画面に出すだけです。色を付ける（`-ForegroundColor Green`）こともできます。	第1巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`-not`	「〜でない」（`True` と `False` を逆にする）。`-not (Test-Path $file)` は「そのファイルが無いなら」。`!` と書いても同じです。	第1巻 UNIT 10
`try`	PowerShell で、失敗するかもしれない処理を `try { … } catch { … }` で囲み、失敗した時の処理を `catch` に書く書き方。	第1巻 UNIT 10
`-ne`	PowerShell の「等しくない」の比べ方（not equal）。`$LASTEXITCODE -ne 0` は「失敗した（0 でない）か」。	第1巻 UNIT 10
`-or`	「または」（どちらか一方でも `True` なら `True`）。`$LASTEXITCODE -ne 0 -or $out.Count -eq 0` は「失敗したか、出力が空か」。	第1巻 UNIT 10
`-eq`	PowerShell の「等しい」の比べ方（equal）。`$LASTEXITCODE -eq 0` は「終わった時の番号が 0（成功）か」。PowerShell では `==` ではなく `-eq` と書きます。大文字・小文字は区別しません（`"ABC" -eq "abc"` も `True`）。	第1巻 UNIT 10
`catch`	`try { … } catch { … }` の `catch` の所は、`try` の中でエラーが起きた時だけ動きます。起きたエラーは `$_` で受け取れます（`$_.Exception.Message` で説明の文）。	第1巻 UNIT 10
`-ge`	PowerShell の「以上」の比べ方（greater or equal）。`$code.Count -ge 3` は「3 つ以上あるか」。	第1巻 UNIT 10
`-split`	文字列を、区切り（正規表現）で分けて並びにする演算子。`"a=b" -split '=', 2` は `"a"` と `"b"`（`2` は「多くても 2 つに分ける」）。	第1巻 UNIT 10
`-gt`	PowerShell の「より大きい」の比べ方（greater than）。`$failed -gt 0` は「失敗が 1 つ以上あるか」。	第1巻 UNIT 10
`exit`	スクリプトを終えるコマンド。`exit 1` は、終了コード 1（失敗）でスクリプトを終えます。	第1巻 UNIT 10
`$( … )`	`"…$($t.Port)…"` の `$( )` は、「中の式やコマンドの結果を、文字の中に入れる」書き方です。`"$x.Name"` のように書くと `$x` だけが置きかわってしまうので、`.` の付いた物や計算は `$( )` で囲みます。	第1巻 UNIT 10
`$script:名前`	`$script:failed` の `script:` は、「関数の中からでも、このスクリプト全体の変数を使う（変える）」という印です。付けないと、関数の中で変えても、外の変数は変わりません。	第1巻 UNIT 10
`$null`	`$null` は「何も無い」を表す値です。`if ($x -eq $null)` で「何も入っていないか」を調べられます。`$null = コマンド` や `コマンド | Out-Null` と書くと、コマンドの出力を捨てられます。	第1巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`$true`	`$true` は「はい（真）」を表す値です。`-Recurse:$true` のように、スイッチに値を渡す時にも使います。反対は `$false` です。	第1巻 UNIT 10
`[Console]`	`[Console]` は、PowerShell の窓（コンソール）を表す .NET の型です。`[Console]::OutputEncoding = [Text.Encoding]::UTF8` は、ほかのプログラムが出す文字を UTF-8 として読む設定で、文字化けを防ぎます。	第1巻 UNIT 10
`[Text.Encoding]`	`[Text.Encoding]::UTF8` は、文字の符号（エンコーディング）の UTF-8 を表す値です。	第1巻 UNIT 10
`[string]`	`[string]$Name` の `[string]` は、「この変数（引数）は文字列」という型の指定です。`[string]123` のように値の前に書くと、文字列に変えます。	第1巻 UNIT 10
`[bool]`	`[bool]` は「はい・いいえ」（`$true`・`$false`）の型です。`[bool]$x` は、`$x` に何か入っていれば `$true`、空や `$null` なら `$false` にします。	第1巻 UNIT 10
`[scriptblock]`	`[scriptblock]` は「あとで動かす命令のかたまり（`{ … }`）」の型です。`& $Command` で、受け取ったかたまりを動かします。	第1巻 UNIT 10
`2>$null`	`2>$null` は、エラーの出力を捨てる指定です。「無くてもかまわない」物を調べる時に、エラーの文で画面がうまらないようにします。	第1巻 UNIT 10
`1..10`	`1..10` は、1 から 10 までの数の並び（10 も入る）を作ります。`foreach ($i in 1..3) { … }` で 3 回くり返せます。	第1巻 UNIT 10
`$並び[0]`	`$list[0]` は、並びの 0 番目（最初の 1 つ）を取り出します。`$list[-1]` は最後の 1 つです。	第1巻 UNIT 10
`.Count`	`$list.Count` は、並びの中の数です。`0` なら空です。	第1巻 UNIT 10
`.Major`	版（`[version]`）の `.Major` は、一番上の数です。`5.1` なら `5`。	第1巻 UNIT 10
`powershell`	Windows PowerShell 5.1 を動かすコマンド。`powershell -ExecutionPolicy Bypass -File スクリプト.ps1` は、実行ポリシーをその 1 回だけ気にせずに、スクリプトを動かします。	第1巻 UNIT 10
`powershell -ExecutionPolicy`	`powershell -ExecutionPolicy Bypass -File x.ps1` の `-ExecutionPolicy Bypass` は、この 1 回だけ、スクリプトを動かす制限を外す指定です。パソコンの設定は変えません。	第1巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`powershell -File`	`-File x.ps1` は、指定したスクリプトのファイルを動かします。後ろに書いた物は、スクリプトへの引数になります。	第1巻 UNIT 10
`$HOME`	`$HOME` は、自分のホームフォルダ（`C:\Users\you`）の場所が入った、特別な変数です。`~` と同じ場所です。	第1巻 UNIT 10
`[version]`	`[version]"1.99.1"` は、文字を「版」の値に変えます。版どうしは `-ge`（以上）などで、正しく大きさを比べられます（文字のままだと `"1.10"` が `"1.9"` より小さいことになります）。	第1巻 UNIT 10
`New-Item -ItemType`	`-ItemType Directory` で、ファイルではなくフォルダを作ります（`File` ならファイル）。`New-Item -ItemType Directory src\bin` は、`src` の中に `bin` フォルダを作ります。	第6巻 この巻で身に付けること
`Copy-Item -Recurse`	`-Recurse` で、フォルダを中身ごと（中のフォルダも全部）コピーします。付けないと、フォルダの中身はコピーされません。	第8巻 この巻で身に付けること
`.\target\debug\rustnote.exe`	`cargo build` で作った RustNote の実行ファイルを、直接動かす書き方。	第11巻 UNIT 07
`.\target\release\rustnote.exe`	`cargo build --release` で作った、最適化した（速い）RustNote の実行ファイルを、直接動かす書き方。	第11巻 UNIT 09
`.\rustnote.exe`	今いるフォルダの `rustnote.exe` を動かす書き方。	第12巻 UNIT 01
`t</code> | "…" の中の `` t `` はタブ（広めの空白のような区切りの文字）です。`	第12巻 UNIT 02	
`param`	PowerShell のスクリプトが受け取る引数を決める書き方。`param([string]$Base = "http://…")` は、`-Base` で渡せる文字列の引数で、渡さなければ既定値になります。	第12巻 UNIT 08
`New-TemporaryFile`	一時的に使う、空のファイルを作るコマンド（作ったファイルの情報を返す）。テストでクッキーをためておく場所などに使い、使い終わったら `Remove-Item` で消します。	第12巻 UNIT 08
`$ErrorActionPreference`	`$ErrorActionPreference = "Stop"` は、「エラーが起きたら、そこでスクリプトを止める」という設定です。付けないと、エラーが起きても次の行へ進んでしまいます（`.exe` のコマンドの失敗は止まらないので、`$LASTEXITCODE` も調べます）。	第12巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`@名前`	`コマンド @CurlArgs` のように、`$` の代わりに `@` を付けて変数を渡すと、変数に入れた並び（や表）の中身を、引数としてばらして渡せます（スプラッティング）。	第12巻 UNIT 08
`param( … )`	スクリプトの最初に書く `param([string]$Base = "http://…")` は、「このスクリプトが受け取る引数」の決まりです。`.\smoke.ps1 -Base http://…` のように、名前を付けて渡せます。`=` の右は、渡さなかった時の値です。	第12巻 UNIT 08
`powershell -NoProfile`	`-NoProfile` は、プロフィール（起動するたびに動く、自分用の設定のスクリプト）を読まずに起動する指定です。どのパソコンでも同じ動きになります。	第12巻 UNIT 08
`Add-Content`	ファイルの後ろに書き足すコマンド。`Add-Content memo.txt "2 行目" -Encoding utf8`。中身を置きかえるのは `Set-Content` です。	第14巻 A. 道具の早見表
`Add-Content -Encoding`	`-Encoding utf8` で、書き足す文字を UTF-8 で書きます（日本語を書く時は付けておきます）。	第14巻 A. 道具の早見表
`Get-Content -Encoding`	`-Encoding utf8` で、ファイルを UTF-8 として読みます。付けないと、Windows PowerShell 5.1 では日本語が化けることがあります。	第14巻 A. 道具の早見表
`Rename-Item`	ファイルやフォルダの名前を変えるコマンド。`Rename-Item 今の名前 新しい名前`。	第14巻 A. 道具の早見表
`Move-Item`	ファイルやフォルダを、別の場所へ移すコマンド（短い呼び名は `mv`）。`Move-Item ファイル 先のフォルダ\`。	第14巻 A. 道具の早見表
`Get-ChildItem -Recurse`	`-Recurse` で、中のフォルダの中まで全部を一覧にします。	第14巻 A. 道具の早見表
`Select-String`	ファイルの中から、文字を含む行を探すコマンド。何行目に見つかったかも表示します。	第14巻 A. 道具の早見表
`Select-String -Path`	`-Path ファイル` で、探すファイルを指定します。`*.log` のように、いくつものファイルも指定できます。	第14巻 A. 道具の早見表
`Select-String -Pattern`	`-Pattern "文字"` で、探す文字を指定します。`"ERROR|WARN"` のように、`|` で「または」にもできます（正規表現）。	第14巻 A. 道具の早見表
`Select-String -Encoding`	`-Encoding utf8` で、ファイルを UTF-8 として読んで探します。	第14巻 A. 道具の早見表
`Remove-Item -Recurse`	フォルダを、中身ごと全部消すオプション。消した物は「ごみ箱」に入らず、元に戻せないので、場所をよく確かめてから使います。	第14巻 A. 道具の早見表

コマンド

コード	意味と使い方	初めて出てくる所
`curl.exe`	URL にアクセスして、返ってきた中身を表示するコマンド（Windows に最初から入っています）。`curl.exe http://127.0.0.1:8080/` のように使います。PowerShell では `curl` だけだと別の命令（`Invoke-WebRequest`）になるので、`.exe` まで付けて打ちます。`-i` で返事の見出し（ヘッダー）も表示します。	第0巻 UNIT 05
`netstat`	ネットワークの接続の一覧を出す、昔からあるコマンド。`netstat -ano` で、ポート番号と、使っているプロセスの番号（PID）が分かります。	第0巻 UNIT 06
`netstat -ano`	`netstat -ano` は、つながりと待ち受けを全部（`-a`）、名前でなく番号で（`-n`）、プログラムの番号（PID、`-o`）つきで表示します。`| findstr :8080` で、8080 番を使っている物だけにしぼれます。	第0巻 UNIT 06
`findstr`	文字を含む行だけを取り出すコマンド（Windows に最初から入っています）。`netstat -ano | findstr :8080` は、`:8080` を含む行だけを表示します。PowerShell の `Select-String` に近い働きです。	第0巻 UNIT 06
`where.exe`	コマンドの本体（`.exe`）が、パソコンのどこにあるかを探すコマンド。`where.exe git` で、`git.exe` の場所が分かります。PowerShell では `where` だけだと別の命令になるので、`.exe` まで付けて打ちます。	第1巻 UNIT 02
`echo`	後ろに書いた文字を、そのまま画面に出すコマンド。`echo "hello"`。bash（WSL）でも PowerShell でも使えます。	第1巻 UNIT 02
`pwd`	今いるフォルダを表示するコマンド（print working directory）。bash（WSL）では `/home/you` のような Linux の形で、PowerShell では `C:\Users\you` のような形で出ます。	第1巻 UNIT 02
`code`	VS Code を動かすコマンド。`code .` で今のフォルダを開き、`code --version` で版を確かめます。	第1巻 UNIT 03
`code --version`	`code --version` は、VS Code の版を表示します。	第1巻 UNIT 03
`code --install-extension`	`code --install-extension rust-lang.rust-analyzer` は、VS Code に拡張機能を入れます。	第1巻 UNIT 03
`code --list-extensions`	`--list-extensions` は、VS Code に入っている拡張機能の一覧を表示します。	第1巻 UNIT 03
`code --show-versions`	`--show-versions` を `--list-extensions` と一緒に付けると、拡張機能の版も表示します。	第1巻 UNIT 03

WSL・bash

コード	意味と使い方	初めて出てくる所
`#`	bash では、`#` から行の終わりまでがコメントです（1 行目の `#!` だけは特別です）。	第0巻 UNIT 03
`set`	bash の設定を変える命令。`set -e` は「途中のコマンドが失敗したら、そこでスクリプトを止める」。	第1巻 UNIT 02
`set -e`	スクリプトの最初に書く `set -e` は、「どれかのコマンドが失敗したら、そこでスクリプトを止める」という指定です。失敗に気づかずに先へ進むのを防ぎます。	第1巻 UNIT 02
`bash`	Linux でよく使われるシェル（文字で命令する画面）。`bash ファイル.sh` で、bash のスクリプトを動かします。WSL の中や、Git for Windows に付いています。	第1巻 UNIT 02
`cat -v`	`cat -v` は、ファイルの中の目に見えない文字（CR など）を、`^M` のような印にして表示します。改行が CRLF かどうかを調べる時に使います。	第1巻 UNIT 02
`|`	bash の `|`（パイプ）は、左のコマンドの出力（文字）を、右のコマンドの入力に渡す記号です。`bash crlf.sh 2>&1 | cat -v`。	第1巻 UNIT 02
`2>&1`	`2>&1` は、エラーの出力（2 番）を、ふつうの出力（1 番）と同じ所へ流す指定です。エラーの文も、パイプやファイルに一緒に渡せます。	第1巻 UNIT 02
`sed`	文字を置きかえるコマンド（Linux）。`sed -i 's/\r$//' ファイル` は、各行の最後の `\r`（Windows の改行の印）を消して、ファイルを書きかえます（`-i` は「そのファイルを直接書きかえる」）。	第1巻 UNIT 02
`sed -i`	`sed -i 's/\r$//' crlf.sh` の `-i` は、ファイルを直接書きかえる指定です。`s/前/後/` は「前を後に置きかえる」で、ここでは各行の最後の `\r`（CR）を消します。	第1巻 UNIT 02
`'…'`	bash の `'…'`（シングルクォート）で囲んだ文字は、そのままの文字で、`$` や `\` も特別な意味になりません。`sed` の命令などを、そのまま渡す時に使います。	第1巻 UNIT 02
`wsl`	WSL（Windows の中で Linux を動かす仕組み）を操作するコマンド。`wsl --install` で入れ、`wsl` だけで Linux の画面（bash）に入ります。	第1巻 UNIT 07
`wsl --install`	`wsl --install` は、WSL（Windows の中で Linux を動かす仕組み）と、Ubuntu を入れます。管理者の PowerShell で打ち、終わったらパソコンを再起動します。	第1巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`wsl --version`	`wsl --version` は、WSL の版を表示します。	第1巻 UNIT 07
`wsl -l`	`wsl -l -v` は、入っている Linux（ディストリビューション）の一覧を、WSL の版（1 か 2）と、動いているか（Running / Stopped）つきで表示します。	第1巻 UNIT 07
`wsl -v`	`-v`（verbose）は、`wsl -l` と一緒に使い、一覧をくわしく（状態と版つきで）表示する指定です。	第1巻 UNIT 07
`wsl -d`	`wsl -d Ubuntu -- uname -m` の `-d Ubuntu` は、どの Linux で動かすかを指定します。`--` の後ろが、その Linux で動かすコマンドです。	第1巻 UNIT 07
`rm`	ファイルやフォルダを消すコマンド（Linux）。`rm -rf フォルダ` は、中身ごと、確かめずに消します（`-r` は中まで、`-f` は聞かずに）。とても強いので、名前をよく確かめて使います。	第1巻 UNIT 07
`rm -rf`	`rm -rf フォルダ` は、フォルダを中身ごと（`-r`）、確かめずに（`-f`）消します。消した物は、ごみ箱にも入らず戻せないので、場所をよく確かめてから打ちます。	第1巻 UNIT 07
`mkdir -p`	`mkdir -p a/b/c` は、途中のフォルダもまとめて作ります。もうあってもエラーにしません。	第1巻 UNIT 07
`s=$`	bash で変数に値を入れる書き方。`s=$(date +%s%N)` は、`$( )` の中のコマンド（今の時刻）を動かした結果を、変数 `s` に入れます。`=` の前後に空白を入れてはいけません。	第1巻 UNIT 07
`date`	今の日時を表示するコマンド（Linux）。`date +%s%N` は「1970 年からの秒数＋ナノ秒」の形で出します（時間を測るのに使います）。	第1巻 UNIT 07
`for`	bash のくり返し。`for i in $(seq 1 2000); do … ; done` は、1〜2000 を `i` に入れて、`do` から `done` までをくり返します。	第1巻 UNIT 07
`seq`	数の並びを出すコマンド（Linux）。`seq 1 2000` は 1 から 2000 まで、1 行に 1 つずつ出します。	第1巻 UNIT 07
`do`	bash のくり返し `for 変数 in 並び; do … ; done` の一部。`do` から `done` までが、くり返す中身です。	第1巻 UNIT 07
`done`	bash のくり返し（`for … do … done`）の終わりの印。	第1巻 UNIT 07
`e=$`	`s=$(…)` と同じく、bash で変数に値を入れる書き方。ここでは終わりの時刻を `e` に入れ、`s` との差で、かかった時間を計算します。	第1巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`$( … )`	`$(seq 1 2000)` の `$( )` は、「中のコマンドを動かして、その出力をここに入れる」書き方です。	第1巻 UNIT 07
`$名前`	bash の `$名前` は、変数の中身です。`name=web` で入れて、`echo $name` で使います（入れる時は `$` を付けず、`=` の前後に空白を入れません）。	第1巻 UNIT 07
`$1`	スクリプトの中の `$1` は、スクリプトに渡した 1 つ目の引数です（`$2` は 2 つ目、`$@` は全部）。	第1巻 UNIT 07
`>`	`echo x > f1` の `>` は、出力を、画面ではなくファイルに書き込む指定です（前の中身は消えます）。	第1巻 UNIT 07
`&&`	`a && b` は、「a が成功したら、b を動かす」です。失敗したら、そこで止まります。	第1巻 UNIT 07
`||`	`a || b` は、「a が失敗したら、b を動かす」です。`cd "$1/many" || return` は、「移れなければ抜ける」。	第1巻 UNIT 07
`;`	`;` は、1 行にコマンドを並べる区切りです。左が終わったら、成功しても失敗しても、右を動かします。	第1巻 UNIT 07
`"…$変数…"`	bash の `"…"`（ダブルクォート）の中の `$名前` は、変数の中身に置きかわります。空白の入った値でも 1 つとして渡せるように、変数は `"$1"` のように `"` で囲んで使います。	第1巻 UNIT 07
`curl`	URL にアクセスするコマンド（Linux・WSL 版）。`curl -sSfL URL | bash` は、URL から取ってきたスクリプトを、そのまま bash で動かします（`-s` 静かに・`-S` エラーは出す・`-f` 失敗を見逃さない・`-L` 移動先を追う）。中身を確かめずに動かすことになるので、信頼できる所の物だけに使います。	第1巻 UNIT 09
`curl -sSfL`	`-sSfL` は 4 つの指定を合わせた物です。`-s` は進み具合を出さない、`-S` はそれでもエラーは表示する、`-f` はサーバーがエラー（404 など）を返したら失敗にする、`-L` は移動先（リダイレクト）についていく、です。`curl -sSfL URL | bash` は、URL のスクリプトを取ってきて、すぐ動かします（信用できる所の物だけにします）。	第1巻 UNIT 09
`apt-get`	Debian・Ubuntu の部品（パッケージ）を入れる・新しくする道具です。`apt-get update` で部品の一覧を新しくし、`apt-get install -y 名前` で入れます。	第12巻 UNIT 02
`apt-get -y`	`-y`（yes）は、「入れますか？」の質問に全部「はい」と答えて進める指定です。Dockerfile のように、人が答えられない所で使います。	第12巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`apt-get --no-install-recommends`	`--no-install-recommends` は、「おすすめ」の部品は入れず、どうしても要る物だけを入れる指定です。コンテナを小さくできます。	第12巻 UNIT 02
`useradd`	Linux に、新しい利用者（ユーザー）を作るコマンドです。コンテナの中で、管理者（root）ではない利用者でアプリを動かすために使います。	第12巻 UNIT 02
`useradd --create-home`	`--create-home` は、利用者を作る時に、その人のホームフォルダ（`/home/名前`）も作る指定です。	第12巻 UNIT 02
`useradd --uid`	`--uid 10001` は、利用者の番号（UID）を決める指定です。番号を決めておくと、フォルダの持ち主の設定がそろいます。	第12巻 UNIT 02
`chown`	ファイルやフォルダの持ち主を変えるコマンドです。`chown app:app /data` は、`/data` の持ち主を、利用者 `app`・グループ `app` にします。	第12巻 UNIT 02

Git

コード	意味と使い方	初めて出てくる所
`git`	ファイルの変更の記録を残す道具（Git）の本体のコマンド。`git --version` で版を確かめられます。`git status`・`git add`・`git commit` のように、後ろに「何をするか」を付けて使います（この巻の UNIT 04）。	第1巻 UNIT 02
`git --version`	`git --version` は、入っている Git の版を表示します。	第1巻 UNIT 02
`git config`	Git の設定を見る・変えるコマンド。`git config --global user.name "名前"` で、記録に残す名前を決めます。`--list` で、今の設定を一覧で見られます。	第1巻 UNIT 02
`git config --system`	`--system` は、パソコン全体（Git を入れたフォルダの設定ファイル）の設定を対象にする指定です。	第1巻 UNIT 02
`git config --list`	`--list` は、今効いている設定を全部一覧にします。	第1巻 UNIT 02
`git config --show-origin`	`--show-origin` は、それぞれの設定が、どのファイルに書かれているかも一緒に表示します。同じ設定がいくつもの場所にある時、どれが効いているかを調べられます。	第1巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`git config --get`	`git config --get core.autocrlf` は、1 つの設定の値だけを表示します。	第1巻 UNIT 02
`git config --global`	`--global` は、自分（このユーザー）の、全部のリポジトリに効く設定を対象にする指定です。`git config --global user.name "名前"` のように、最初に 1 回だけ設定します。	第1巻 UNIT 02
`git init`	今いるフォルダで、Git の記録を始めるコマンド。フォルダの中に、記録を入れる `.git` という隠しフォルダができます（`cargo new` は、これを自動でしてくれます）。	第1巻 UNIT 02
`git status`	今の状態（変わったファイル・記録に入れる予定のファイル・Git がまだ知らないファイル）を表示します。迷ったら、まずこれを打ちます。	第1巻 UNIT 02
`git check-ignore`	そのファイルが `.gitignore`（記録しないファイルの一覧）で外されているかを調べるコマンド。`-v` を付けると、`.gitignore` の何行目の決まりで外されたかも表示します。	第1巻 UNIT 02
`git check-ignore -v`	`git check-ignore -v .env` は、そのファイルが Git に無視されるかを調べ、`-v` で「`.gitignore` の何行目のどの決まりで無視されるか」まで表示します。	第1巻 UNIT 02
`git add`	次の記録（コミット）に入れるファイルを選ぶコマンド。`git add ファイル名`、全部なら `git add .` です。選んだだけで、まだ記録はされていません。	第1巻 UNIT 02
`git ls-files`	Git が記録しているファイルの一覧を出すコマンド。`--eol` を付けると、各ファイルの改行の形（LF か CRLF か）も表示します。	第1巻 UNIT 02
`git ls-files --eol`	`--eol` は、Git に入っているファイルの改行（LF か CRLF か）を、リポジトリの中（`i/`）と作業フォルダ（`w/`）の両方について表示します。	第1巻 UNIT 02
`git commit`	選んだ変更を、1 つの記録（コミット）として残すコマンド。`git commit -m "何をしたか"` の `-m` の後ろに、変更の説明を書きます。	第1巻 UNIT 02
`git commit -m`	`git commit -m "説明"` の `-m` は、コミットの説明文を、その場で書く指定です。付けないと、説明を書くための画面（エディター）が開きます。	第1巻 UNIT 02
`git log`	これまでの記録（コミット）の一覧を、新しい順に出すコマンド。`--oneline` で 1 行ずつ短く、`--format="…"` で表示の形を自分で決められます。	第1巻 UNIT 02
`git log --format`	`--format="%h %an <%ae> %s"` は、1 件ずつの表示の形を決めます。`%h` は短い番号、`%an` は作った人の名前、`%ae` はメール、`%s` は説明の 1 行目です。	第1巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`git rm`	ファイルを、Git の記録から外すコマンド。`git rm --cached .env` は、ファイルそのものは残したまま、記録からだけ外します（間違えて記録した秘密のファイルを外す時に使います）。	第1巻 UNIT 02
`git rm --cached`	`git rm --cached .env` は、ファイルは消さずに、Git の管理からだけ外します。うっかり入れてしまった秘密のファイルを、次のコミットから外す時に使います。	第1巻 UNIT 02
`git status --short`	`--short` は、変わったファイルを短い形で表示します。`M` は変更、`??` はまだ Git が知らないファイル、`A` は追加です。	第1巻 UNIT 02
`git log --oneline`	`--oneline` は、1 件のコミットを 1 行（短い番号と、説明の 1 行目）で表示します。	第1巻 UNIT 02
`git show`	記録の中身を見るコマンド。`git show HEAD~1:.env` は「1 つ前の記録の時の `.env` の中身」を表示します。一度記録した物は、こうして後からでも見えてしまう、という例です。	第1巻 UNIT 02
`git diff`	まだ記録していない変更の中身を見るコマンド。`-` で始まる行が前の内容、`+` で始まる行が新しい内容です。	第1巻 UNIT 03
`git restore`	保存したけれど記録していない変更を捨てて、最後の記録の状態に戻すコマンド。`git restore src/main.rs`。捨てた変更は戻せないので、確かめてから使います。	第1巻 UNIT 03

GitHub Actions

コード	意味と使い方	初めて出てくる所
`-`	YAML の行の頭の `-` は、「並びの 1 つ」を表します。`steps:` の下の `- name: …` は、手順の 1 つ目・2 つ目…です。字下げ（空白の数）で、どこに入るかが決まるので、そろえて書きます。	第1巻 UNIT 03
`name:`	`name:` は、名前です。いちばん上の `name: CI` は、自動の確かめ（ワークフロー）の名前、`- name: Clippy…` は、手順 1 つの名前で、GitHub の画面に表示されます。	第1巻 UNIT 03
`run:`	`run: cargo clippy …` は、その手順で動かすコマンドです。`run: |` と書くと、次の行から何行も書けます。	第1巻 UNIT 03
`#`	YAML（GitHub Actions の設定などの書き方）では、`#` から行の終わりまでがコメントです。	第11巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`on:`	`on:` は、「いつ動かすか」です。`push:`（送った時）・`pull_request:`（プルリクエストの時）・`schedule:`（決まった時刻）などを並べます。	第11巻 UNIT 10
`push:`	`push:` は、「リポジトリに送った（push）時に動かす」指定です。	第11巻 UNIT 10
`branches:`	`branches: [main]` は、「`main` ブランチに送った時だけ」としぼる指定です。	第11巻 UNIT 10
`[ … ]`	YAML の `[main]` のように `[ ]` で囲むと、並び（いくつかの値）を 1 行で書けます。	第11巻 UNIT 10
`pull_request:`	`pull_request:` は、「プルリクエスト（変更を取り込んでほしいというお願い）が作られた・更新された時に動かす」指定です。	第11巻 UNIT 10
`jobs:`	`jobs:` の下に、動かす仕事（ジョブ）を並べます。ジョブは、ふつう同時に動きます。	第11巻 UNIT 10
`test:`	`jobs:` の下の `test:` は、仕事（ジョブ）に付けた名前です（自分で決めます）。	第11巻 UNIT 10
`runs-on:`	`runs-on: ubuntu-26.04` は、その仕事を、どの OS の機械で動かすかです。GitHub が用意した機械を、その時だけ借ります。	第11巻 UNIT 10
`defaults:`	`defaults:` は、その仕事の全部の手順に共通の設定です。	第11巻 UNIT 10
`working-directory:`	`working-directory: Rust` は、コマンドを動かすフォルダです。リポジトリの一番上ではなく、その中のフォルダで動かす時に書きます。	第11巻 UNIT 10
`steps:`	`steps:` の下に、その仕事の手順を、上から順に `-` で並べます。	第11巻 UNIT 10
`uses:`	`uses: actions/checkout@v7` は、ほかの人が作った決まった手順（アクション）を使う指定です。`actions/checkout` は、リポジトリのファイルを取ってくる手順、`@v7` はその版です。	第11巻 UNIT 10
`with:`	`with:` は、`uses:` で使うアクションに渡す設定です。	第11巻 UNIT 10
`toolchain:`	Rust を入れるアクションの `toolchain: 1.99.1` は、入れる Rust の版です。	第11巻 UNIT 10
`components:`	`components: rustfmt, clippy` は、Rust と一緒に入れる部品です。	第11巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`workspaces:`	`workspaces: Rust` は、翻訳した物を取っておく（キャッシュ）アクションに、どのフォルダの `target` を取っておくかを伝える設定です。	第11巻 UNIT 10
`docker:`	`jobs:` の下の `docker:` は、仕事（ジョブ）に付けた名前です（自分で決めます）。ここでは、イメージを作れるかを確かめる仕事です。	第11巻 UNIT 10
`needs:`	`needs: test` は、「`test` の仕事が成功してから動かす」という順番の指定です。	第11巻 UNIT 10
`schedule:`	`schedule:` は、「決まった時刻に動かす」指定です。下に `- cron: "…"` で時刻を書きます。	第11巻 UNIT 10
`cron:`	`cron: "0 0 * * *"` は、動かす時刻の書き方（cron）です。5 つの欄は左から「分 時 日 月 曜日」で、`*` は「毎」。`0 0 * * *` は「毎日 0 時 0 分」です。時刻は UTC（日本は 9 時間進んでいるので、日本の 9 時）です。	第11巻 UNIT 10
`|`	`run: |` の `|` は、「次の行から、字下げした所までを、改行も含めてそのまま 1 つの文字にする」という印です。コマンドを何行も書く時に使います。	第11巻 UNIT 10

Docker

コード	意味と使い方	初めて出てくる所
`docker version`	Docker（アプリを、決まった環境ごと箱に入れて動かす道具）の版を表示します。表示されれば、Docker が動いています。	第1巻 UNIT 07
`docker version --format`	`--format 'Client: {{.Client.Version}}'` は、表示の形を、`{{ }}` で値を入れる型紙（テンプレート）で決めます。必要な値だけを取り出せます。	第1巻 UNIT 07
`docker run`	イメージ（箱の設計図）から、コンテナ（動いている箱）を作って動かすコマンド。`docker run --rm hello-world` は、試しの箱を動かして、終わったら消します（`--rm`）。	第1巻 UNIT 07
`docker run --rm`	`--rm` は、コンテナが止まったら、自動でコンテナを消す指定です。試しに 1 回動かすだけの時に付けると、止まったコンテナが残りません。	第1巻 UNIT 07
`docker run --platform`	`--platform linux/amd64` は、どの種類の CPU 向けのイメージで動かすかを指定します。	第1巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`docker image`	手元にあるイメージ（箱の設計図）を見る・消すコマンド。`docker image ls` で一覧を出します。	第1巻 UNIT 07
`docker image ls --format`	`--format` は、イメージの一覧の表示の形を、`{{.Repository}}` のような型紙で決めます。	第1巻 UNIT 07
`docker run -d`	`-d`（detach）は、コンテナを裏で動かし、すぐに PowerShell に戻る指定です。サーバーのように、ずっと動かす物に付けます。	第1巻 UNIT 07
`docker run --name`	`--name u07a` は、コンテナに名前を付けます。あとで `docker stop u07a`・`docker logs u07a` のように、名前で操作できます。	第1巻 UNIT 07
`docker ps`	動いているコンテナ（箱）の一覧を出すコマンド。`--filter name=…` で名前で絞り、`--format` で表示する項目を選べます。	第1巻 UNIT 09
`docker ps --filter`	`--filter name=u09sqld` は、動いているコンテナの一覧を、条件でしぼります。	第1巻 UNIT 09
`docker ps --format`	`--format "{{.Names}} {{.Status}}"` は、コンテナの一覧の表示の形を決めます。	第1巻 UNIT 09
`docker build`	`Dockerfile`（箱の作り方を書いたファイル）から、イメージ（箱の設計図）を作るコマンド。`-t 名前:タグ` で名前を付けます。最後の `.` は「今のフォルダの中身を使う」という意味です。	第11巻 UNIT 10
`docker build -t`	`docker build -t rustnote:u02 .` の `-t` は、作ったイメージに名前と札（`名前:札`）を付ける指定です。最後の `.` は「今いるフォルダの Dockerfile で作る」という意味です。	第11巻 UNIT 10
`#`	Dockerfile では、`#` で始まる行がコメントです。	第12巻 UNIT 01
`FROM`	`FROM rust:1.99.1-trixie` は、「どのイメージ（ひな形）から作り始めるか」です。Dockerfile は、いつも `FROM` から始めます。	第12巻 UNIT 01
`FROM … AS 名前`	`FROM … AS builder` の `AS builder` は、その段に名前を付ける書き方です。1 つの Dockerfile に段をいくつも書き（multi-stage）、後の段で `COPY --from=builder` と書いて、前の段で作った物だけを持ってこられます。作るための道具を、最後のイメージに入れずに済みます。	第12巻 UNIT 01
`WORKDIR`	`WORKDIR /app` は、イメージの中の「今いるフォルダ」を決めます（無ければ作ります）。後の `COPY`・`RUN` は、このフォルダで動きます。	第12巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`RUN`	`RUN コマンド` は、イメージを作る途中で、そのコマンドを動かします（部品を入れる・翻訳する、など）。結果は、イメージの 1 つの層として残ります。	第12巻 UNIT 01
`COPY`	`COPY . .` は、パソコンのファイル（左。`.` は Dockerfile のあるフォルダ）を、イメージの中（右。`.` は `WORKDIR`）にコピーします。	第12巻 UNIT 01
`\`	Dockerfile の行の最後の `\` は、「次の行へ続く」という印です。長い `RUN` を、見やすく分けて書けます。	第12巻 UNIT 02
`COPY --from=`	`COPY --from=builder /app/target/release/rustnote /app/rustnote` は、前の段（`builder`）で作った物を、この段にコピーします。	第12巻 UNIT 02
`USER`	`USER app` は、これより後の命令と、コンテナでアプリを動かす時の利用者を `app` にします。管理者（root）で動かさないことで、もし乗っ取られても、できることを小さくします。	第12巻 UNIT 02
`ENV`	`ENV HOST=0.0.0.0` は、イメージに環境変数を書き込みます。イメージを見ればだれでも読めるので、秘密の値は書きません。	第12巻 UNIT 02
`EXPOSE`	`EXPOSE 8080` は、「このコンテナは 8080 番で待ち受けます」という説明の印です。これだけではポートは開かず、`docker run -p` で開きます。	第12巻 UNIT 02
`CMD`	`CMD ["/app/rustnote"]` は、コンテナを動かした時に、最初に動かすプログラムです。	第12巻 UNIT 02
`["…"]`	`CMD ["/app/rustnote"]` のように `[ ]` と `"…"` で書く形（exec 形式）は、シェルを通さずに、プログラムを直接動かします。止める合図（Ctrl+C など）が、アプリに正しく届きます。	第12巻 UNIT 02
`docker build --progress`	`--progress=plain` は、組み立ての途中の表示を、まとめずに全部、1 行ずつそのまま出す指定です。どの段で何が起きたかを調べる時に使います。	第12巻 UNIT 02
`docker history`	イメージが、どの手順で、どれだけの大きさを使って作られたかを表示するコマンド。	第12巻 UNIT 02
`docker history --format`	`--format "{{.Size}}`t{{.CreatedBy}}"` は、イメージの層（Dockerfile の 1 行ずつ）の一覧を、大きさと作った命令だけにします。	第12巻 UNIT 02
`docker run -e`	`-e DATABASE_URL=…` は、コンテナの中のプログラムに、環境変数を渡す指定です。	第12巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`docker image inspect --format`	`docker image inspect イメージ --format "{{range .Config.Env}}{{println .}}{{end}}"` は、イメージのくわしい情報から、環境変数の一覧だけを 1 行ずつ取り出します。	第12巻 UNIT 05
`docker history --no-trunc`	`--no-trunc` は、長い命令を途中で切らずに、全部表示する指定です。	第12巻 UNIT 05
`docker network`	コンテナ（箱）どうしをつなぐネットワークを作る・見るコマンド。`docker network create rn-net` で作り、同じネットワークの箱どうしは名前で呼び合えます。	第12巻 UNIT 06
`docker start`	止まっているコンテナを動かすコマンド。	第12巻 UNIT 06
`docker tag`	イメージに、別の名前（タグ）を付けるコマンド。`docker tag rustnote:u02 rustnote:v1` 。	第12巻 UNIT 10
`docker rm`	コンテナ（箱）を消すコマンド。`-f` で、動いていても止めて消します。	第12巻 UNIT 10

Fly.io

コード	意味と使い方	初めて出てくる所
`fly version`	Fly.io（アプリをインターネットに公開するサービス）の道具 `fly` の版を表示します（第12巻）。	第1巻 UNIT 10
`fly launch`	Fly.io に、新しいアプリを作る準備をするコマンド。`fly.toml`（設定のファイル）を作ります。`--no-deploy` で、まだ公開はしません（第12巻）。	第12巻 UNIT 04
`fly launch --no-deploy`	`--no-deploy` は、アプリの登録と `fly.toml` 作りだけをして、まだ公開（deploy）しない指定です。設定を見直してから公開できます。	第12巻 UNIT 04
`fly launch --name`	`--name 自分用のアプリ名` は、Fly.io の上のアプリの名前を決めます。名前は世界中で 1 つだけなので、ほかの人と重ならない名前にします。	第12巻 UNIT 04
`fly secrets`	Fly.io のアプリに、秘密の設定（データベースの合言葉など）を渡すコマンド。`fly secrets set 名前=値` 。アプリには環境変数として届き、`fly.toml` や Git には書きません（第12巻）。	第12巻 UNIT 05
`fly deploy`	アプリを作って（Docker のイメージ）、Fly.io に公開するコマンド。新しい版に、少しずつ入れかえます（第12巻）。	第12巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`fly status`	Fly.io のアプリのマシンの状態と、ヘルスチェック（元気か）の結果を見るコマンド。	第12巻 UNIT 08
`fly logs`	Fly.io で動いているアプリのログ（動作の記録）を見るコマンド。	第12巻 UNIT 08
`fly certs`	Fly.io のアプリに、自分の名前（ドメイン）の HTTPS の証明書を付けるコマンド。	第12巻 UNIT 08
`fly releases`	Fly.io のアプリの、公開した版の歴史を見るコマンド。`--image` で、どのイメージだったかも表示し、前の版に戻す時に使います。	第12巻 UNIT 10
`fly releases --image`	`fly releases --image` は、公開した版（リリース）の一覧に、それぞれが使ったイメージの名前も表示します。前の版に戻す時に使います。	第12巻 UNIT 10
`fly deploy --image`	`fly deploy --image registry.fly.io/…` は、作り直さずに、指定したイメージで公開します。前に動いていたイメージを指定すれば、前の版に戻せます。	第12巻 UNIT 10

Node.js

コード	意味と使い方	初めて出てくる所
`DatabaseSync`	`DatabaseSync` は、Node.js に付いている SQLite の部品（`node:sqlite`）で、データベースのファイルを開く型です。	第8巻 UNIT 08
`rmSync()`	`rmSync(場所, { force: true })` は、Node.js（`node:fs`）で、ファイルを消す関数です。`force: true` で、無くてもエラーにしません。	第8巻 UNIT 08
`node`	Node.js（パソコンの上で JavaScript を動かす道具）を動かすコマンド。`node probe.mjs` で、そのファイルを動かします。	第8巻 UNIT 08
`.env`	`process.env` は、Node.js のプログラムが読める環境変数の入れ物です。`process.env.NODE_ENV` で 1 つ読めます。	第8巻 UNIT 08
`.argv`	`process.argv` は、プログラムを動かした時の引数の並びです。0 番目は `node` の場所、1 番目はスクリプトの場所、2 番目からが渡した引数です。	第9巻 UNIT 09
`npm init`	Node.js のプロジェクトを始めるコマンド。`package.json`（使う部品やコマンドを書くファイル）を作ります。`-y` を付けると、質問に全部「既定のまま」と答えます。	第11巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`npm init -y`	`npm init -y` の `-y`（yes）は、質問に全部「決まりの答え」で答えて、`package.json` をすぐに作る指定です。	第11巻 UNIT 05
`npm install`	Node.js の部品を入れるコマンド。`--save-dev` は「作る時・試す時だけ使う部品」として、`--save-exact` は版を固定して、`package.json` に書きます。	第11巻 UNIT 05
`npm install --save-dev`	`--save-dev` は、開発やテストの時だけ使う部品として、`package.json` の `devDependencies` に書く指定です。	第11巻 UNIT 05
`npm install --save-exact`	`--save-exact` は、版を `^1.64.1`（これ以上なら新しくてもよい）ではなく、`1.64.1` ぴったりで記録する指定です。いつ入れても同じ版になります。	第11巻 UNIT 05
`npx playwright`	`npx` は、入れた部品のコマンドを動かす道具。`npx playwright install chromium` で、Playwright（ブラウザーを自動で操作してテストする道具）が使うブラウザーを入れ、`npx playwright test` でテストを動かします。	第11巻 UNIT 05
`npx playwright --version`	`npx playwright --version` は、入っている Playwright の版を表示します。	第11巻 UNIT 05
`join()`	`join(tmpdir(), "rustnote")` は、Node.js（`node:path`）で、フォルダの場所をつなぐ関数です（OS に合った区切り文字を使います）。	第11巻 UNIT 05
`tmpdir()`	`tmpdir()` は、Node.js（`node:os`）で、一時ファイルを置くフォルダの場所を返す関数です。	第11巻 UNIT 05
`.argv`	`process.argv` は、プログラムを動かした時の引数の並びです。0 番目は `node` の場所、1 番目はスクリプトの場所、2 番目からが渡した引数です。	第13巻 UNIT 09

設定ファイル（JSON）

コード	意味と使い方	初めて出てくる所
`"files.eol":`	VS Code の設定（`settings.json`）の `"files.eol": "\n"` は、新しく作るファイルの改行を、LF（`\n`）にする設定です。	第1巻 UNIT 03
`"files.insertFinalNewline":`	`"files.insertFinalNewline": true` は、保存する時に、ファイルの最後に改行を 1 つ入れる設定です。	第1巻 UNIT 03

コード	意味と使い方	初めて出てくる所
`"files.trimTrailingWhitespace":`	`"files.trimTrailingWhitespace": true` は、保存する時に、行の終わりの余分な空白を消す設定です。	第1巻 UNIT 03
`"editor.formatOnSave":`	`"editor.formatOnSave": true` は、保存するたびに、コードを自動で整える（字下げなどをそろえる）設定です。	第1巻 UNIT 03
`"[rust]":`	`"[rust]": { … }` は、「Rust のファイルの時だけ」の設定を、`{ }` の中にまとめる書き方です。	第1巻 UNIT 03
`"editor.defaultFormatter":`	`"editor.defaultFormatter": "rust-lang.rust-analyzer"` は、コードを整える係に、どの拡張機能を使うかです。	第1巻 UNIT 03
`"rust-analyzer.check.command":`	`"rust-analyzer.check.command": "clippy"` は、保存した時の確かめに、`cargo check` ではなく `cargo clippy`（もっと細かく教える係）を使う設定です。	第1巻 UNIT 03
`"[powershell]":`	`"[powershell]": { … }` は、「PowerShell のファイル（ `.ps1` ）の時だけ」の設定をまとめる書き方です。	第1巻 UNIT 03
`"files.encoding":`	`"files.encoding": "utf8bom"` は、ファイルを保存する時の文字の符号です。ここでは、Windows PowerShell 5.1 が日本語を正しく読めるように、PowerShell のファイルだけを BOM 付きの UTF-8 にします。	第1巻 UNIT 03
`"recommendations":`	`.vscode/extensions.json` の `"recommendations"` は、このフォルダを開いた人に、入れるのをすすめる拡張機能の一覧です。	第1巻 UNIT 03

Python の道具

コード	意味と使い方	初めて出てくる所
`python`	Python を動かすコマンド。`python -c "…"` は、`"…"` の中の Python のコードを 1 回だけ動かします。	第12巻 UNIT 04
`python -c`	`python -c "…"` は、後ろの文字を Python のプログラムとして、その場で動かします。ファイルを作るほどでない、短い確かめに使います。	第12巻 UNIT 04

コメント

コード	意味と使い方	初めて出てくる所
`//`	`//` から行の終わりまでは「コメント」（人が読むためのメモ）で、プログラムとしては無視されます。`// OS の種類を表示する` のように、何をしているかを日本語で書いておくと、あとで読み返す時に助かります。	第0巻 UNIT 01
`/* */`	`/*` から `*/` までもコメントです。`{ /* 値があったとき */ }` のように行の途中にも書け、何行にもまたがって書くこともできます。ふだんは `//` を使います。	第0巻 UNIT 01
`///`	`///` で始まるコメントは、すぐ下の関数や型の「説明書」（ドキュメントコメント）です。`cargo doc` で説明のページになり、VS Code では、名前にマウスを乗せると表示されます。ふつうのコメント（`//`）とちがい、何の説明かが決まっています。	第1巻 UNIT 06
`//!`	`//!` で始まるコメントは、そのファイル（モジュール）全体の説明書です。ファイルのいちばん上に書きます。`///`（すぐ下の関数や型の説明）と使い分けます。	第4巻 UNIT 10

画面に出す

コード	意味と使い方	初めて出てくる所
`println!`	かっこの中の文字を画面に出して、最後に改行するマクロ。`println!("Hello, world!");` は `Hello, world!` と表示します。`{}` を書くと、後ろに並べた値がそこに入ります（例: `println!("{}点", 80);` → `80点`）。名前の最後の `!` は「マクロ」（コードを作り出す特別な仕組み）の印で、ふつうの関数と見分けるために付いています。	第0巻 はじめに: 手を動かす準備
`{名前}`	`println!` や `format!` の文字の中に `{os}` のように `{ }` で変数の名前を書くと、その変数の中身がそこに入ります（`let os = "windows"; println!("OS: {os}");` → `OS: windows`）。`println!("OS: {}", os)` と書くのと同じ結果です。`{ }` の中に書けるのは名前だけで、`{a + 1}` や `{p.x}` のような式は書けません（その時は `{}` と、後ろに並べる値を使います）。`{name:<22}` のように、名前の後ろに `:` と書式を付けることもできます。	第0巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`eprintln!`	`println!` のエラー版。「標準エラー出力」というエラー用の出口に書きます。画面ではふつうの出力と同じに見えますが、`> ファイル` で結果をファイルに書き出した時も、エラーは画面に残ります。失敗の知らせを出す時に使います。	第0巻 UNIT 01
`{}`	`println!("{}", 値)` の `{}` は、「ここに値を入れる」場所の印です。後ろに `,` で並べた値が、左から順に入ります（`println!("{} と {}", 1, 2)` → `1 と 2`）。`{}` は、人が読む形（Display）で表示します。	第0巻 UNIT 01
`print!`	`println!` と同じく画面に出すマクロですが、最後に改行しません。続けて出したい時（例: `print!("入力してください: ");` のあとに入力を待つ）に使います。	第0巻 UNIT 02
`{:?}`	`{:?}` は、値を「プログラマー向けの形」（Debug）で表示する指定です。`Some("main.rs")` のように、`Option` の `Some` や、文字列の `"` まで、そのまま見せます。`{}` では表示できない型（`Vec`・`Option` など）も、`{:?}` なら表示できます。`{path:?}` のように名前と一緒にも書けます。自分で作った構造体は、`#[derive(Debug)]` を付けると使えます。	第0巻 UNIT 02
`{:<幅}`	`{ }` の中の `:` の後ろは、「どう並べて表示するか」の指定（書式）です。`<` は**左寄せ**、その後ろの数は**幅**（何文字分の場所を取るか）です。`{candidate:<22}` は、`candidate` の中身を 22 文字分の場所に左寄せで入れ、足りない分を右側に空白で埋めます（`"main.rs"` なら、7 文字の後ろに空白 15 個）。だから、名前の長さがばらばらでも、後ろに続く `->` の位置が縦にそろいます。右寄せは `{:>22}`、中央寄せは `{:^22}` です。中身が幅より長い時は、切られずに全部出ます。日本語も 1 文字と数えるので、全角の文字が混ざると、画面の上では少しずれます。	第0巻 UNIT 02
`{:0幅}`	`{:02}` のように、幅の前に `0` を付けると、「この幅になるまで、前を 0 で埋める」指定になります。`{:02}` で `7` は `07`、`{:06}` で `42` は `000042`。`{:016x}` は「16 けたになるまで 0 で埋めた 16 進数」です。時刻（`09:05`）や番号（`000042`）の表示に使います。	第0巻 UNIT 08
`{:x}`	`{:x}` は、整数を 16 進数（0〜9 と a〜f の 16 種類の字で数える書き方）で表示する指定です。`255` は `ff`。大文字にしたい時は `{:X}`、前に `0x` を付けたい時は `{:#x}` です。	第0巻 UNIT 08
`{:.桁}`	`{:.1}` の `.` と数は、「小数点の下を何けた出すか」の指定です。`{:.1}` で `3.14159` は `3.1`、`{:.2}` で `3.14`、`{:.0}` なら `3`。けたを切りそろえて（丸めて）表示します。`{:.2?}` のように Debug（`?`）と組み合わせると、かかった時間（`Duration`）を `1.23ms` のように短く出せます。	第2巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`{:X}`	`{:X}` は、整数を大文字の 16 進数（0〜9 と A〜F）で表示する指定です。`255` は `FF`、`'A' as u32`（65）は `41`。文字の番号や色の値を見る時に使います。	第2巻 UNIT 02
`{:x?}`	`{:x?}`・`{:02X?}` は、Debug の表示（`{:?}`）の中の数を、16 進数で出す指定です。`b"Hi"` を `{:x?}` で出すと `[48, 69]` のようになり、バイトの並びを調べる時に使います。	第2巻 UNIT 05
`{:#?}`	`{:#?}` は、Debug の表示（`{:?}`）を、項目ごとに改行して字下げした、読みやすい形で出す指定です。項目の多い構造体の中身を見る時に使います。	第2巻 UNIT 07
`{:p}`	`{:p}` は、参照（`&x`）やポインタが指している、メモリの中の場所（アドレス）を、`0x7ff6…` のような 16 進数で表示する指定です。2 つが同じ場所を指しているかを確かめる時に使います。	第3巻 UNIT 02
`{:>幅}`	`{:>3}` の `>` は**右寄せ**の指定で、数は幅（何文字分の場所を取るか）です。`{:>3}` で `7` は `7`（前に空白 2 つ）。数を表のように並べる時に、けたがそろいます。左寄せは `<`、中央寄せは `^` です。	第3巻 UNIT 03
`{0}`	`{0}`・`{1}` のように `{ }` の中に番号を書くと、「後ろに並べた値の、何番目（0 から数える）を入れるか」を指定できます。`format!("{0} と {1} と {0}", "a", "b")` → `a と b と a` のように、同じ値を何度も使えます。`thiserror` の `#[error("…{0}…")]` では、名前の無い項目の 0 番目（`.0`）を入れます。	第4巻 UNIT 05
`{{ }}`	`println!` や `format!` の文字の中で、`{` や `}` そのものを出したい時は、`{{`・`}}` と 2 つ重ねて書きます。`format!("{{\"id\": {id}}}")` は `{"id": 7}` になります。1 つだけ書くと、値を入れる場所の印だと思われます。	第5巻 UNIT 09
`{:^幅}`	`{:^8}` の `^` は**中央寄せ**の指定で、数は幅です。8 文字分の場所の真ん中に置き、左右を空白で埋めます。左寄せは `<`、右寄せは `>` です。	第14巻 B. サンプル集
`{:#}`	`{:#}` は、人が読む形（`{}`）の「もう 1 つの形」で表示する指定です。`anyhow` のエラーを `{:#}` で出すと、「書き足した説明: 元のエラー」のように、原因までを 1 行につないで表示します（`{}` だと、いちばん外側の説明だけです）。	第14巻 B. サンプル集

変数

コード	意味と使い方	初めて出てくる所
`let`	値に名前を付けて覚えておく（変数を作る）書き方。`let age = 20;` で、`age` という名前で 20 を覚えます。最初は「変えられない」変数です。あとで変えたい時は `let mut age = 20;` のように `mut` を付けます。	第0巻 UNIT 01
`mut`	「あとで変えてよい」という印。`let mut total = 0;` と書くと、`total += 10;` のように値を変えられます。付けないと変えられません（変えようとすると、コンパイルのエラーで教えてくれます）。	第0巻 UNIT 01
`let _ =`	`let _ = fs::remove_file("count.txt");` の `let _ =` は、「結果は受け取るが、使わずに捨てる」という書き方です。`Result` を返す関数の結果を使わないと「使っていない」と警告が出るので、わざと捨てる時に書きます。ここでは「ファイルが無くて失敗しても、気にしない」という意味です。	第0巻 UNIT 07
`const`	ずっと変わらない値（定数）に名前を付ける書き方。`const MAX_TITLE_CHARS: usize = 80;`。名前は大文字と `_` で書き、型（`usize` など）を必ず書きます。プログラムのどこからでも使えます。	第2巻 UNIT 01
`static`	プログラムが動いている間、ずっとある値。`&'static str` は「プログラムの中に書いた、ずっと消えない文字列」を借りている型です（`"回覧中"` のような文字列はこの型）。	第2巻 UNIT 08
`_名前`	`_name` のように名前の頭に `_` を付けると、「使わないかもしれない変数」という意味になり、使っていなくても警告が出ません。	第2巻 UNIT 08

数

コード	意味と使い方	初めて出てくる所
`+=`	`total += n;` は `total = total + n;` を短く書いた形です。`total` は `let mut` で作った（変えられる）変数でないといけません。文字列（`String`）でも `s += "続き";` で書き足せます。	第0巻 UNIT 01
`u64`	0 以上の整数の型（64 ビット。0〜約 1844 京）。とても大きな数を数える時に使います。	第0巻 UNIT 04
`%`	`a % b` は「a を b で割った余り」です。`10 % 3` は `1`。偶数かどうかは `n % 2 == 0` で調べられます。	第0巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`1_000`	数の中の `_` は、読みやすくするための区切りで、値には関係ありません。`1_000_000` は `1000000` と同じです。	第0巻 UNIT 04
`10u8`	数のすぐ後ろに型の名前を付けると、その型の数になります。`10u8` は `u8` 型の 10、`0u64` は `u64` 型の 0 です。型を書かないと、整数は `i32` と決まります（使い方から別の型に決まることもあります）。	第0巻 UNIT 04
`i32`	マイナスもある整数の型（32 ビット。約 −21 億〜21 億）。`i` は integer（整数）の頭文字です。型を書かない整数は、ふつう `i32` になります。	第0巻 UNIT 04
`u32`	0 以上の整数の型（32 ビット。0〜約 42 億）。数を数える・番号を付ける時に使います。	第0巻 UNIT 07
`1.5`	`1.08` のように `.` の付いた数は小数で、型を書かなければ `f64` になります。整数（`i32` など）と小数は、そのままでは足したり掛けたりできないので、`as f64` などで型をそろえます。	第0巻 UNIT 09
`f64::from()`	整数などを、小数（`f64`）に変える関数。`f64::from(price) * 1.08` のように、計算の前に型をそろえます（`as` と違い、値が変わってしまう変換はできません）。	第0巻 UNIT 09
`f64`	小数を入れる型（64 ビット）。`3.14` のような数です。小数は、`0.1 + 0.2` が `0.30000000000000004` になるような、わずかな誤差が出るので、お金の計算には使いません（整数の「円」で数えます）。	第0巻 UNIT 09
`usize`	0 以上の整数の型で、大きさはパソコンに合わせて決まります（64 ビットのパソコンなら `u64` と同じ）。並びの長さ（`.len()`）や、何番目か（`v[i]` の `i`）は、いつも `usize` です。	第0巻 UNIT 10
`u16`	0〜65535 の整数の型（16 ビット）。ポート番号（`8080` など）はこの範囲に収まるので、`u16` で表します。	第2巻 UNIT 01
`MIN`	数の型が持っている「いちばん小さい値」。`i32::MIN` は -2147483648。`u8::MIN` は 0 です。	第2巻 UNIT 02
`MAX`	数の型が持っている「いちばん大きい値」。`i32::MAX` は 2147483647、`u8::MAX` は 255 です。これより大きくなる計算は「あふれ（オーバーフロー）」になります。	第2巻 UNIT 02
`u8`	0〜255 の整数の型（8 ビット＝1 バイト）。`u` は unsigned（マイナスが無い）の意味です。ファイルや通信の中身（バイトの並び `Vec<u8>`）は、この型の並びです。	第2巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`.wrapping_add()`	足し算をして、型に入りきらない時は、0（いちばん小さい値）に戻って数え直すメソッド。時計の針が 12 を過ぎて 1 に戻るのと同じです（`255u8.wrapping_add(1)` は 0）。	第2巻 UNIT 02
`.checked_add()`	足し算をして、型に入りきらない時は `None`（計算できなかった）を返すメソッド。入りきれば `Some(結果)`。お金の合計など、あふれると困る所で使います。	第2巻 UNIT 02
`.saturating_add()`	足し算をして、型に入りきらない時は、いちばん大きい値で止めるメソッド（`250u8.saturating_add(10)` は 255）。	第2巻 UNIT 02
`.overflowing_add()`	足し算の結果（あふれたら一周した値）と、「あふれたかどうか」の両方を、組 `(値, true/false)` で返すメソッド。	第2巻 UNIT 02
`as`	型を変える書き方。`x as f64` は整数を小数に、`300 as u8` は 300 を u8 に変えます。入りきらない分は**黙って切り捨てられる**ので（300 → 44）、範囲が分からない時は `try_from` を使います。	第2巻 UNIT 02
`u32::from()`	小さい整数の型などを、`u32` に変える関数（値が変わらない変換だけ）。	第2巻 UNIT 02
`u8::try_from()`	別の型の数を `u8`（0〜255）に変える関数。入りきらない数（300 など）だと `Err` を返すので、`as u8`（はみ出た分を黙って切り捨てる）より安全です。結果は `Result` です。	第2巻 UNIT 02
`.abs()`	絶対値（マイナスを取った値）を返すメソッド。`(-5).abs()` は 5。小数に使う時は、値の型が `f64` などと決まっている必要があります。	第2巻 UNIT 02
`1e6`	`1e6` は「1 × 10 の 6 乗」（1000000.0）を表す、小数の書き方です。`2.5e-3` は 0.0025 です。とても大きい数や小さい数を、短く書けます。	第2巻 UNIT 02
`0xff`	`0x` で始まる数は 16 進数です。`0xff` は 255、`0x3C` は 60。バイトの値や色（`0xFF0000`）を書く時に使います。`0b1010`（2 進数）という書き方もあります。	第2巻 UNIT 02
`.checked_mul()`	掛け算をして、型に入りきらない時は `None` を返すメソッド（`checked_add` の掛け算版）。	第2巻 UNIT 02
`u64::try_from()`	別の型の数を `u64`（0 以上の大きな整数）に変える関数。マイナスの数などは入らないので `Err` を返します。	第2巻 UNIT 02
`i64`	マイナスもある整数の型（64 ビット。約 ±922 京）。データベースの整数の列や、番号（id）によく使います。	第2巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`.max()`	2 つの数のうち、大きい方を返すメソッド。`page.max(1)` は「`page` と 1 の大きい方」なので、0 やマイナスが来ても 1 になります（下限を決める時の書き方）。イテレーターに使うと、いちばん大きい物を `Option` で返します。	第2巻 UNIT 02
`u16::try_from()`	別の型の数を `u16`（0〜65535。ポート番号などに使う）に変える関数。入りきらないと `Err` を返します。	第2巻 UNIT 02
`*=`	`n *= 2;` は `n = n * 2;` を短く書いた形です。同じように `/=`（割る）・`%=`（余りにする）もあります。	第2巻 UNIT 04
`-=`	`remaining -= 1;` は `remaining = remaining - 1;` を短く書いた形です。	第2巻 UNIT 04
`.div_ceil()`	割り算をして、余りがあれば切り上げるメソッド。`23.div_ceil(10)` は 3（23 件を 10 件ずつ分けると 3 ページ）。ページ数の計算によく使います。	第2巻 UNIT 04
`.min()`	2 つのうち小さい方を返すメソッド。`(start + per_page).min(items.len())` は「どちらか小さい方」。イテレーターに使うと、並びの中でいちばん小さい物を `Option` で返します（大きい方は `.max()`）。	第2巻 UNIT 06
`.pow()`	累乗（同じ数を何回も掛ける）を計算するメソッド。`2u32.pow(3)` は 8。待ち時間を 1・2・4・8 倍と増やしていく時に使います。	第5巻 UNIT 08
`.is_multiple_of()`	その数が、指定した数で割り切れるなら `true` を返すメソッド。`n.is_multiple_of(5)` は「5 の倍数か」。	第5巻 UNIT 09
`.wrapping_sub()`	引き算をして、型に入りきらない時（0 より小さくなる時）は、いちばん大きい値の方へ一周するメソッド。`0u64.wrapping_sub(1)` は大きな数になるので、`get` で「無い」と判断させるのに使えます。	第6巻 UNIT 10
`i64::from()`	小さい整数の型などを、`i64` に変える関数（値が変わらない変換だけ）。	第7巻 UNIT 09
`.saturating_sub()`	引き算をして、0 より小さくなる時は 0 で止めるメソッド。`names.len().saturating_sub(keep)` は「数が足りなければ 0」。	第7巻 UNIT 10
`.to_be_bytes()`	数を、大きい桁から順に並べたバイトの並び（ビッグエンディアン）にするメソッド。	第8巻 UNIT 09
`u32::from_be_bytes()`	4 つのバイト（大きい桁から順）から、`u32` の数を作る関数。	第8巻 UNIT 09
`u64::from()`	小さい整数の型などを、`u64` に変える関数（値が変わらない変換だけ）。	第10巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.clamp()`	値を、決めた範囲（最小〜最大）に収めるメソッド。`x.clamp(1, 5)` は、1 より小さければ 1、5 より大きければ 5。	第11巻 UNIT 07
`usize::from()`	小さい整数の型などを、`usize` に変える関数（値が変わらない変換だけ）。	第11巻 UNIT 07
`u32::from_le_bytes()`	4 つのバイト（小さい桁から順）から、`u32` の数を作る関数。実行ファイルの中の決まった場所の数を読む時に使います。	第12巻 UNIT 01
`.cmp()`	2 つの値を比べて、「小さい・等しい・大きい」（`Ordering`）を返すメソッド。`sort_by` の中でよく使います。`.then(…)` で「同じなら次の基準で」とつなげられます。	第14巻 B. サンプル集
`PI`	`std::f64::consts::PI` は、円周率（3.14159…）の決まった値です。	第14巻 B. サンプル集

文字

コード	意味と使い方	初めて出てくる所
`'a'`	`'='` のように 1 文字を `'`（シングルクォート）で囲むと、`char`（1 文字）の値になります。何文字でも入る文字列（`"…"`）とは別の物です。`split_once('=')` は「最初の `=` で 2 つに分ける」。	第0巻 UNIT 05
`char`	1 文字を入れる型です。`'a'`・`'あ'`・`'😀'` のように `'` で囲んで書きます。日本語の文字も絵文字も、1 つの `char` です（大きさはいつも 4 バイト）。	第2巻 UNIT 02
`.to_digit()`	1 文字（`char`）を、その文字が表す数に変えるメソッド。`'7'.to_digit(10)` は `Some(7)`（10 は「10 進数で」という意味）。数字でない文字なら `None` です。	第2巻 UNIT 02
`.is_alphabetic()`	その文字が「文字（英字・かな・漢字など）」なら `true` を返すメソッド。	第2巻 UNIT 02
`.is_ascii_alphanumeric()`	その文字が、ASCII（半角）の英字か数字（`A`〜`Z`・`a`〜`z`・`0`〜`9`）なら `true` を返すメソッド。`'あ'` は `false` です。	第2巻 UNIT 02
`char::from_u32()`	番号（文字コード）から 1 文字を作る関数。`char::from_u32(0x41)` は `Some('A')`。文字に無い番号なら `None` です。	第2巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`.len_utf8()`	その 1 文字が、UTF-8 で何バイトかを返すメソッド。英数字は 1、日本語の多くは 3 です。	第2巻 UNIT 05
`.is_alphanumeric()`	その文字が、文字（英字・かな・漢字など）か数字なら `true` を返すメソッド。	第9巻 UNIT 04
`b'a'`	`b'A'` は、1 文字を `char` ではなく 1 バイト（`u8`）。`b'A'` は 65）として書く書き方です。`b'A'..=b'Z'` で、「バイトが A〜Z のどれか」を調べられます。	第13巻 UNIT 10

文字列

コード	意味と使い方	初めて出てくる所
`String::from()`	`"…"`（`&str`）から、自分の物の文字列（`String`）を作る関数。`String::from("こんにちは")`。`"こんにちは".to_string()` と同じです。	第0巻 UNIT 03
`String`	自分の物として持つ、変えられる文字列の型。後ろに足す（`push_str`）・書きかえる、ができます。`"…"` と書いた文字列は `&str`（借りている読むだけの文字列）で、関数の引数は `&str` にしておくと、どちらも渡せます（第2巻 UNIT 05）。	第0巻 UNIT 03
`.trim_end()`	文字列の後ろ（右側）の空白や改行を取り除くメソッド。キーボードから読んだ 1 行の最後に付いている改行を取る時によく使います。前後両方を取る時は `.trim()` です。	第0巻 UNIT 03
`.parse()`	文字列を、数などの別の型に変えるメソッド。`"42".parse::<i32>()` や、`let n: i32 = "42".parse().unwrap();` のように、何に変えるかを型で伝えます。変えられない文字（`"abc"` など）もあるので、結果は `Result` です。	第0巻 UNIT 04
`.split_once()`	文字列を、最初に見つかった区切りの所で 2 つに分けるメソッド。`"title=買い物".split_once('=')` は `Some(("title", "買い物"))`。区切りが無い時は `None` です。	第0巻 UNIT 05
`.find()`	文字列の中から、文字や文字列を探して、見つかった位置（何バイト目か）を `Option` で返すメソッド。`"a/b".find('/')` は `Some(1)`、無ければ `None`。イテレーターに使うと、条件に合う最初の 1 つを返します。	第0巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`.split()`	文字列を、区切り文字のたびに分けるメソッド。`"a,b,c".split(',')` は `a`・`b`・`c` の 3 つ。結果は 1 つずつ取り出せる形（イテレーター）なので、`Vec` にしたい時は `.collect()` を付けます。	第0巻 UNIT 05
`.as_bytes()`	文字列を、中のバイト（コンピューターが持っている 0〜255 の数の並び）として見るメソッド。ネットやファイルに書く関数がバイトを求める時に使います。	第0巻 UNIT 05
`.chars()`	文字列を、1 文字ずつ（`char`）取り出せる形にするメソッド。`s.chars().count()` で文字の数、`s.chars().take(60).collect::<String>()` で先頭の 60 文字。日本語も 1 文字ずつ正しく扱えます（`len()` はバイト数）。	第0巻 UNIT 05
`String::new()`	空の `String` を作る関数。あとで中身を足していく時（`read_line(&mut s)` で 1 行読み込む など）に使います。	第0巻 UNIT 05
`\`	文字列の中で、行の終わりに `\` を書くと、「次の行へ続く」という意味になります。改行と、次の行の頭の空白は、文字列に入りません。長い文字列を、見やすく何行かに分けて書く時に使います。	第0巻 UNIT 05
`format!`	`println!` と同じ書き方で、画面に出さずに文字列（`String`）を作るマクロ。`let msg = format!("{}さん", name);` のように、値を埋め込んだ文を作る時に使います。	第0巻 UNIT 06
`.split_whitespace()`	文字列を、空白（スペース・タブ・改行）の所で分けるメソッド。空白がいくつ続いても 1 つの区切りとして扱います。`"GET / HTTP/1.1".split_whitespace()` は `GET`・`/`・`HTTP/1.1`。	第0巻 UNIT 06
`.trim()`	文字列の前後の空白や改行を取り除いたものを返すメソッド。入力された文字の、うっかり入った空白を取る時によく使います。	第0巻 UNIT 07
`.to_string()`	値を、自分の物の文字列（`String`）に変えるメソッド。`"abc".to_string()` や `42.to_string()` のように使います。	第0巻 UNIT 07
`.lines()`	文字列を、1 行ずつに分けて取り出せる形にするメソッド。行の終わりの改行（`\n` や `\r\n`）は含みません。`text.lines().count()` で行の数が分かります。	第0巻 UNIT 07
`str`	`&str` の `str` は「文字列の中身」を表す型で、ふつうは `&str`（借りた文字列）の形で使います。`"こんにちは"` のように `"` で囲んで直接書いた文字列は `&str` です。自分で持って、あとから書き足せる文字列は `String` です。	第0巻 UNIT 07
`.as_deref()`	`Option<String>` を、中身を借りて `Option<&str>` に変えるメソッド。`env::args().nth(1).as_deref() == Some("reset")` のように、「引数が `reset` かどうか」を文字列と比べる時に使います。	第0巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`.contains()`	含まれていれば `true` を返すメソッド。文字列なら `"report.csv".contains("2026")`、範囲なら `(1..=80).contains(&length)`（1〜80 に入っているか）、`Vec` なら `v.contains(&x)` です。	第0巻 UNIT 10
`.repeat()`	文字列をくり返してつないだ、新しい文字列を作るメソッド。`"あ".repeat(80)` は「あ」が 80 個並んだ文字列。長い入力を試すテストでよく使います。	第0巻 UNIT 10
`.is_empty()`	空（長さが 0）なら `true` を返すメソッド。文字列にも `Vec` にも使えます。`title.trim().is_empty()` は「空白を取ったら何も残らないか」を調べます。	第1巻 UNIT 06
`.push_str()`	`String` の後ろに文字列を足すメソッド。`s.push_str("さん");`。`s` は書きかえるので、`let mut s` のように `mut` が要ります。	第2巻 UNIT 05
`.to_uppercase()`	英字を大文字にした、新しい文字列を返すメソッド。`"abc".to_uppercase()` は `"ABC"`。日本語は変わりません。	第2巻 UNIT 05
`.is_char_boundary()`	そのバイトの位置が、文字の切れ目かどうかを調べるメソッド。日本語は 1 文字が 3 バイトなので、文字の途中で切ると止まってしまいます。切る前の確かめに使います。	第2巻 UNIT 05
`.to_lowercase()`	英字を小文字にした、新しい文字列を返すメソッド。メールアドレスをそろえる時などに使います。	第2巻 UNIT 05
`.starts_with()`	その文字（文字列）で始まっていれば `true` を返すメソッド。`"report.csv".starts_with("report")` は `true`。終わりを調べるのは `.ends_with()`、含むかは `.contains()` です。	第2巻 UNIT 05
`.replace()`	文字列の中の、ある部分を全部置きかえた、新しい文字列を返すメソッド。`"a-b".replace('-', "/")` は `"a/b"`。元の文字列は変わりません。	第2巻 UNIT 05
`.as_str()`	`String`（持ち主のいる文字列）を、借りた `&str` として見せるメソッド。`&str` を受け取る関数に渡す時などに使います。	第2巻 UNIT 08
`write!`	`println!` と同じ書き方で、画面ではなく、決めた書き込み先に書くマクロ。`Display` を実装する時の `write!(f, "…")` は、「表示する文を `f` に書く」という意味です。	第2巻 UNIT 09
`.char_indices()`	文字列を、「何バイト目か」と「その文字」の組で、1 文字ずつ取り出すメソッド。日本語を、文字の切れ目で安全に切る時に使います。	第3巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`.rfind()`	文字列の後ろから探して、最初に見つかった位置（バイトの番号）を `Option` で返すメソッド。前から探すのは `.find()` です。	第3巻 UNIT 04
`.strip_prefix()`	文字列が、決めた文字で始まっていれば、それを取り除いた残りを `Some(残り)` で返すメソッド。始まっていなければ `None` です。	第3巻 UNIT 05
`.write_str()`	`Display` などの実装の中で、書き込み先（`Formatter`）に文字列をそのまま書くメソッド。`f.write_str(&self.0)` は「自分の中の文字列を、そのまま表示する」。値を埋め込む時は `write!` を使います。	第4巻 UNIT 01
`r#"…"#`	`r#"…"#` も「生の文字列」で、`"#` が来るまでを、`\` も `"` もそのまま文字にします。`r#"{"title": "牛乳"}"#` のように、`"` の多い JSON や、何行もの SQL を書く時に便利です。	第4巻 UNIT 08
`b"…"`	`b"…"` は、文字列ではなく「バイトの並び」（`&[u8; N]`）を作る書き方です。中には英数字と記号（ASCII）だけを書けます。通信やファイルの中身を、バイトのまま扱う時に使います。	第5巻 UNIT 05
`.eq_ignore_ascii_case()`	英字の大文字と小文字を区別せずに、同じかどうかを比べるメソッド。HTTP のヘッダーの名前（`Content-Type` と `content-type`）を比べる時などに使います。	第5巻 UNIT 08
`String::from_utf8_lossy()`	バイトの並びを、文字列として読む関数。文字として読めない所があっても止まらず、`&#xFFFD;` に置きかえて読みます。	第5巻 UNIT 08
`.to_str()`	値（HTTP のヘッダーの値・パスなど）を、`&str`（文字列）として読むメソッド。文字として読めないこともあるので、`Result` や `Option` で返します。	第6巻 UNIT 04
`String::from_utf8()`	バイトの並び（`Vec<u8>`）を、文字列（`String`）に変える関数。UTF-8 の文字として読めない時は `Err` を返します。	第6巻 UNIT 07
`String::with_capacity()`	「これくらいの長さになる予定」と容量を決めて、空の `String` を作る関数。少しずつ足していく時に、広げ直す手間を減らせます。	第7巻 UNIT 02
`include_str!`	ファイルの中身を、翻訳（コンパイル）の時に文字列として埋め込むマクロ。`include_str!("../../sql/schema.sql")`。動かす時にファイルが無くても使えます。	第7巻 UNIT 04
`.to_string_lossy()`	ファイル名など、文字として読めないかもしれない物を、読めない所を `&#xFFFD;` に置きかえて文字列として読むメソッド。	第7巻 UNIT 10
`.into_owned()`	借りている文字列（など）を、自分の物（`String`）にするメソッド。	第7巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`.ends_with()`	その文字（文字列）で終わっていれば `true` を返すメソッド。`name.ends_with(".db")`。	第7巻 UNIT 10
`.trim_start_matches()`	文字列の先頭から、決めた文字（文字列）をくり返し取り除いたものを返すメソッド。`pair.trim_start_matches("__Host-session=")` は、クッキーの名前の部分を取って値だけにします。	第8巻 UNIT 01
`.replace_range()`	`String` の、決めた範囲を、別の文字列に置きかえるメソッド。	第8巻 UNIT 09
`.trim_end_matches()`	文字列の後ろから、決めた文字（文字列）をくり返し取り除いたものを返すメソッド。`name.trim_end_matches(".rs")` は、`.rs` を取ります。	第9巻 UNIT 04
`.match_indices()`	文字列の中で、探す文字が見つかった所の「位置と、見つかった文字」を、全部順に返すメソッド。	第9巻 UNIT 04
`.trim_start()`	文字列の前（左側）の空白を取り除いたものを返すメソッド。	第9巻 UNIT 04
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。`r"C:\Users\you"` や、正規表現の `r"\d+"` のように `\` の多い文字を、そのまま書けます。中に `"` を入れたい時は `r#"…"#` を使います。	第10巻 UNIT 07
`.encode_utf16()`	文字列を、UTF-16（ブラウザーの JavaScript が使う文字の数え方）の単位で並べるメソッド。`.count()` と合わせて、ブラウザーの `maxlength` と同じ数え方で長さを数えます。	第11巻 UNIT 01
`.rsplit()`	文字列を、後ろから順に区切り文字で分けるメソッド。`line.rsplit(' ').next()` は、最後の単語です。	第11巻 UNIT 07
`.bytes()`	文字列を、1 バイトずつ取り出せる形にするメソッド。`s.bytes().all(|b| b.is_ascii_digit())` で「全部、半角の数字か」を調べます。	第13巻 UNIT 10

文字列の中の特別な書き方

コード	意味と使い方	初めて出てくる所
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	第0巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`\r`	文字列の中の `\r` は「行の頭に戻る」（CR）という、目に見えない文字です。Windows のファイルや HTTP の通信では、行の終わりを `\r\n`（CR と LF の 2 文字）で表します。	第0巻 UNIT 05
`\t`	文字列の中の `\t` は「タブ」という、広めの空白のような区切りの文字です。表のように、列と列を区切る時に使います。	第0巻 UNIT 07
`\"`	文字列の中に `"` そのものを入れたい時は、`\"` と書きます。`"\"Rust\""` は、`"Rust"` という `"` 付きの文字になります。`\` を付けないと、そこで文字列が終わったと思われます。	第4巻 UNIT 04
`\'`	`'\''` のように、`'` で囲んだ 1 文字の中に `'` そのものを入れる時は、`\'` と書きます。	第7巻 この巻で身に付けること
`\\`	文字列の中に `\` そのもの（1 つ）を入れたい時は、`\\` と 2 つ書きます。`\` は、`\n` のような特別な書き方の始まりの印だからです。Windows の場所を `"C:\\Users"` と書くのも、この理由です（生の文字列 `r"C:\Users"` なら 1 つで済みます）。	第7巻 UNIT 02
`\u{…}`	`\u{3000}` のように `\u{ }` の中に 16 進数を書くと、その番号（Unicode という、世界中の文字の通し番号）の文字を表します。`\u{3000}` は全角の空白、`\u{1F600}` は 😀 です。見えにくい文字を、はっきり書きたい時に使います。	第9巻 UNIT 03
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	第10巻 UNIT 10
`\0`	文字列の中の `\0` は、番号 0 の文字（NUL）です。目に見えない文字で、ほかのプログラムとのやり取りでは「ここで終わり」の印に使われることがあるので、入力に混ざっていたら断るのが安全です。	第11巻 UNIT 04
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	第12巻 UNIT 09

正規表現

コード	意味と使い方	初めて出てくる所
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	第1巻 UNIT 01
`[^…]`	正規表現の `[^ ]` は、「中に書いた文字**以外**の 1 文字」を表します。`[^;]+` は「`;` 以外の文字が 1 つ以上」、`[^\x21-\x7E]` は「英数字と記号（ASCII の見える文字）以外」。	第1巻 UNIT 01
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	第8巻 UNIT 08
`(?=…)`	正規表現の `(?= )` は「この後ろに、こういう形が続く時だけ」、`(?! )` は「この後ろに、こういう形が続かない時だけ」を表します（先読み）。`^(?!api/|admin/).*$` は「`api/` や `admin/` で始まらない物、全部」。中の文字は、合わせた文字には入りません。	第8巻 UNIT 08
`\s`	正規表現の `\s` は、「空白 1 つ」（スペース・タブ・改行など）を表します。`\s*` は「空白が 0 個以上」。	第8巻 UNIT 08
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	第8巻 UNIT 08
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	第9巻 UNIT 09
`\s`	正規表現の `\s` は、「空白 1 つ」（スペース・タブ・改行など）を表します。`\s*` は「空白が 0 個以上」。	第9巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`[…]`	正規表現の `[ ]` は、「中に書いた文字の、どれか 1 つ」を表します。`[0-9a-zA-Z_-]` は「数字・英字・`_`・`-` のどれか」、`[.-]` は「`.` か `-`」。`-` で範囲を書けます。	第9巻 UNIT 09
`{n,m}`	正規表現の `{6}` は「すぐ前の物が、ちょうど 6 回」、`{8,12}` は「8 回から 12 回」を表します。`\d{6}` は「数字 6 けた」。	第9巻 UNIT 09
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	第9巻 UNIT 09
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	第9巻 UNIT 09
`(…)`	正規表現の `( )` は、まとまり（グループ）を作ります。合った文字のうち、`( )` の中の部分だけを、あとから取り出せます（`m[1]`・`m.group(1)`・`$1` など）。`(…)?` のように、まとまりごとくり返しや省略もできます。	第9巻 UNIT 09
`\d`	正規表現の `\d` は、「数字 1 つ（0〜9）」を表します。	第11巻 UNIT 05
`\.`	正規表現の `\.` のように、記号の前に `\` を付けると、その記号を特別な意味にせず、文字そのものとして表します。`\.` は `.`、`\/` は `/` です。	第11巻 UNIT 05
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	第11巻 UNIT 05
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	第11巻 UNIT 05
`\d`	正規表現の `\d` は、「数字 1 つ（0〜9）」を表します。	第12巻 UNIT 04
`.`	正規表現の `.` は、「どんな 1 文字でも」を表します。`.` という文字そのものを表したい時は、`\.` と書きます。	第12巻 UNIT 04

Vec（並び）

コード	意味と使い方	初めて出てくる所
`vec!`	`Vec`（ベクタ：同じ種類の値を、順番に並べて持つ入れ物）を、中身を書いて作るマクロ。`vec![80, 95, 70]` は、80・95・70 の 3 つが入った並びです。`scores[0]` で最初の 80 を取り出せます（番号は 0 から数えるので、3 つなら使える番号は 0・1・2）。範囲の外の番号（`scores[10]` など）を使うと、プログラムは止まります（パニック）。あとから `push` で足したり、`len()` で数を数えたりできます（第2巻 UNIT 06）。	第0巻 UNIT 01
`[a, b]`	`[ "Cargo.toml", "src/main.rs" ]` のように `[ ]` の中に `,` で区切って値を並べると、「配列」（決まった数の値の並び）ができます。`for x in 配列 { … }` で 1 つずつ取り出せます。あとから数を増やしたい時は、`vec![…]` で作る `Vec` を使います。	第0巻 UNIT 02
`.len()`	中身の数（長さ）を返すメソッド。`Vec` なら入っている個数、`String`・`&str` ならバイト数です。日本語は 1 文字が 3 バイトなので、文字の数を数える時は `.chars().count()` を使います。	第0巻 UNIT 06
`Vec`	同じ種類の値を、順番に並べて持つ入れ物（ベクタ）。`Vec<i32>` は「整数の並び」、`Vec<String>` は「文字列の並び」です。`vec![…]` か `Vec::new()` で作り、`push` で足し、`len()` で数を数え、`v[0]` や `v.get(0)` で取り出します（第2巻 UNIT 06）。	第0巻 UNIT 07
`&[T]`	`&[User]` のような `&[型]` は、「並び（`Vec` や配列）の全部か一部を、借りて見る」型です（スライス）。関数の引数を `&[User]` にすると、`Vec<User>` も配列も渡せます。	第0巻 UNIT 08
`.push()`	後ろに 1 つ足すメソッド。`Vec` なら `v.push(10);` で 10 を足し、`String` なら `s.push('!');` で 1 文字足します。`mut` の付いた変数に使います。	第2巻 UNIT 05
`.get()`	番号やキーで中身を取り出すメソッド。`v.get(10)` は、範囲の外なら止まらずに `None` を返します（`v[10]` はプログラムが止まる）。文字列の `s.get(0..3)` は、文字の区切りでない位置なら `None` です。`HashMap` の `map.get("キー")` も、無ければ `None` です。	第2巻 UNIT 05
`[値; 個数]`	`[0; 5]` は「0 を 5 個並べた配列」（`[0, 0, 0, 0, 0]`）を作る書き方です。型の所に書く `[u8; 32]` は、「`u8` が 32 個の配列」という型です。	第2巻 UNIT 06
`.pop()`	`Vec` の最後の 1 つを取り出すメソッド。空なら `None` を返すので、結果は `Option` です。`String` なら最後の 1 文字を取り出します。	第2巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`.sort()`	`Vec` の中身を、小さい順（文字なら辞書の順）に並べかえるメソッド。新しい `Vec` は作らず、その `Vec` 自身を並べかえるので、`mut` が要ります。	第2巻 UNIT 06
`.remove()`	`Vec` から i 番目を取り除いて、その値を返すメソッド。後ろの物が前に詰められます。`HashMap` の `.remove(キー)` は、そのキーの組を取り除き、値を `Option` で返します。	第2巻 UNIT 06
`Vec::new()`	空の `Vec` を作る関数。あとで `push` で足すので、`let mut v = Vec::new();` のように `mut` を付けます。中身の型は、足す物から Rust が推し量ります。	第2巻 UNIT 06
`.capacity()`	`Vec` が、今の場所のまま入れられる数（容量）を返すメソッド。`len()`（今入っている数）とは別で、足りなくなると自動で広げられます。	第2巻 UNIT 06
`Vec::with_capacity()`	「これくらい入れる予定」と容量を決めて、空の `Vec` を作る関数。たくさん足すと分かっている時に、広げ直す手間を減らせます。	第2巻 UNIT 06
`.sort_by_key()`	「何で並べるか」を関数で決めて、`Vec` を並べかえるメソッド。`notes.sort_by_key(|note| note.id)` は、番号の小さい順にします。	第2巻 UNIT 10
`.clear()`	中身を全部消して、空にするメソッド。`String` にも `Vec` にも使えます。	第3巻 UNIT 04
`.extend()`	`Vec` の後ろに、別の並びの中身をまとめて足すメソッド。`v.extend(extra);`。	第3巻 UNIT 07
`.retain()`	`Vec` の中から、条件に合う物だけを残すメソッド（合わない物は消える）。`scores.retain(|&s| s >= 60);` は 60 点以上だけを残します。	第3巻 UNIT 07
`.last()`	並びの最後の 1 つを `Option` で返すメソッド（空なら `None`）。最初の 1 つは `.first()`。	第4巻 UNIT 06
`.extend_from_slice()`	`Vec` の後ろに、別の並び（スライス）の中身を写して足すメソッド。受け取ったバイトを、ためておく時などに使います。	第5巻 UNIT 08
`.windows()`	並びから、「隣どうし n 個ずつ」を、1 つずつずらしながら取り出すメソッド。`buffer.windows(4)` は、4 バイトずつの組を順に見ます（区切りの `\r\n\r\n` を探すのに使う）。	第5巻 UNIT 08
`.to_vec()`	並び（スライス）の中身を写して、新しい `Vec` を作るメソッド。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.dedup()`	`Vec` の中で、隣どうしで同じ値が続く所を 1 つにまとめるメソッド。先に `sort` しておくと、全部の重複が消えます。	第5巻 UNIT 10
`.drain()`	`Vec` から、決めた範囲の中身を取り出して（元からは消して）、1 つずつ渡すメソッド。`names.drain(..3)` は、先頭の 3 つを取り出します。	第7巻 UNIT 10
`.as_slice()`	`Vec` や配列を、「借りた一続きの並び」（スライス `&[T]`）として見るメソッド。	第8巻 UNIT 10
`.first()`	並びの最初の 1 つを `Option` で返すメソッド（空なら `None`）。最後の 1 つは `.last()` です。	第10巻 UNIT 08
`.reverse()`	`Vec` の順番を逆にするメソッド（その `Vec` 自身を変える）。	第14巻 B. サンプル集
`.sort_by()`	「2 つを比べる関数」を渡して、`Vec` を並べかえるメソッド。`v.sort_by(|a, b| b.cmp(a))` は大きい順。	第14巻 B. サンプル集
`.chunks()`	並びを、決めた数ずつのかたまりに分けて取り出すメソッド。`[1,2,3,4,5].chunks(2)` は `[1,2]`・`[3,4]`・`[5]`。	第14巻 B. サンプル集
`.split_at()`	並びを、決めた位置で 2 つに分けるメソッド。`data.split_at(3)` は、前の 3 つと残りに分けます。	第14巻 B. サンプル集

HashMap（表）

コード	意味と使い方	初めて出てくる所
`.insert()`	`HashMap` に「キーと値」の組を入れるメソッド。`stock.insert("りんご", 3)`。同じキーがすでにあれば、値を上書きします（前の値は `Some(前の値)` で返ります）。`Vec` の `.insert(位置, 値)` は、決めた位置に差し込みます。	第0巻 UNIT 09
`HashMap::new()`	空の `HashMap`（名前で引ける表）を作る関数。あとで `insert` で中身を入れるので、`let mut map = HashMap::new();` のように `mut` を付けます。	第0巻 UNIT 09
`HashMap`	「キー → 値」の組を持つ入れ物（名前で引ける表）。`map.insert("りんご", 3)` で入れ、`map.get("りんご")` で引きます。キーが無いこともあるので、`get` は `Option` を返します。使うには `use std::collections::HashMap;` が要ります。	第0巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.contains_key()`	`HashMap` に、そのキーがあれば `true` を返すメソッド。	第2巻 UNIT 10
`.keys()`	`HashMap` のキーだけを、1 つずつ取り出せる形で返すメソッド。順番は決まっていないので、並べたい時は `Vec` に集めて `sort` します。	第2巻 UNIT 10
`.entry()`	`HashMap` の、そのキーの場所（あるか無いか）を取り出すメソッド。`map.entry(キー).or_insert(0)` のように、「無ければ入れる」と組み合わせて使います。	第2巻 UNIT 10
`.or_insert()`	`map.entry(キー).or_insert(0)` の形で、「キーが無ければ 0 を入れる。そして、その値の場所を返す」メソッド。`*map.entry(word).or_insert(0) += 1;` は「数える」時の決まり文句です。	第2巻 UNIT 10
`BTreeMap`	キーの順（小さい順・辞書の順）に並んでいる「キー → 値」の表。使い方は `HashMap` とほぼ同じで、回すと必ずキーの順に出てきます。	第2巻 UNIT 10
`BTreeMap::new()`	空の `BTreeMap`（キーの順に並ぶ表）を作る関数。	第2巻 UNIT 10
`.get_mut()`	`HashMap` や `Vec` から、書きかえるために中身を借りるメソッド。キー（番号）が無い時は `None` です。`if let Some(n) = map.get_mut("a") { *n += 1; }` のように使います。	第2巻 UNIT 10
`.values()`	`HashMap` の値だけを、1 つずつ取り出せる形で返すメソッド（順番は決まっていない）。	第2巻 UNIT 10
`HashSet::new()`	空の `HashSet`（重ならない値の集まり）を作る関数。	第8巻 UNIT 09
`HashSet`	同じ値は 1 つしか入らない集まり（値だけの `HashMap`）。重複を消す・「もう見たか」を覚える時に使います。`set.insert(x)` は、新しく入った時だけ `true` を返します。	第8巻 UNIT 09
`.or_default()`	`map.entry(キー).or_default()` は、キーが無ければ、その型の初期値（空の `Vec` など）を入れて、その場所を返すメソッド。	第9巻 UNIT 04
`BTreeSet`	同じ値は 1 つしか入らず、しかも小さい順に並んでいる集まり（`HashSet` の並ぶ版）。	第9巻 UNIT 04
`BTreeSet::new()`	空の `BTreeSet`（重ならず、小さい順に並ぶ集まり）を作る関数。	第9巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`BTreeSet::from()`	配列などから、`BTreeSet` を作る関数。`BTreeSet::from(["a", "b"])`。	第9巻 UNIT 04

範囲

コード	意味と使い方	初めて出てくる所
`..=`	「〜から〜まで（最後も含む）」の範囲。`1..=100` は 1, 2, ..., 100。`..` だけだと最後を含みません（`1..100` は 1〜99）。`for n in 1..=100 { … }` のように、くり返しでよく使います。	第0巻 UNIT 01
`a..b`	`0..5` は「0 から 5 の手前まで」（0, 1, 2, 3, 4）の範囲です。`for i in 0..5` で 5 回くり返します。5 も含めたい時は `0..=5` と書きます。`&s[0..3]` のように、文字列や並びの一部を取り出す時にも使い、`..3`（最初から）・`2..`（最後まで）と片方を省けます。	第0巻 UNIT 05

条件

コード	意味と使い方	初めて出てくる所
`if`	条件によって処理を分ける書き方。`if 条件 { 合う時 } else { 合わない時 }`。条件は `true`・`false` になる式（`age >= 20` など）で、`( )` で囲む必要はありませんが、`{ }` は必ず要ります。	第0巻 UNIT 02
`else`	`if` の条件が合わなかった時の処理を書く所。`else if 条件 { … }` で、条件をいくつも続けられます。	第0巻 UNIT 02
`if let`	`if let Some(x) = 値 { … }` は、「値が `Some` の形なら、中身を `x` に入れて `{ }` の中を動かす」という書き方です。`match` で 1 つの形の時だけ書きたい時の、短い形です。`else { … }` も付けられ、`if let Ok(n) = s.parse::<i32>()` のように `Result` にも使えます。	第0巻 UNIT 05
`==`	`a == b` は「a と b が等しいか」を調べ、`true` か `false` になります。`=`（1 つ）は「値を入れる」なので、比べる時は必ず 2 つ書きます。	第0巻 UNIT 05
`||`	`a || b` は「a か b の、どちらかが true なら true」（または）です。`if n == 0 || line == "\r\n"` は「0 か、空の行なら」。左が true なら、右は調べません。	第0巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`&&`	`a && b` は「a と b の両方が true なら true」（かつ）です。左が false なら、右は調べません。	第0巻 UNIT 08
`bool`	`true`（はい）か `false`（いいえ）の、どちらかだけが入る型です。`if` の条件や、`5 > 3` のように比べた結果は `bool` です。	第0巻 UNIT 10
`!`	`!valid` のように値の前に付けた `!` は、「〜ではない」（true と false を入れかえる）です。`if !path.exists()` は「無かったら」。`println!` の `!` はマクロの印で、これとは別物です。	第0巻 UNIT 10
`<=`	`a <= b` は「a が b 以下」、`a >= b` は「a が b 以上」です。`<`（より小さい）・`>`（より大きい）と合わせて、比べた結果は `true` か `false` になります。	第2巻 UNIT 03
`!=`	`a != b` は「a と b が等しくないか」を調べます。等しくなければ `true` です。	第3巻 UNIT 03

くり返し

コード	意味と使い方	初めて出てくる所
`for`	並びや範囲から 1 つずつ取り出して、くり返す書き方。`for n in 1..=100 { … }` は、`n` に 1, 2, ..., 100 を順に入れて、中を 100 回動かします。`for x in &v` で、`Vec` の中身を借りて回せます。	第0巻 UNIT 01
`in`	`for 名前 in 並び` の一部で、「この並びの中から」という意味です。	第0巻 UNIT 01
`loop`	終わりのないくり返し。`loop { … }` の中で `break` を書いた所で抜けます。サーバーが「次の接続を待つ」のをずっと続ける時などに使います。	第0巻 UNIT 06
`break`	`loop`・`while`・`for` のくり返しを、途中で抜ける命令。	第0巻 UNIT 06
`while`	条件が `true` の間、くり返す書き方。`while 条件 { … }`。条件が最初から `false` なら、1 回も動きません。	第2巻 UNIT 01
`'名前: loop`	`'outer: for …` のように、くり返しの前に `'名前:` を付けると、くり返しに名前（ラベル）が付きます。内側のくり返しから `break 'outer;` と書くと、外側のくり返しまで一気に抜けられます。	第2巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`continue`	くり返しの「今回の残り」を飛ばして、次の回へ進む命令。`for` や `while` の中で、条件に合わない物を飛ばす時に使います。	第2巻 UNIT 04
`while let`	`while let Some(c) = cause { … }` は、「`Some` の形である間、中身を `c` に入れてくり返す」書き方です。`None` になったら止まります。`while let Some(x) = stack.pop()` のように、並びから 1 つずつ取り出す時によく使います。	第4巻 UNIT 05

場合分け

コード	意味と使い方	初めて出てくる所
`match`	値の形に合わせて、処理を分ける書き方。`match 値 { 形1 => 処理1, 形2 => 処理2 }` と書き、上から順に合う物を探します。全部の場合を書かないとコンパイルが通らないので、書き忘れを防げます。`_ =>` は「それ以外全部」です。	第0巻 UNIT 01
`=>`	`match` の中で「この形なら、→ この処理」をつなぐ矢印。`Some(name) => println!("{name}"),` は「値があったら、その名前を表示」という意味です。	第0巻 UNIT 01
`パターン if 条件 =>`	`match` の腕に `None if scheme == "https" =>` のように `if 条件` を付けると、「形が合って、しかも条件も合う時だけ」その腕を選びます（マッチガード）。条件に合わなければ、次の腕を調べます。	第0巻 UNIT 05
`_ =>`	`match` の最後の `_ =>` は、「ここまでのどれにも当てはまらない、その他すべて」を受け止める腕です。`_` は「何でもよい（名前を付けない）」という印です。`match` は、ありうる値を全部受け止めないとエラーになるので、最後によく書きます。	第0巻 UNIT 06
`A | B =>`	`match` の腕の `A | B =>` は、「A か B のどちらかの形なら」という意味で、同じ処理をまとめて書けます。`1 | 2 | 3 => "小さい"` のように使います。	第2巻 UNIT 04
`_`	`(0, _)` や `Err(_)` の `_` は、「ここの値は何でもよい・使わない」という印です。名前を付けないので、「使っていない」という警告も出ません。	第2巻 UNIT 04
`matches!`	値が、ある形（パターン）に合うかを `true`・`false` で返すマクロ。`matches!(filter, Filter::Active | Filter::Done)` のように、`match` を 1 行で書けます。	第2巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`名前 @ パターン`	`n @ 1..=9 =>` のように、`match` の形の前に `名前 @` を付けると、「この形に合った値を、`n` という名前で受け取る」ことができます。範囲で調べつつ、その値も使いたい時に書きます。	第2巻 UNIT 08
`..`	`Point { x, .. }` や `(first, ..)` の `..` は、「残りの項目は何でもよい（見ない）」という印です。項目の多い構造体から、使う物だけを取り出す時に書きます。	第2巻 UNIT 08

Option

コード	意味と使い方	初めて出てくる所
`Some`	`Option`（値が「ある」か「無い」か）の、「ある」の方。`Some(値)` の形で中身を包みます。`match` で `Some(name) => { … }` と書くと、中身を `name` という名前で取り出して使えます。「無い」の方は `None` です。	第0巻 UNIT 01
`None`	`Option` の「値が無い」側。`Some(値)`（値がある）か `None`（無い）のどちらかになります。Rust には「null」が無い代わりに、無いかもしれない値は必ず `Option` で表し、`match` などで両方の場合を書きます。	第0巻 UNIT 01
`Option`	「値があるかもしれないし、無いかもしれない」を表す型。中身は `Some(値)` か `None`。`Option<u32>` は「整数があるかもしれない」。取り出す時は `match`・`if let`・`unwrap_or` などを使います（第2巻 UNIT 08）。	第0巻 UNIT 08
`let … else`	`let Some(x) = 値 else { return …; };` は、「`Some` の形なら中身を `x` に入れて先へ進み、そうでなければ `else` の中で抜ける」という書き方です。`else` の中は、必ず `return` や `continue` などで抜けます。思ったとおりでない時に早めに抜けて、残りの処理を平らに書けます。	第0巻 UNIT 08
`.is_some()`	`Option` が `Some`（値がある）なら `true` を返すメソッド。値そのものは要らず、あるかどうかだけ知りたい時に使います。	第2巻 UNIT 08
`.copied()`	借りている値（`&i32` など）を、写した値（`i32`）にするメソッド。`map.get("a").copied()` は `Option<&i32>` を `Option<i32>` にします。	第2巻 UNIT 10
`.is_none()`	`Option` が `None`（値が無い）なら `true` を返すメソッド。	第2巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`.cloned()`	借りている値を、`.clone()` で複製した値にするメソッド。`Option<&String>` を `Option<String>` に、イテレーターなら借りた物の並びを複製の並びにします。	第4巻 UNIT 02
`.then_some()`	`bool` が `true` なら `Some(値)`、`false` なら `None` を返すメソッド。`on_about.then_some("page")`。Topcoat の `view!` では、属性に `None` を入れると、その属性ごと書かれません。	第6巻 UNIT 03
`.then()`	`bool` が `true` の時だけ、渡した関数を動かして `Some(結果)` を返すメソッド（`false` なら `None`）。`then_some` と違い、値を作る処理が、必要な時だけ動きます。（Future の `.then` は別の物です。）	第6巻 UNIT 07
`.is_some_and()`	`Option` に値があり、しかもその値が条件に合う時だけ `true` を返すメソッド。`cookie.is_some_and(|c| c == "admin=yes")`。	第6巻 UNIT 09
`.map_or()`	`Option` に値があれば、それを関数で変換し、無ければ決めた値を返すメソッド。`depth.get(&k).map_or(0, |p| p + 1)` は「あれば +1、無ければ 0」。	第7巻 この巻で身に付けること
`.flatten()`	入れ子になった物を、1 段平らにするメソッド。`Option<Option<T>>` を `Option<T>` にし、イテレーターなら「並びの並び」を 1 つの並びにします。	第13巻 UNIT 07

エラー処理

コード	意味と使い方	初めて出てくる所
`Result`	「うまくいった（`Ok(値)`）」か「失敗した（`Err(エラー)`）」かのどちらかを入れる型。失敗するかもしれない関数は、`Result` を返します。`fn main() -> std::io::Result<()>` は「`main` も、入出力の失敗を返すことがある」という意味で、こう書くと中で `?` を使えます。受け取った側は `match`・`?`・`.unwrap()` などで中身を取り出します（くわしくは第2巻・第4巻）。	第0巻 UNIT 02
`Ok`	`Result` の「うまくいった」の方。`Ok(値)` の形で結果を包みます。`Ok(())` は「うまくいった。返す値は特に無い（`()` は空っぽの値）」という意味で、`Result<()>` を返す関数の最後によく書きます。	第0巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`?`	`Result` や `Option` の後ろに付ける記号。「成功なら中身を取り出して先へ進む。失敗なら、その場で関数を終えて、エラーを呼び出し元へ返す」という意味です。`let text = fs::read_to_string("a.txt")?;` のように使います。`?` を使える関数は、戻り値が `Result`（または `Option`）の関数だけです。	第0巻 UNIT 02
`.unwrap_or()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、代わりに決めた値を使うメソッド。`name.unwrap_or("名無し".to_string())` のように使います。止まらない安全な取り出し方です。	第0巻 UNIT 03
`.unwrap()`	`Option` や `Result` から中身を取り出すメソッド。中身が無い（失敗した）時は、その場でプログラムが止まります（パニック）。練習や「絶対に失敗しない」と分かっている所だけで使い、本番のコードでは `?` や `unwrap_or` を使います。	第0巻 UNIT 04
`Err`	`Result` の「失敗した」の方。`Err(エラー)` の形で、何が起きたかを包みます。`match` で `Err(e) => …` と書くと、エラーを `e` で受け取れます。`?` を付けると、`Err` の時はその場で呼び出し元へ返します。	第0巻 UNIT 04
`.unwrap_or_default()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、その型の「空の値」（文字列なら `""`、数なら 0）を使うメソッド。	第0巻 UNIT 07
`.expect()`	`Option` や `Result` から中身を取り出すメソッド。中身が無い（失敗した）時は、かっこの中の文を表示して、その場でプログラムが止まります（パニック）。`unwrap()` と同じですが、止まった理由が分かる文を付けられるので、`unwrap()` よりこちらがおすすめです。	第2巻 UNIT 01
`ParseIntError`	文字列を整数に直す（`parse`）のに失敗した時のエラーの型。`"12a".parse::<i32>()` は、この型の `Err` を返します。	第2巻 UNIT 09
`.ok_or()`	`Option` を `Result` に変えるメソッド。`Some(値)` は `Ok(値)` に、`None` は `Err(指定したエラー)` になります。`?` と組み合わせて「無ければエラーで返す」と書けます。	第2巻 UNIT 09
`.map_err()`	`Result` のエラーの方だけを、別の物に変えるメソッド。`.map_err(|_| ConfigError::BadPort)` で、元のエラーを自分のエラーの型に変えます。成功の値はそのままです。	第2巻 UNIT 09
`.ok()`	`Result` を `Option` に変えるメソッド。`Ok(値)` は `Some(値)` に、`Err(…)` は `None` になります（エラーの中身は捨てる）。`"42".parse().ok()` のように、「失敗したら無かったことにする」時に使います。	第2巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`Error`	エラーを表す型が守る決まりごと（トレイト）。`use std::error::Error;` と書くと、`.source()`（元になったエラー）などを使えます。`Box<dyn Error>` は「どんなエラーでも入る箱」です。	第4巻 UNIT 05
`#[error]`	`thiserror` の印（属性）。エラーの種類ごとに、表示する文を決めます。`#[error("タイトルが長すぎます（{0} 文字）")]` のように書くと、`Display` を自分で書かずに済みます。	第4巻 UNIT 05
`#[source]`	`thiserror` の印（属性）。「このエラーの元になったエラー」の項目に付けます。`.source()` や `anyhow` の原因の表示で、元のエラーまでたどれるようになります。	第4巻 UNIT 05
`#[from]`	`thiserror` の印（属性）。付けた項目の型のエラーから、自分のエラーへ自動で変換できるようにします（`From` の実装を作る）。これで `?` を使った時に、自動で自分のエラーに変わります。	第4巻 UNIT 05
`.source()`	エラーの「元になったエラー」を返すメソッド。`while let Some(e) = err.source()` のようにたどると、原因をさかのぼれます。	第4巻 UNIT 05
`std::error::Error::source()`	エラーの「元になったエラー」を返す関数（`.source()` と同じ）。`Option` で返るので、`while let Some(e) = cause` のようにたどって、原因をさかのぼれます。	第4巻 UNIT 05
`std::io::Error::new()`	入出力のエラー（`std::io::Error`）を自分で作る関数。`std::io::Error::new(ErrorKind::Other, "説明")` のように、エラーの種類と説明を渡します。	第4巻 UNIT 05
`ErrorKind`	入出力のエラーの「種類」。`std::io::ErrorKind::NotFound`（無い）・`ConnectionRefused`（つなぐのを断られた）などがあり、`e.kind()` で調べて、種類ごとに処理を分けられます。	第4巻 UNIT 05
`ConnectionRefused`	`ErrorKind::ConnectionRefused` は、「相手のパソコンに、その番号で待っているプログラムがいない」ために、つなぐのを断られた、というエラーの種類です。	第4巻 UNIT 05
`#[error(transparent)]`	`thiserror` の `#[error(transparent)]` は、「自分では文を作らず、中に入れた元のエラーの文を、そのまま見せる」という印です。`#[from]` と組み合わせて、ほかのエラーを包むだけの種類に使います。	第4巻 UNIT 05
`bail!`	`anyhow` のマクロ。その場でエラーを作って、関数から返します。`bail!("ノートが 1 件もありません");` は `return Err(anyhow!("…"));` の短い書き方です。	第4巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`.with_context()`	`anyhow` のメソッド。失敗した時のエラーに、「何をしていて失敗したか」の説明を書き足します。`.with_context(|| format!("{path} を読めませんでした"))?`。ログを見た人が、原因をすぐ分かるようになります。	第4巻 UNIT 05
`.unwrap_or_else()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、渡した関数を動かして代わりの値を作るメソッド。`unwrap_or` と違い、代わりの値を作る処理が、必要な時だけ動きます。	第4巻 UNIT 05
`.context()`	`anyhow` の部品で、エラーに「何をしていた時のエラーか」の説明を足すメソッド。`fs::read_to_string(path).context("設定ファイルを読めません")?`。説明を `format!` で作る時は、必要な時だけ作る `.with_context(|| …)` を使います。	第4巻 UNIT 05
`Context`	`anyhow::Context` は、`with_context`・`context` メソッドを使えるようにする機能（トレイト）。`use anyhow::Context;` と書いておくと、`Result` や `Option` に説明を付けられます。	第4巻 UNIT 05
`.is_ok()`	`Result` が `Ok`（成功）なら `true` を返すメソッド。中身は要らず、成功したかだけ知りたい時に使います。	第4巻 UNIT 06
`.or_else()`	`Option` や `Result` が空（失敗）の時だけ、渡した関数で「別の `Option`・`Result`」を作って返すメソッド。「無ければ、別の所から探す」に使います。	第4巻 UNIT 09
`panic!`	その場でプログラム（今のスレッド）を止めるマクロ。`panic!("起きるはずのないことが起きました")`。直しようのない、プログラムのまちがいを知らせる時に使います。ふつうの失敗には `Result` を使います。	第5巻 UNIT 02
`.is_err()`	`Result` が `Err`（失敗）なら `true` を返すメソッド。	第5巻 UNIT 04
`.and_then()`	`Option` や `Result` に中身があれば、それを使って、次の `Option`・`Result` を作るメソッド（無ければ、そのまま `None`・`Err`）。失敗するかもしれない処理を、続けてつなぐ時に使います。	第5巻 UNIT 08
`.downcast_ref()`	いろいろな種類が入るエラー（`anyhow::Error` など）の中身が、指定した型なら、それを借りて `Some` で返すメソッド。「このエラーは〇〇の種類か」を調べる時に使います。	第5巻 UNIT 09
`.chain()`	`anyhow` のエラーの、原因を外側から順にたどれる形で返すメソッド。`error.chain()` で、説明を足した順に、元のエラーまで 1 つずつ見られます。（イテレーターの `.chain()` は「2 つの並びをつなぐ」別の意味です。）	第5巻 UNIT 09
`.ok_or_else()`	`Option` を `Result` に変えるメソッド。`None` の時に、渡した関数でエラーを作ります（必要な時だけ作る `ok_or` の版）。	第6巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`todo!`	「ここはまだ書いていない」という印のマクロ。翻訳は通りますが、動かしてここに来ると止まります（パニック）。練習問題で、自分で書く所に置いてあります。	第7巻 UNIT 07
`.err()`	`Result` から、エラーの方だけを `Option` で取り出すメソッド。`Err(e)` は `Some(e)` に、`Ok(…)` は `None` になります（`.ok()` の逆）。	第10巻 UNIT 02
`Forbidden`	`AppError::Forbidden` は、この本のアプリのエラーの種類の 1 つで、「その操作は許されていない」（403）を表します。	第10巻 UNIT 03

関数

コード	意味と使い方	初めて出てくる所
`fn`	関数（ひとまとまりの処理に名前を付けた物）を作る書き方。`fn main() { … }` の `main` は、プログラムを動かすと最初に動く特別な関数です。`fn add(a: i32, b: i32) -> i32 { a + b }` は「整数を 2 つ受け取って、整数を返す関数 `add`」。`{ }` の中が中身で、最後の `;` の無い式が戻り値になります。	第0巻 はじめに: 手を動かす準備
`->`	`fn main() -> std::io::Result<()>` の `->` は、「この関数が返す値の型」を書く印です。`fn add(a: i32, b: i32) -> i32` なら「`i32` を返す」。`->` が無い関数は、何も返しません（`()` という「空」を返します）。	第0巻 UNIT 02
`|x|`（クロージャ）	その場で作る小さな関数（クロージャ）。`|x| x * 2` は「`x` を受け取って 2 倍を返す関数」です。`| |` の間が引数です。並べかえの基準やエラーの時の処理などを、ほかの関数に渡す時によく使います（第2巻）。	第0巻 UNIT 06
`return`	関数を、その場で終えて値を返す命令。`return Err(…);` のように、途中で抜ける時に使います。Rust では、関数の最後の `;` の無い式が自動で戻り値になるので、最後の行では `return` を書かないのがふつうです。	第0巻 UNIT 07
`Fn`	「呼び出せる物（関数やクロージャ）」を表す型の書き方。`f: impl Fn(i32) -> i32` は「`i32` を受け取って `i32` を返す関数なら何でもよい」という引数です。	第2巻 UNIT 10
`move`	クロージャが使う外の変数を、借りるのではなく「持っていく」印。`thread::spawn(move || { … })` のように、別のスレッドに値を渡す時に必要です。	第3巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`FnOnce`	「1 回だけ呼べる関数（クロージャ）」を表す型の書き方。中で値を使い切る（`move` した値を渡してしまう）クロージャは、この形になります。	第5巻 UNIT 04
`FnMut`	「呼ぶたびに中の値を変えてよい関数（クロージャ）」を表す型の書き方。外の変数を書きかえるクロージャは、この形になります。	第8巻 UNIT 02

型とメソッド

コード	意味と使い方	初めて出てくる所
`::<型>`	`"42".parse::<i32>()` の `::<i32>` は、「どの型にするか」をその場で指定する書き方です（形が魚に見えるので turbofish と呼ばれます）。`let n: i32 = "42".parse().unwrap();` のように、受け取る変数の型で決めても同じです。`collect::<Vec<_>>()` もよく使います。	第0巻 UNIT 04
`self`	メソッドの中で「このもの自身」を表す名前。`impl` の中の `fn label(&self)` の `&self` は「自分を借りて読むだけ」、`&mut self` は「自分を書きかえる」という意味です。`use` の中の `self`（例: `use std::io::{self, Write};`）は「`io` そのもの」を表します。	第0巻 UNIT 07
`struct`	いくつかの値をまとめて、新しい型を作る書き方（構造体）。`struct Note { id: u32, title: String }` で、「番号と題名を持つノート」という型ができます。`Note { id: 1, title: "買い物".to_string() }` で 1 つ作ります（第2巻）。	第0巻 UNIT 08
`impl`	2 つの使い方があります。① `impl 型 { … }` で、その型にメソッド（関数）を付ける。② `impl View` のように型の場所に書くと、「`View` の機能を持つ何かの型」という意味になります（戻り値の型を細かく書かずに済む）。	第1巻 UNIT 08
`.build()`	設定を少しずつ足していく「組み立て係」（ビルダー）の最後に呼んで、でき上がった物を作るメソッド。`Router::builder()…build()` の形で、ルーターなどを作る時に使います。	第1巻 UNIT 08
`impl トレイト`	`-> Result<impl View>` の `impl View` は、「`View` という決まり（トレイト）を満たす、何かの型」という意味です。本当の型の名前は長くて書けないので、「`View` として使える物を返す」とだけ書きます。引数の `x: impl Display` なら、「表示できる物なら何でも受け取る」です。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.0`	`pair.0`・`pair.1` のように、タプル（`(1, "a")` のような、いくつかの値の組）は、`.` と番号（0 から数える）で中の値を取り出します。`struct Meters(f64);` のような、項目に名前の無い構造体でも `.0` を使います。	第2巻 UNIT 06
`#[derive]`	型に、よく使う機能を自動で付ける印（属性）。`#[derive(Debug, Clone, PartialEq)]` のように、付けたい機能の名前を並べます（例: `Debug` → `{:?}` で表示できる、`Clone` → `.clone()` できる）。	第2巻 UNIT 07
`Debug`	`{:?}` で中身を表示できる機能（トレイト）。`#[derive(Debug)]` を付けた型は、`println!("{:?}", 値)` で中身を確かめられます。	第2巻 UNIT 07
`..値`	`Note { title: …, ..note }` の `..note` は、「書かなかった残りの項目は、`note` と同じ値にする」という書き方です（構造体の更新）。`..Default::default()` なら、残りを初期値にします。`..` の後ろの物は、いつも最後に書きます。	第2巻 UNIT 07
`Self`	`impl` の中で、「今、関数を付けている型そのもの」を表す名前。`fn new(…) -> Self` は「この型の値を返す」という意味で、型の名前を何度も書かずに済みます。	第2巻 UNIT 07
`Self`	`impl` の中で、「今、関数を付けている型そのもの」を表す名前（`-> Self`・`Self { … }` など）。	第2巻 UNIT 07
`Clone`	`.clone()` で値を丸ごと複製できる機能（トレイト）。`#[derive(Clone)]` で付けます。	第2巻 UNIT 07
`Copy`	代入や関数に渡した時に、移る（ムーブ）のではなく、自動で写される機能（トレイト）。数や `bool` のような小さな値が持っています。`#[derive(Clone, Copy)]` で自分の型にも付けられます（中身が全部 `Copy` の時だけ）。	第2巻 UNIT 07
`PartialEq`	`==` と `!=` で比べられる機能（トレイト）。`#[derive(PartialEq)]` で付けます。	第2巻 UNIT 07
`enum`	「いくつかの種類のうち、どれか 1 つ」を表す型（列挙型）。`enum Filter { All, Active, Done }` のように種類を並べ、`match` で種類ごとに処理を分けます。種類ごとに違うデータを持たせることもできます（第2巻）。	第2巻 UNIT 08
`Display`	`{}` で表示できる機能（トレイト）。利用者に見せる形を自分で決めたい時に、`impl fmt::Display for 型 { fn fmt(…) { … } }` と書いて実装します（`Debug` は開発者向けの表示）。	第2巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`Formatter`	`Display` や `Debug` の実装の中で受け取る「書き込み先」。`write!(f, "…")` で、そこに表示する文を書きます。	第2巻 UNIT 09
`Eq`	`==` の比べ方が、いつも筋が通っている（自分と比べると必ず等しい）ことを表す印（トレイト）。`HashMap` のキーにする型などに要ります。`PartialEq` と一緒に `#[derive]` で付けます。	第2巻 UNIT 10
`PartialOrd`	`<`・`>` などで大小を比べられる機能（トレイト）。`#[derive(PartialOrd)]` で付けます。	第2巻 UNIT 10
`Ord`	いつでも大小がはっきり決まる比べ方を持つ機能（トレイト）。`sort` や `BTreeMap` のキーに使う型に要ります。	第2巻 UNIT 10
`Self::default()`	その型の「初期値」（`Default` の値）を作る書き方。`Self` は「今 `impl` している型そのもの」を表します。	第2巻 UNIT 10
`Default`	「初期値（空の値）を作れる」機能（トレイト）。`#[derive(Default)]` を付けると、数は 0、文字列は `""`、`bool` は `false` の初期値を作れます。	第2巻 UNIT 10
`<T>`	`fn longest<'a>(…)` や `struct Wrapper<T>` の、名前のすぐ後ろの `< >` は、「型（やライフタイム）を、使う時に決める」ための入れ物（ジェネリクス）です。`T` は「何かの型」の仮の名前で、使う時に `i32` や `String` などに決まります（第4巻で、くわしく学びます）。	第3巻 UNIT 05
`trait`	「この型は、この関数を持っています」という約束（トレイト）を決める書き方。`trait Notifier { fn send(&self, msg: &str); }` と決めて、`impl Notifier for 型 { … }` で、それぞれの型が約束を守ります。	第3巻 UNIT 08
`dyn`	「このトレイトを持つ、何かの型」を表す印。`Box<dyn Notifier>` は「`Notifier` を持つ何か（中身の型は動かしてみるまで分からない）を指す箱」です。違う型を 1 つの `Vec` に入れる時に使います。	第3巻 UNIT 08
`type`	型に別の名前を付ける書き方（型の別名）。`type DynRepo = Arc<dyn NoteRepository + Send + Sync>;` のように、長い型を短く呼べます。トレイトの中の `type Err = IdError;` は「このトレイトで使うエラーの型は `IdError`」と決める書き方です。	第4巻 UNIT 01
`FromStr`	「文字列から作れる」機能（トレイト）。`impl FromStr for 型 { … }` を書くと、`"42".parse::<型>()` で、文字列から自分の型を作れるようになります。	第4巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`<_>`	`Vec<_>` の `_` は、「中の型は、Rust が使い方から決めて」という印です。`.collect::<Vec<_>>()` は「`Vec` に集めて。中身の型は任せる」という意味です。	第4巻 UNIT 01
`T: トレイト`	`<T: Display>` の `: Display` は、「`T` は、`Display`（表示できる）という決まりを満たす型だけ」という条件です（トレイト境界）。`where T: Display` と後ろに書いても同じです。	第4巻 UNIT 03
`A + B`	`T: Clone + Debug` の `+` は、「`Clone` と `Debug` の両方を満たす」という意味です。条件をいくつも並べられます。	第4巻 UNIT 03
`where`	ジェネリクス（型を後から決める書き方）の条件を、関数の後ろにまとめて書く所。`where T: Display, I: IntoIterator<Item = T>` のように、条件が長い時に読みやすくなります。	第4巻 UNIT 03
`Sized`	「大きさ（メモリで使う量）が、翻訳の時に決まっている」ことを表す印（トレイト）。`where Self: Sized` は「`dyn` 越しには呼べない」メソッドにする時の書き方です。ふだんは気にしなくて大丈夫です。	第4巻 UNIT 04
`From`	「ある型から作れる」機能（トレイト）。`impl From<RepoError> for AppError` を書くと、`RepoError` を `AppError` に変えられます。`?` は、エラーを返す時にこの変換を自動で使います。	第4巻 UNIT 05
`TryFrom`	「ある型から作れるが、失敗することもある」変換の機能（トレイト）。`impl TryFrom<CreateNoteInput> for NewNote` を書くと、`NewNote::try_from(input)` で、入力を確かめながら変換でき、だめなら `Err` を返します。	第4巻 UNIT 08
`.into()`	値を、受け取る側が求める型に変えるメソッド（`From` の実装を使う）。`let s: String = "abc".into();` のように、変換先の型が分かる所で使います。	第4巻 UNIT 09
`#[must_use]`	「この戻り値を使わずに捨てたら、警告を出して」という印（属性）。結果の確かめ忘れを防ぎます。	第4巻 UNIT 10
`Builder`	「組み立て係」（ビルダー）の型。設定をメソッドでつなげて決め、最後の `.build()` ででき上がりを作ります。tokio のランタイム・HTTP のクライアント・データベースの接続などで使われます。	第5巻 UNIT 02
`Into`	「ある型に変えられる」機能（トレイト。`From` の逆向き）。`body: impl Into<String>` は「`String` に変えられる物なら何でも（`&str` でも `String` でも）受け取る」という引数で、中で `.into()` を呼んで変えます。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`Fut`	ジェネリクス（型を後から決める書き方）で使う、型の名前の代わりの記号。`F: Fn() -> Fut, Fut: Future<Output = T>` は、「`F` は呼ぶと `Fut` を返す関数で、`Fut` は終わると `T` を返す Future」という意味です。名前は自由に付けられます。	第5巻 UNIT 08
`.try_into()`	値を、別の型に変えようとして、変えられなければ `Err` を返すメソッド（`TryFrom` の逆向き）。`bytes.try_into()` で、長さの決まった配列（`[u8; 32]` など）に変えます。	第8巻 UNIT 10
`NoteFormState::default()`	この巻で作る型 `NoteFormState`（入力フォームの中身と、まちがいの知らせ）を、空の状態で作る書き方。最初にフォームを表示する時に使います。	第10巻 UNIT 01
`unsafe`	「ここはコンパイラーが安全を確かめきれないので、書いた人が責任を持つ」という印。メモリを直接扱う所などで使います。ふつうのアプリのコードでは、ほとんど使いません。	第11巻 UNIT 08
`AsRef`	「借りた形（`&Path` など）に変えられる」機能（トレイト）。引数を `path: impl AsRef<Path>` にすると、`&str` でも `String` でも `PathBuf` でも受け取れます。	第13巻 UNIT 02
`.db_dir()`	この本のテストの手伝いの係が持つ、テスト用のデータベースを置く一時フォルダの場所を返すメソッド。	第13巻 UNIT 10

モジュール

コード	意味と使い方	初めて出てくる所
`use`	ほかの場所にある道具に、短い名前で呼べるようにする宣言。`use std::env;` と書くと、`std::env::args()` を `env::args()` と短く書けます。`std` は Rust に最初から付いている道具箱（標準ライブラリ）です。	第0巻 UNIT 01
`::`	`std::env` の `::` は、「〜の中の」を表す区切りです。`std::env::args()` は「標準ライブラリ（`std`）の中の `env` の中の `args` 関数」という意味です。型の関数（`String::new()`）や、enum の種類（`Ordering::Less`）を指す時にも使います。	第0巻 UNIT 01
`mod`	コードをまとめる箱（モジュール）を作る書き方。`mod tests { … }` のように、関係する関数をまとめます。`mod 名前;` と書くと、別のファイル（`名前.rs`）の中身を読み込みます。	第0巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`super`	「1 つ外側のモジュール」を表す名前。テストの中の `use super::*;` は、「外側（テストする本体）にある物を全部使えるようにする」という決まり文句です。	第0巻 UNIT 10
`::*`	`use super::*;` の `*` は、「その中の物を全部、短い名前で呼べるようにする」という意味です。テストの `mod tests` の中で、外側（`super`、1 つ上）の関数を全部使えるようにする時によく書きます。	第0巻 UNIT 10
`pub`	「外から使ってよい」（公開）という印。`pub fn title_length(…)` は、ほかのファイル（モジュール）からも呼べる関数になります。付けないと、同じモジュールの中からしか使えません。	第1巻 UNIT 06
`crate`	「このプロジェクト（クレート）のいちばん上」を表す言葉。`use crate::note::Note;` は「このプロジェクトの `note` モジュールの `Note` を使う」という意味です。	第2巻 UNIT 10
`pub(crate)`	`pub(crate)` は、「このクレート（このプロジェクト）の中からは使えるが、外からは使えない」という公開の範囲です。`pub` だけなら、どこからでも使えます。	第4巻 UNIT 06
`#[cfg(feature)]`	`#[cfg(feature = "markdown")]` は、「`markdown` という機能（feature）が ON の時だけ、この部分を翻訳する」という印です。機能は `Cargo.toml` の `[features]` で決め、`cargo run --features markdown` などで ON にします。	第4巻 UNIT 07
`#[cfg(unix)]`	`#[cfg(unix)]` は、「Linux や macOS（Unix の仲間）向けに翻訳する時だけ、この部分を使う」という印です。`#[cfg(windows)]` なら Windows の時だけです。OS ごとにちがう処理を書き分けます。	第5巻 UNIT 06
`#[cfg(not)]`	`#[cfg(not(unix))]` の `not( )` は、「その条件に合わない時だけ」という意味です。`#[cfg(not(test))]` なら、テストでない時だけ。`all( )`（全部に合う）・`any( )`（どれかに合う）もあります。	第5巻 UNIT 06
`#[path]`	`mod` で読み込むファイルの場所を、自分で指定する印（属性）。`#[path = "../src/note.rs"] mod note;` のように使います。	第11巻 UNIT 04
`#![…]`	`#![no_main]` のように `#!` で始まる印は、その印を書いたファイル（モジュール）や、クレート全体に付く印です。すぐ下の 1 つに付く `#[…]` とちがい、ファイルのいちばん上に書きます。	第11巻 UNIT 04

所有権と借用

コード	意味と使い方	初めて出てくる所
`&`	値の前の `&` は「貸す（借用）」の印です。`fs::write(&relative, …)` は、`relative` を手放さずに、見せるだけ貸します。型に付けた `&str`・`&Vec<i32>` は「借りた値」の型です（第3巻で、くわしく学びます）。	第0巻 UNIT 02
`&mut`	「書きかえてよい」という約束で値を貸す書き方。`read_to_string(&mut text)` は、関数に `text` を貸して、中身を書き込んでもらいます。貸す元の変数は `let mut text` のように `mut` が付いている必要があります（第3巻）。	第0巻 UNIT 05
`.clone()`	値を丸ごと複製するメソッド。`let b = a.clone();` で、`a` と `b` の両方を使えるようになります（`clone` しないと、`let b = a;` で中身が `b` に移り、`a` は使えなくなります。第3巻）。	第0巻 UNIT 09
`'_`	`Slot<'_>` の `'_` は、「何かを借りているが、その期間（ライフタイム）は Rust に任せる」という印です。名前（`'a`）を付けずに済みます。	第1巻 UNIT 08
`*`	`*price` のように、借りた値（参照）の前の `*` は「借りている先の、値そのもの」を表します。`*count += 1` は、借りた先の数を 1 増やします。かけ算の `*` とは別物です。	第2巻 UNIT 06
`'static`	`&'static str` の `'static` は、「プログラムが動いている間ずっと有効」という意味のライフタイムの印です。`"文字"` のように、コードに直接書いた文字列は `&'static str` です。	第2巻 UNIT 08
`Drop`	値が片づけられる（持ち主がいなくなって消える）時に、自動で動く処理を書くための機能（トレイト）。`impl Drop for 型 { fn drop(&mut self) { … } }` と書くと、片づけの時にその処理が動きます。ファイルを閉じる・鍵を返す、などに使われています。	第3巻 UNIT 01
`.as_ptr()`	値の本体がメモリのどこにあるか（アドレス）を返すメソッド。「同じ場所を指しているか」を見るための、仕組みの確かめ用です。ふだんのプログラムではあまり使いません。	第3巻 UNIT 02
`'a`	`&'a str` の `'a` は、「借りている期間」（ライフタイム）に付けた名前です。`fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` は、「返す参照は、`x` と `y` の両方が有効な間だけ使える」という約束を表します（第3巻で、くわしく学びます）。	第3巻 UNIT 05
`.as_ref()`	中身を借りた形に変えるメソッド。`Option<String>` を `Option<&String>` にするなど、中身を移さずに覗きたい時に使います。	第4巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.as_mut()`	中身を、書きかえられる形で借りるメソッド。`Pin` に使うと、ピン留めしたまま `poll` を呼べる形にします。	第5巻 UNIT 01

スマートポインタ

コード	意味と使い方	初めて出てくる所
`Box::new()`	値をヒープに置き、それを指す `Box` を作る関数。`Box::new(5)`。	第3巻 UNIT 08
`Box`	値をヒープ（大きな物を置く広い場所）に置き、その場所を指す「持ち手」を手元に置く型。`Box<dyn Notifier>` のように、違う型の物を同じ並びに入れる時によく使います（第3巻）。	第3巻 UNIT 08
`Rc::new()`	値をヒープに置き、何人もで持てる `Rc` を作る関数。持ち主の数は 1 から始まります。	第3巻 UNIT 09
`Rc::strong_count()`	`Rc` の本体を、今いくつの持ち手が指しているかを返す関数。0 になると本体が片づけられます。	第3巻 UNIT 09
`Rc::clone()`	`Rc` の持ち手を 1 つ増やす関数。本体は複製せず、「同じ物を指す持ち手」を作って、持ち主の数を 1 増やすだけなので、軽い操作です。	第3巻 UNIT 09
`Rc::ptr_eq()`	2 つの `Rc` が、同じ本体を指しているかを調べる関数。	第3巻 UNIT 09
`Rc`	1 つの値を、何人もで持てるようにする型（参照カウント）。持ち手を `Rc::clone` で増やし、全員がいなくなった時に本体が片づけられます。1 つのスレッドの中だけで使います（スレッドをまたぐ時は `Arc`）。	第3巻 UNIT 09
`.borrow_mut()`	`RefCell` の中身を、書きかえるために借りるメソッド。ほかで借りている最中に呼ぶと、その場で止まる（パニック）ので注意します。	第3巻 UNIT 09
`.borrow()`	`RefCell` の中身を、読むために借りるメソッド。	第3巻 UNIT 09
`.try_borrow_mut()`	`RefCell` の中身を書きかえるために借りられるかを、止まらずに確かめるメソッド。借りられなければ `Err` が返ります。	第3巻 UNIT 09
`RefCell::new()`	値を `RefCell`（動かしている時に借りるルールを確かめる入れ物）に入れる関数。	第3巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`RefCell`	「借りるルール」の確かめを、コンパイルの時ではなく、動かしている時に行う入れ物。`Rc<RefCell<T>>` の形で、「何人もで持って、しかも書きかえたい」時に使います（第3巻）。	第3巻 UNIT 09
`.upgrade()`	`Weak`（弱い持ち手）から、本体を使うためのふつうの持ち手（`Rc`）を取り出すメソッド。本体がもう片づけられていれば `None` です。	第3巻 UNIT 09
`Weak::new()`	まだ何も指していない、空の `Weak`（弱い持ち手）を作る関数。	第3巻 UNIT 09
`Rc::downgrade()`	`Rc` から、`Weak`（弱い持ち手）を作る関数。弱い持ち手は、本体を生かし続ける数に入りません。	第3巻 UNIT 09
`Rc::weak_count()`	本体を指している `Weak`（弱い持ち手）の数を返す関数。	第3巻 UNIT 09
`Weak`	本体を指すけれど、本体を生かし続ける数には入らない「弱い持ち手」。親と子が互いを指す時に、片方を `Weak` にして、いつまでも片づけられない（メモリが漏れる）のを防ぎます。	第3巻 UNIT 09
`.set()`	`Cell` の中身を、新しい値に置きかえるメソッド。`&self`（変えられない借り）しか無くても、中身を変えられます。読む時は `.get()` です。	第3巻 UNIT 09
`Cell::new()`	値を `Cell`（借りずに丸ごと入れかえられる入れ物）に入れる関数。	第3巻 UNIT 09
`Cell`	中身を、借りずに丸ごと入れかえられる入れ物。`Copy` できる小さな値（数など）を、`&` しか持っていない所から書きかえる時に使います。	第3巻 UNIT 09

イテレーター

コード	意味と使い方	初めて出てくる所
`.nth()`	並びの中から「n 番目」を 1 つ取り出すメソッド。番号は 0 から数えます。`env::args().nth(1)` は、コマンドの引数の 1 番目（0 番目はプログラム自身の場所）。無いかもしれないので、結果は `Some(値)` か `None`（`Option`）で返ります。	第0巻 UNIT 01
`.enumerate()`	並びを 1 つずつ取り出す時に、「何番目か」の番号を一緒に付けるメソッド。`for (i, x) in list.iter().enumerate()` で、`i` に 0, 1, 2…、`x` に中身が入ります。	第0巻 UNIT 03

コード	意味と使い方	初めて出てくる所
`.skip()`	並びの先頭から、決めた数だけ飛ばすメソッド。`env::args().skip(1)` は、0 番目（プログラム自身）を飛ばします。	第0巻 UNIT 03
`.count()`	イテレーター（1 つずつ取り出せる物）の中身を最後まで数えて、その数を返すメソッド。`env::args().skip(1).count()` は「渡された引数の数」、`s.chars().count()` は「文字の数」です。	第0巻 UNIT 03
`.take()`	イテレーターの先頭から、決めた数だけを取り出すメソッド。`body.chars().take(60)` は先頭の 60 文字。`.collect::<String>()` で文字列に戻せます。	第0巻 UNIT 05
`.collect()`	1 つずつ取り出せる形（イテレーター）の中身を、`Vec` や `String` などの入れ物に集めるメソッド。何に集めるかを、`let v: Vec<&str> = ….collect();` のように変数の型で教えます。	第0巻 UNIT 05
`.next()`	イテレーター（1 つずつ取り出せる物）から、次の 1 つを取り出すメソッド。無くなると `None` を返すので、結果は `Option` です。	第0巻 UNIT 06
`.iter()`	並び（`Vec` など）の中身を、借りて 1 つずつ取り出せる形（イテレーター）にするメソッド。`for x in v.iter()` や、`v.iter().map(…)`・`.filter(…)` のように、加工してから使う時の入り口です。	第0巻 UNIT 08
`.rev()`	並びを逆の順番で取り出すメソッド。`(1..=3).rev()` は 3, 2, 1 の順です。	第2巻 UNIT 04
`.step_by()`	並びから、決めた数おきに取り出すメソッド。`(0..10).step_by(3)` は 0, 3, 6, 9。	第2巻 UNIT 04
`.map()`	中身を 1 つずつ変換するメソッド。イテレーターなら `v.iter().map(|x| x * 2)` で全部を 2 倍に、`Option` なら `Some` の中身だけを変換します（`None` は `None` のまま）。`Result` なら `Ok` の中身だけを変換します。	第2巻 UNIT 08
`.into_iter()`	入れ物の中身を、借りずに「もらって」1 つずつ取り出す形にするメソッド。使ったあと、元の入れ物は使えなくなります（`.iter()` は借りるだけ）。	第2巻 UNIT 10
`.filter()`	条件に合う物だけを残すメソッド。`v.iter().filter(|n| **n % 2 == 0)` は偶数だけ。`|n|` の中の `n` は借りた値なので、`*` が付くことがあります。	第2巻 UNIT 10
`.sum()`	イテレーターの中身を全部足した合計を返すメソッド。`let sum: i32 = numbers.iter().sum();` のように、結果の型を書いておきます。	第2巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`.any()`	並びの中に、条件に合う物が 1 つでもあれば `true` を返すメソッド。全部が合うかは `.all()` で調べます。	第2巻 UNIT 10
`.all()`	イテレーターの中身が、全部条件に合うかを `true` / `false` で返すメソッド。`v.iter().all(|n| *n > 0)` は「全部 0 より大きいか」。1 つでも合うかは `.any()` です。	第2巻 UNIT 10
`.iter_mut()`	並びの中身を、書きかえられるように借りて、1 つずつ取り出すメソッド。`for v in values.iter_mut() { *v *= 2; }` のように、`*` を付けて中身を書きかえます。	第3巻 UNIT 03
`.position()`	並びの中で、条件に合う最初の物が「何番目か」を `Option` で返すメソッド。`notes.iter().position(|n| n.id == id)`。無ければ `None` です。	第3巻 UNIT 07
`IntoIterator`	「`for` で回せる物」の機能（トレイト）。`Vec`・配列・`HashMap` などが持っています。`I: IntoIterator<Item = T>` は「`T` が 1 つずつ出てくる、回せる物なら何でも」という条件です。	第4巻 UNIT 03
`Item`	イテレーターが 1 つずつ取り出す物の型（関連型）。`IntoIterator<Item = T>` は「`T` を 1 つずつ取り出せる物なら何でもよい」という意味です。	第4巻 UNIT 03
`Iterator`	「1 つずつ取り出せる物」（イテレーター）の機能（トレイト）。`map`・`filter`・`sum` などのメソッドは、全部この機能に付いています。`impl Iterator<Item = &Note>` は「`&Note` を 1 つずつ出す何か」を返す、という戻り値の書き方です。	第4巻 UNIT 03
`.filter_map()`	1 つずつ変換して、`None` になった物は捨て、`Some` の中身だけを残すメソッド。`.filter_map(|line| line.split_once('='))` は、`=` を含む行だけを「名前と値の組」にします。	第4巻 UNIT 09
`.zip()`	2 つの並びを、同じ番号どうしの組にして取り出すメソッド。`names.iter().zip(ages.iter())` は `(名前, 年齢)` の組を順に出します。短い方が終わった所で終わります。	第7巻 この巻で身に付けること
`.peekable()`	次の 1 つを、取り出さずに「覗く」（`peek`）ことができるイテレーターにするメソッド。文字を 1 つずつ読みながら、次の文字で処理を変える時に使います。	第7巻 この巻で身に付けること
`.peek()`	次の 1 つを、取り出さずに覗くメソッド（`peekable` にしたイテレーターで使う）。	第7巻 この巻で身に付けること
`.by_ref()`	イテレーターを借りたまま、途中まで使うメソッド。`for c in chars.by_ref()` のあとでも、残りを続けて読めます。	第7巻 この巻で身に付けること

コード	意味と使い方	初めて出てくる所
`.next_back()`	並びを後ろから 1 つ取り出すメソッド。`line[..at].chars().next_back()` は、その位置の 1 つ前の文字です。	第9巻 UNIT 04
`.fold()`	並びを 1 つずつ見ながら、1 つの値にまとめていくメソッド。`fold(最初の値, |ここまで, x| 次の値)`。合計・文字列をつなぐ、などに使えます。	第14巻 B. サンプル集
`.flat_map()`	1 つから、いくつもの物を作って、それを全部 1 つの並びに平らにつなぐメソッド。	第14巻 B. サンプル集
`.max_by_key()`	「何で比べるか」を関数で決めて、いちばん大きい物を探すメソッド。`people.iter().max_by_key(|p| p.age)`。	第14巻 B. サンプル集

ファイルとフォルダ

コード	意味と使い方	初めて出てくる所
`.display()`	ファイルやフォルダの場所（`Path`・`PathBuf`）を、`println!` で表示できる形にするメソッド。場所は、そのままでは `{}` で出せないので、`path.display()` を通して出します。	第0巻 UNIT 02
`.join()`	場所（パス）に、フォルダやファイルの名前をつなげた新しい場所を作るメソッド。`dir.join("notes.txt")` は、`dir` の中の `notes.txt` を表します。区切りの `\` は自動で入ります。（文字列の配列に使う `.join(", ")` は、「区切り文字でつなげて 1 つの文字列にする」別の意味です。）	第0巻 UNIT 02
`fs::create_dir_all()`	フォルダを作る関数。`fs::create_dir_all("data/notes")` のように、途中のフォルダもまとめて作ります。もうあってもエラーになりません。	第0巻 UNIT 02
`Path::new()`	文字列から、ファイルやフォルダの場所（`Path`）を作る関数。`Path::new("memo").join("today.txt")` のように、場所をつなげたり、名前や拡張子を取り出したりできるようになります。	第0巻 UNIT 02
`fs::write()`	ファイルに文字を書く関数。`fs::write("memo.txt", "こんにちは")` で、ファイルが無ければ作り、あれば中身を全部置きかえます。失敗することもあるので `Result` が返ります。	第0巻 UNIT 02
`fs::read_to_string()`	ファイルの中身を全部読んで、1 つの文字列（`String`）にする関数。ファイルが無い時などは `Err`（エラー）が返ります。	第0巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`Path`	ファイルやフォルダの場所を表す型（借りている形）。`.join()`・`.file_name()`・`.extension()`・`.exists()` などで扱います。自分の物として持つ形は `PathBuf` です。	第0巻 UNIT 02
`.file_name()`	場所（パス）から、最後の部分（ファイル名やフォルダ名）だけを取り出すメソッド。`C:\data\memo.txt` なら `memo.txt`。無いこともあるので `Option` で返ります。	第0巻 UNIT 02
`.extension()`	ファイル名の拡張子（`.txt` の `txt` の部分）を取り出すメソッド。拡張子が無い時は `None` が返ります。	第0巻 UNIT 02
`.parent()`	場所（パス）の、1 つ上のフォルダを取り出すメソッド。`C:\data\memo.txt` なら `C:\data`。	第0巻 UNIT 02
`.exists()`	その場所に、ファイルかフォルダがあれば `true` を返すメソッド。`Path::new("memo.txt").exists()`。	第0巻 UNIT 02
`.path()`	`fs::read_dir` で 1 つずつ取り出したフォルダの中身（`DirEntry`）から、その場所（`PathBuf`）を取り出すメソッド。`entry.path().is_dir()` でフォルダかどうか、`entry.path().display()` で表示用の文字にできます。	第0巻 UNIT 02
`.is_dir()`	その場所がフォルダなら `true`、ファイルなら `false` を返すメソッド。ファイルかどうかは `.is_file()` で調べます。	第0巻 UNIT 02
`fs::read_dir()`	フォルダの中の一覧（ファイルとフォルダ）を、1 つずつ取り出せる形で返す関数。順番は決まっていないので、並べたい時は `Vec` に集めて `sort` します。	第0巻 UNIT 02
`.create()`	`OpenOptions`（ファイルの開き方の設定）で、「ファイルが無ければ作る」と決めるメソッド。`OpenOptions::new().create(true).append(true).open(file)` は「無ければ作り、あれば後ろに書き足す」開き方です。	第0巻 UNIT 07
`.append()`	`OpenOptions::new().append(true)` で、「ファイルの後ろに書き足す」ように開く設定にします（指定しないと、最初から書き直します）。	第0巻 UNIT 07
`.open()`	`OpenOptions` で決めた開き方で、ファイルを開くメソッド。開けないこともあるので、結果は `Result` です（`?` で受け取ります）。	第0巻 UNIT 07
`OpenOptions::new()`	ファイルの開き方を決める係を作る関数。`.create(true).append(true).open("memo.txt")?` のように、設定をつなげて書いてから開きます。	第0巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`OpenOptions`	ファイルをどう開くか（読む・書く・書き足す・無ければ作る）を細かく決めてから開くための型。`OpenOptions::new().create(true).append(true).open("memo.txt")?` のように、設定をつなげて書きます。	第0巻 UNIT 07
`fs::remove_file()`	ファイルを消す関数。ごみ箱には入りません。ファイルが無い時はエラー（`Err`）が返ります。	第0巻 UNIT 07
`std::fs::read_to_string()`	ファイルの中身を全部読んで文字列にする関数（`fs::read_to_string` と同じ）。	第5巻 UNIT 07
`std::fs::write()`	ファイルに書く関数（`fs::write` と同じ）。無ければ作り、あれば中身を置きかえます。	第5巻 UNIT 07
`std::fs::remove_dir_all()`	フォルダを、中身ごと全部消す関数。ごみ箱を通さずに消えるので、場所をよく確かめて使います。	第5巻 UNIT 10
`std::fs::create_dir_all()`	フォルダを、途中のフォルダもまとめて作る関数（`fs::create_dir_all` と同じ）。	第5巻 UNIT 10
`std::fs::remove_file()`	ファイルを消す関数（`fs::remove_file` と同じ）。ごみ箱には入りません。	第7巻 UNIT 05
`std::fs::metadata()`	ファイルの情報（大きさ・日時など）を取り出す関数。`std::fs::metadata(path)?.len()` で、ファイルの大きさ（バイト）が分かります。	第7巻 UNIT 10
`std::fs::read_dir()`	フォルダの中の一覧を返す関数（`fs::read_dir` と同じ）。	第7巻 UNIT 10
`tempfile::tempdir()`	使い終わると自動で消える、一時的なフォルダを作る関数（`tempfile` の部品）。テストで、ファイルやデータベースを置く場所に使います。	第8巻 UNIT 01
`PathBuf`	ファイルやフォルダの場所（パス）を、自分の物として持つ型（`String` のパス版）。`path.join("a")` で場所をつなげ、`.push("b")` で足せます。借りるだけの形は `&Path` です。	第9巻 UNIT 04
`std::env::temp_dir()`	OS の、一時的なファイルを置くフォルダの場所を返す関数（Windows なら `C:\Users\you\AppData\Local\Temp`）。	第11巻 UNIT 02
`TempDir`	`tempfile` の部品の、一時的なフォルダの型。この値が片づけられると、フォルダも中身ごと自動で消えます。	第11巻 UNIT 02
`std::fs::create_dir()`	フォルダを 1 つ作る関数。途中のフォルダが無い・もうある時はエラーです（まとめて作るのは `create_dir_all`）。	第13巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`fs::OpenOptions::new()`	ファイルの開き方を決める係を作る関数（`OpenOptions::new()` と同じ）。	第14巻 B. サンプル集
`fs::metadata()`	ファイルの情報（大きさ・日時など）を取り出す関数。`fs::metadata(path)?.len()` で大きさが分かります。	第14巻 B. サンプル集
`fs::copy()`	ファイルをコピーする関数。`fs::copy(元, 先)?`。先にファイルがあれば上書きします。	第14巻 B. サンプル集
`fs::remove_dir_all()`	フォルダを、中身ごと全部消す関数。ごみ箱は通らないので、場所をよく確かめて使います。	第14巻 B. サンプル集

入出力

コード	意味と使い方	初めて出てくる所
`.write_all()`	用意した中身を、全部まとめて書き込むメソッド。ネットの接続（`TcpStream`）やファイルに、バイト（`&[u8]`）を書きます。途中で失敗することもあるので `Result` が返ります。	第0巻 UNIT 05
`.read_to_string()`	接続やファイルから、残りを全部読んで文字列に入れるメソッド。`stream.read_to_string(&mut text)` の `&mut text` は「この変数に書き込んでよい」と貸す書き方です。使うには `use std::io::Read;` が要ります。	第0巻 UNIT 05
`Read`	「読める物」が持っている機能（トレイト）。`read_to_string` などの読むメソッドを使うには、`use std::io::Read;` で、この機能を使えるようにしておく必要があります。	第0巻 UNIT 05
`Write`	「書ける物」が持っている機能（トレイト）。`write_all` や `writeln!` を使うには、`use std::io::Write;` で、この機能を使えるようにしておく必要があります。	第0巻 UNIT 05
`.read_line()`	1 行だけ読んで、文字列に書き足すメソッド。読んだ行の最後には改行が付いたままなので、`.trim_end()` などで取ってから使います。	第0巻 UNIT 06
`BufReader::new()`	読む物（接続やファイル）を包んで、1 行ずつ読めるようにした物を作る関数。	第0巻 UNIT 06
`BufRead`	「1 行ずつ読める物」が持っている機能（トレイト）。`read_line` や `lines` を使うには、`use std::io::BufRead;` が要ります。	第0巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`BufReader`	読む物（接続やファイル）を包んで、少しずつまとめて読む（バッファーする）ことで速くし、1 行ずつ読めるようにする型。`BufReader::new(&stream)` のように使います。	第0巻 UNIT 06
`writeln!`	`println!` と同じ書き方で、画面ではなく、ファイルや接続に 1 行書くマクロ。`writeln!(file, "{}", text)?;` のように、最初に書き込み先を渡します。使うには `use std::io::Write;` が要ります。	第0巻 UNIT 07
`.flush()`	ためておいた出力を、すぐに画面やファイルへ書き出すメソッド。`print!` は改行しないので、すぐ表示されないことがあり、`io::stdout().flush()` で確実に出します。使うには `use std::io::Write;` が要ります。	第2巻 UNIT 10
`io::stdout()`	標準出力（ふつうの画面への出口）を表す物を返す関数。`println!` も、ここへ書いています。	第2巻 UNIT 10
`io::stdin()`	標準入力（キーボードからの入口）を表す物を返す関数。`io::stdin().read_line(&mut s)` で 1 行読み、`io::stdin().lines()` で 1 行ずつくり返し読めます。	第2巻 UNIT 10
`.read_exact()`	決めた数のバイトを、ちょうどその数だけ読むメソッド。途中で終わってしまったら、エラーになります。HTTP の本文を、`Content-Length` の長さだけ読む時に使います。	第6巻 UNIT 01

プロセス

コード	意味と使い方	初めて出てくる所
`process::id()`	OS に、今動いているこのプログラム（プロセス）の番号を問い合わせる関数。`use std::process;` と書いてから使います。番号は動かすたびに変わります。	第0巻 UNIT 01
`env::args()`	プログラムを動かした時に渡された引数（`cargo run -- 花子` の `花子` など）を、1 つずつ取り出せる形で返す関数。0 番目はプログラム自身の場所なので、渡した値は 1 番目から。`env::args().nth(1)` で 1 つ目、`env::args().skip(1).collect::<Vec<_>>()` で全部を `Vec` にできます。	第0巻 UNIT 01
`process::exit()`	プログラムを、その場で終わらせる関数。`process::exit(2)` は、終了コード 2 を OS に知らせて終わります。終了コードは「0 なら成功、0 以外なら失敗」の決まりで、PowerShell では `$LASTEXITCODE` で見られます。	第0巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`env::current_dir()`	プログラムを動かした時の、今いるフォルダ（カレントディレクトリ）を返す関数。PowerShell の `Get-Location` と同じ場所です。取れないこともあるので、結果は `Result` で返り、`?` や `unwrap()` で中身を取り出します。	第0巻 UNIT 02
`env::var()`	環境変数（プログラムの外から渡す設定の値）を読む関数。`env::var("GREETING")`。設定されていないこともあるので、`Result` で返ります。`unwrap_or_else(|_| "既定値".to_string())` で、無い時の値を決めるのがふつうです。	第0巻 UNIT 03
`std::env::args()`	プログラムに渡された引数を取り出す関数（`env::args()` と同じ）。	第4巻 UNIT 05
`std::process::exit()`	プログラムを、その場で終わらせる関数（`process::exit` と同じ）。`std::process::exit(1)` は、終了コード 1（失敗）で終わります。	第4巻 UNIT 09
`VarError`	環境変数を読む `env::var` が失敗した時のエラーの型。`VarError::NotPresent`（設定されていない）と `VarError::NotUnicode`（文字として読めない）の 2 種類があります。	第4巻 UNIT 09
`NotPresent`	`VarError::NotPresent` は、「その名前の環境変数が設定されていない」というエラーの種類です。	第4巻 UNIT 09
`NotUnicode`	`env::var` が返すエラーの 1 つ（`env::VarError::NotUnicode`）。環境変数はあったが、中身が正しい文字（UTF-8）でなかった時です。もう 1 つは「無かった」の `NotPresent` です。	第4巻 UNIT 09
`std::env::var()`	環境変数を読む関数（`env::var` と同じ）。`std::env::var("PORT")` は `Result` を返し、設定されていなければ `Err` です。	第4巻 UNIT 09
`env::set_var()`	このプログラムの中の環境変数を設定する関数。テストで設定を試す時などに使います（Rust 2024 では、ほかのスレッドとの関係で `unsafe` の中で使う決まりです）。	第4巻 UNIT 09
`ExitCode`	`main` が返す、終了コードの型。`fn main() -> ExitCode` にして、`ExitCode::SUCCESS`（0）か `ExitCode::FAILURE`（1）を返します。`process::exit` と違い、片づけをしてから終われます。	第7巻 この巻で身に付けること
`SUCCESS`	`ExitCode::SUCCESS` は、終了コード 0（成功）です。	第7巻 この巻で身に付けること
`FAILURE`	`ExitCode::FAILURE` は、終了コード 1（失敗）です。	第7巻 この巻で身に付けること

コード	意味と使い方	初めて出てくる所
`std::env::current_exe()`	今動いている実行ファイル（`.exe`）の場所を返す関数。実行ファイルの隣に置いたフォルダを探す時に使います。	第9巻 UNIT 10
`env::remove_var()`	このプログラムの中の環境変数を消す関数（テストで使う）。	第13巻 UNIT 03
`std::env::set_var()`	このプログラムの中の環境変数を設定する関数（`env::set_var` と同じ）。	第13巻 UNIT 03

設定

コード	意味と使い方	初めて出てくる所
`dotenvy::dotenv()`	`.env` ファイルに書いた「名前=値」を、環境変数として読み込む関数（`dotenvy` の部品）。開発のパソコンで、設定をファイルにまとめておく時に使います。`.env` は Git に入れません。	第5巻 UNIT 01
`env!`	翻訳（コンパイル）の時の環境変数の値を、文字列として埋め込むマクロ。`env!("CARGO_MANIFEST_DIR")` は、`Cargo.toml` があるフォルダの場所です。	第9巻 UNIT 04

時間

コード	意味と使い方	初めて出てくる所
`.elapsed()`	`Instant::now()` で覚えた時から、どれだけたったかを `Duration` で返すメソッド。`start.elapsed()`。	第0巻 UNIT 04
`Instant::now()`	「今この瞬間」を覚えておく関数（`use std::time::Instant;`）。あとで `.elapsed()` を呼ぶと、そこからたった時間が分かります。時刻（何時何分）ではなく、時間を計るための物です。	第0巻 UNIT 04
`Instant`	「ある瞬間」を表す型。`let start = Instant::now();` で今を覚えておき、`start.elapsed()` で「そこから何秒たったか」を測れます。処理の速さを測る時に使います。	第0巻 UNIT 04
`.as_secs_f64()`	時間の長さ（`Duration`）を、秒を単位にした小数で返すメソッド。`0.25` なら 0.25 秒です。	第3巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`thread::sleep()`	今のスレッドを、決めた時間だけ止めて待つ関数。`thread::sleep(Duration::from_millis(200))` は 0.2 秒待ちます。非同期（`async`）の中では、代わりに `tokio::time::sleep(…).await` を使います。	第3巻 UNIT 10
`Duration::from_millis()`	ミリ秒（1000 分の 1 秒）から、時間の長さ（`Duration`）を作る関数。秒からなら `Duration::from_secs()` です。	第3巻 UNIT 10
`Duration`	時間の長さを表す型。`Duration::from_secs(5)` で 5 秒。`{:?}` で表示すると `5s` や `200ms` のように単位つきで出ます。	第3巻 UNIT 10
`.duration_since()`	「ある時刻から、どれだけたったか」を `Duration` で返すメソッド。`SystemTime::now().duration_since(UNIX_EPOCH)` は、1970 年 1 月 1 日から今までの時間です。	第4巻 UNIT 02
`.as_secs()`	時間の長さ（`Duration`）を、秒の整数で返すメソッド（小数点以下は切り捨て）。	第4巻 UNIT 02
`SystemTime`	カレンダーの時刻（今が何時か）を表す型。`SystemTime::now()` で今の時刻を取ります。記録に日時を残す時に使います（処理の時間を測るのは `Instant`）。	第4巻 UNIT 02
`Duration::from_secs()`	秒から、時間の長さ（`Duration`）を作る関数。`Duration::from_secs(10)` は 10 秒。	第5巻 UNIT 02
`.as_millis()`	時間の長さ（`Duration`）を、ミリ秒の整数で返すメソッド。	第5巻 UNIT 03
`std::thread::sleep()`	今のスレッドを、決めた時間だけ止める関数（`thread::sleep` と同じ）。async の中では使わず、`tokio::time::sleep` を使います。	第5巻 UNIT 06
`ZERO`	`Duration::ZERO` は、長さ 0 の時間。合計を数え始める時の最初の値に使います。	第5巻 UNIT 08
`std::time::Duration::from_millis()`	ミリ秒から、時間の長さを作る関数（`Duration::from_millis` と同じ）。	第5巻 UNIT 09
`.as_micros()`	時間の長さを、マイクロ秒（100 万分の 1 秒）の整数で返すメソッド。	第7巻 UNIT 04
`Duration::minutes()`	分から、時間の長さを作る関数（`time` の部品の `Duration`）。`Duration::minutes(5)` は 5 分。	第10巻 UNIT 09
`.as_nanos()`	時間の長さを、ナノ秒（10 億分の 1 秒）の整数で返すメソッド。	第11巻 UNIT 08

標準ライブラリ

コード	意味と使い方	初めて出てくる所
`OS`	`std::env::consts::OS` は、プログラムが動いている OS の種類を表す決まった値（定数）。Windows なら `"windows"`、Linux なら `"linux"` という文字列が入っています。	第0巻 UNIT 01
`ARCH`	`std::env::consts::ARCH` は、CPU の種類を表す決まった値（定数）。多くの Windows のパソコンでは `"x86_64"`、Arm の CPU なら `"aarch64"` です。	第0巻 UNIT 01
`.hash()`	値をハッシュ（内容から計算した、決まった長さの番号）の材料として、`Hasher` に流し込むメソッド（`Hash` トレイトのメソッド）。`"abc".hash(&mut hasher)` のあと、`hasher.finish()` で番号を受け取ります。	第0巻 UNIT 08
`.finish()`	`Hasher` に流し込んだ材料から、ハッシュの番号（`u64`）を計算して返すメソッド。同じ材料なら、何度計算しても同じ番号になります。	第0巻 UNIT 08
`DefaultHasher::new()`	ハッシュを計算する係を新しく作る関数。`value.hash(&mut hasher)` で値を入れ、`hasher.finish()` で結果の数を取り出します。	第0巻 UNIT 08
`DefaultHasher`	値からハッシュ（指紋のような数）を計算する、標準の道具。同じ値からは同じ数、少し違う値からはまったく違う数ができます。ここでは仕組みを見るために使います（パスワードの保存には、専用の Argon2 などを使います。第8巻）。	第0巻 UNIT 08
`Hash`	「ハッシュ（指紋のような数）を計算できる物」が持っている機能（トレイト）。`value.hash(&mut hasher)` を使うには `use std::hash::Hash;` が要ります。	第0巻 UNIT 08
`Hasher`	ハッシュを計算する係のトレイト（決まりごと）。`use std::hash::{Hash, Hasher};` と書くと、`.hash()`・`.finish()` を呼べるようになります。`DefaultHasher` は、標準ライブラリにある `Hasher` の 1 つです。	第0巻 UNIT 08
`.build_hasher()`	`RandomState` などから、ハッシュを計算する係（`Hasher`）を作るメソッド。ここでは、毎回違う値（でたらめな数）を作るのに使っています。	第5巻 UNIT 08
`RandomState::new()`	でたらめな「種」でハッシュの係を作るための物を作る関数。	第5巻 UNIT 08
`RandomState`	`HashMap` が、ハッシュを計算する時に使う「種」を、でたらめに決める係。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`BuildHasher`	「ハッシュを計算する係を作れる」機能（トレイト）。`.build_hasher()` を使うには、これを `use` します。	第5巻 UNIT 08

スレッドと共有

コード	意味と使い方	初めて出てくる所
`.fetch_add()`	数を、ほかのスレッドとぶつからずに足すメソッド（`AtomicU64` などに使う）。`COUNT.fetch_add(1, Ordering::Relaxed)` で 1 足します。鍵（`Mutex`）を使わずに、安全に数えられます。	第3巻 UNIT 06
`.load()`	アトミック（`AtomicUsize` など、複数のスレッドから安全に読み書きできる数）の今の値を読むメソッド。`count.load(Ordering::Relaxed)`。書く時は `.store()`、足す時は `.fetch_add()` です。	第3巻 UNIT 06
`AtomicU64::new()`	いくつものスレッドから安全に数えられる整数を、最初の値で作る関数。`static COUNT: AtomicU64 = AtomicU64::new(0);`。	第3巻 UNIT 06
`AtomicU64`	いくつものスレッドから同時に読み書きしても壊れない、整数の型。数を数える（アクセス数など）時に使います。	第3巻 UNIT 06
`Ordering`	`std::sync::atomic::Ordering` は、`AtomicU64` などを読み書きする時の「順番の守り方」の指定。ただ数えるだけなら、いちばん軽い `Ordering::Relaxed` で足ります。（大小を比べた結果の `std::cmp::Ordering` は別の物です。）	第3巻 UNIT 06
`Relaxed`	`Ordering::Relaxed` は、「この数の読み書きだけが壊れなければよい」という、いちばん軽い順番の守り方です。	第3巻 UNIT 06
`thread::spawn()`	新しいスレッド（同時に動く処理の流れ）を作って動かす関数。`thread::spawn(move || { … })`。返ってくる物の `.join()` で、終わるのを待って結果を受け取ります。	第3巻 UNIT 10
`.lock()`	`Mutex` の鍵を取って、中身を使えるようにするメソッド。返ってきた物（ガード）を通して中身を読み書きし、ガードが片づけられると鍵は自動で返されます。ほかのスレッドが鍵を持っている間は、返されるまで待ちます。	第3巻 UNIT 10
`Arc::new()`	値を、いくつものスレッドで一緒に持てる `Arc` に入れる関数。`Arc::new(Mutex::new(0))` は「鍵付きの 0 を、みんなで持つ」。	第3巻 UNIT 10
`Mutex::new()`	値を、鍵付きの入れ物（`Mutex`）に入れる関数。使う時は `.lock()` で鍵を取ります。	第3巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`Arc::clone()`	`Arc` の持ち手を 1 つ増やす関数。スレッドに渡す前に増やしておき、`move` で持たせます。	第3巻 UNIT 10
`Arc::strong_count()`	`Arc` の本体を、今いくつの持ち手が指しているかを返す関数。	第3巻 UNIT 10
`Arc`	いくつものスレッドから、1 つの値を一緒に持てるようにする型（`Rc` のスレッド版）。書きかえもしたい時は、`Arc<Mutex<T>>` の形にします。	第3巻 UNIT 10
`Mutex`	中身を、1 度に 1 つのスレッドだけが使えるようにする鍵付きの入れ物。`.lock()` で鍵を取ってから使います。	第3巻 UNIT 10
`.spawn()`	`thread::scope` の中で、新しいスレッドを動かすメソッド（`s.spawn(|| { … })`）。範囲の終わりで、全部のスレッドが終わるのを待ってくれます。	第3巻 UNIT 10
`thread::scope()`	「この中で作ったスレッドが全部終わるまで、外に出ない」範囲を作る関数。外の変数を借りたままスレッドで使えます（`move` や `Arc` が要らない）。	第3巻 UNIT 10
`.write()`	`RwLock` の「書き込み用の鍵」を取るメソッド。書いている間は、ほかのスレッドは読むことも書くこともできません。（ファイルや接続に使う `write` は「書き込む」意味の別の物です。）	第3巻 UNIT 10
`.read()`	`RwLock`（読む人は何人でも同時に、書く人は 1 人だけにする鍵）で、読むための鍵を取るメソッド。`state.notes.read().unwrap()` で中身を読めます。書く時は `.write()` です。	第3巻 UNIT 10
`RwLock::new()`	値を、「読むのは何人でも、書くのは 1 人」の鍵付きの入れ物（`RwLock`）に入れる関数。	第3巻 UNIT 10
`RwLock`	「読むだけなら何人でも同時に、書く時は 1 人だけ」にできる鍵付きの入れ物。読むことが多く、書くことが少ない物に向いています。読む時は `.read()`、書く時は `.write()` で鍵を取ります。	第3巻 UNIT 10
`Send`	「別のスレッドへ渡してもよい」ことを表す印（トレイト）。`Arc<dyn NoteRepository + Send + Sync>` の `+ Send` は「スレッドをまたいで渡せる物に限る」という条件です。多くの型は自動で持っています（`Rc` などは持っていません）。	第4巻 UNIT 04
`Sync`	「いくつものスレッドから同時に借りて読んでもよい」ことを表す印（トレイト）。Web サーバーのように、いくつもの処理が 1 つの物を共有する時に必要です。	第4巻 UNIT 04
`.id()`	スレッドの番号（`ThreadId`）を返すメソッド。`thread::current().id()` で、今どのスレッドで動いているかを表示できます。	第5巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`std::thread::current()`	今動いているスレッドの情報（名前・番号）を返す関数。どのスレッドで動いたかを表示する時に使います。	第5巻 UNIT 02
`LazyLock::new()`	初めて使われた時に 1 回だけ作る値を、作り方の関数と一緒に用意する関数。	第5巻 UNIT 05
`LazyLock`	初めて使われた時に 1 回だけ作られ、その後はずっと同じ値を使う入れ物。`static DUMMY: LazyLock<String> = LazyLock::new(|| { … });` のように、作るのに手間がかかる物を、プログラム全体で共有する時に使います。	第5巻 UNIT 05
`AtomicU32::new()`	いくつものスレッドから安全に数えられる 32 ビットの整数を、最初の値で作る関数。	第5巻 UNIT 08
`AtomicU32`	いくつものスレッドから同時に読み書きしても壊れない、32 ビットの整数の型（`AtomicU64` の小さい版）。	第5巻 UNIT 08
`SeqCst`	`Ordering::SeqCst` は、いちばん厳しい順番の守り方。どのスレッドからも、同じ順番で変わったように見えます。迷ったらこれを選べば安全です（少し遅くなります）。	第5巻 UNIT 08
`.fetch_max()`	今の値と比べて大きい方を、ぶつからずに入れるメソッド（`AtomicUsize` など）。最大の値を覚えておく時に使います。	第11巻 UNIT 08
`.fetch_sub()`	数を、ほかのスレッドとぶつからずに引くメソッド。	第11巻 UNIT 08
`AtomicUsize::new()`	いくつものスレッドから安全に数えられる `usize` の数を、最初の値で作る関数。	第11巻 UNIT 08
`AtomicUsize`	いくつものスレッドから同時に読み書きしても壊れない、`usize` の数の型。	第11巻 UNIT 08

非同期

コード	意味と使い方	初めて出てくる所
`async`	「待つ所で、ほかの仕事に順番をゆずれる関数」にする印。`async fn root(…)` のように付けます。呼ぶ側は `.await` で結果を待ちます。Web サーバーが、たくさんの人を同時にさばけるのは、この仕組みのおかげです（第5巻）。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`await`	`async` の関数（Future）が終わるのを待って、結果を受け取る書き方。`sleep(…).await` のように、後ろに `.await` を付けます。待っている間は、ほかの仕事に順番をゆずります。`.await` を付けないと、その仕事は始まりません。	第5巻 UNIT 01
`#[tokio]`	`#[tokio::main]` は、`main` を `async` にして、tokio（非同期の仕事を回す仕組み）の上で動かす印（属性）。`#[tokio::test]` は、テストを `async` で書けるようにする印です。	第5巻 UNIT 01
`tokio::time::sleep()`	決めた時間だけ待つ、非同期の関数。`tokio::time::sleep(Duration::from_millis(300)).await`。待っている間、ほかの仕事は止まりません（`std::thread::sleep` はスレッドごと止めてしまうので、`async` の中では使いません）。	第5巻 UNIT 01
`join!`	`tokio::join!(a, b)` は、いくつかの非同期の仕事を同時に進めて、全部が終わるのを待つマクロ。かかる時間は、いちばん長い物の分だけです。	第5巻 UNIT 01
`try_join!`	`tokio::try_join!(a, b)` は、`join!` と同じく同時に進めますが、どれか 1 つでも `Err` になったら、すぐに止めてそのエラーを返すマクロ。	第5巻 UNIT 01
`pin!`	Future を、メモリの中で動かないように「ピン留め」するマクロ。`poll` を自分で呼ぶ時などに必要です（ふつうは `.await` が自動でしてくれます）。	第5巻 UNIT 01
`.waker()`	`Context` から、「準備ができたら起こしてね」と頼むための物（`Waker`）を取り出すメソッド。非同期の仕組みの中身を見る時の物で、ふだんは使いません。	第5巻 UNIT 01
`.wake_by_ref()`	`Waker` を使って、「もう一度 `poll` してほしい」と知らせるメソッド。	第5巻 UNIT 01
`Context::from_waker()`	`Waker` から、`poll` に渡す `Context` を作る関数。非同期の仕組みの中身を確かめる時に使います。	第5巻 UNIT 01
`Waker::noop()`	「起こしても何もしない」`Waker` を作る関数。仕組みを試すための物です。	第5巻 UNIT 01
`Future`	「いつか終わる仕事」を表す型（トレイト）。`async fn` を呼ぶと、すぐには動かず、この Future が返ります。`.await` するか、tokio に渡して、初めて動きます。	第5巻 UNIT 01
`Pin`	中身がメモリの中で動かないことを約束する入れ物。Future の中には、自分の中の場所を指している物があるので、動かすと壊れてしまいます。それを防ぐための仕組みです。	第5巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`Poll`	Future に「終わった？」と聞いた答えの型。`Poll::Ready(値)`（終わった）か `Poll::Pending`（まだ）です。	第5巻 UNIT 01
`Waker`	「この仕事の準備ができた」と知らせて、もう一度動かしてもらうための物。tokio が、待っている仕事を起こすのに使います。	第5巻 UNIT 01
`Ready`	`Poll::Ready(値)` は「仕事が終わった。結果はこれ」という答え。まだなら `Poll::Pending` です。Future の `poll` は、この 2 つのどちらかを返します。	第5巻 UNIT 01
`Pending`	`Poll::Pending`（Future が「まだです」と答える時の値）のこと。終わった時は `Poll::Ready(値)` です。	第5巻 UNIT 01
`tokio::spawn()`	非同期の仕事を、別のタスクとしてすぐ動かし始める関数。`tokio::spawn(async move { … })`。返ってくる物（`JoinHandle`）を `.await` すると、結果を受け取れます。	第5巻 UNIT 02
`.worker_threads()`	tokio の組み立て係で、仕事を回すスレッドの数を決めるメソッド。	第5巻 UNIT 02
`.enable_all()`	tokio の組み立て係で、時間（タイマー）やネットワークなど、全部の機能を使えるようにするメソッド。	第5巻 UNIT 02
`tokio::runtime::Builder::new_multi_thread()`	いくつものスレッドで仕事を回す tokio の仕組み（ランタイム）を、組み立て始める関数。`.worker_threads(2).enable_all().build()` のように設定してから作ります。	第5巻 UNIT 02
`.is_panic()`	`tokio::spawn` した仕事の失敗（`JoinError`）が、パニック（止まった）のせいなら `true` を返すメソッド。	第5巻 UNIT 02
`.abort()`	`tokio::spawn` した仕事を、途中で取り消すメソッド。	第5巻 UNIT 02
`.is_cancelled()`	`tokio::spawn` した仕事の失敗が、取り消し（`abort`）のせいなら `true` を返すメソッド。	第5巻 UNIT 02
`tokio::runtime::Runtime::new()`	tokio の、非同期の仕事を回す仕組み（ランタイム）を、おすすめの設定で作る関数。	第5巻 UNIT 02
`Runtime`	tokio の、非同期の仕事を回す仕組み（ランタイム）の型。`#[tokio::main]` は、中でこれを作っています。`runtime.block_on(future)` で、ふつうの関数から async の仕事を動かせます。	第5巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`select!`	`tokio::select!` は、いくつかの非同期の仕事のうち、**先に終わった 1 つだけ**を使い、残りは取り消すマクロ。「時間切れ」や「止めての合図」と組み合わせて使います。	第5巻 UNIT 02
`tokio::task::spawn_blocking()`	時間のかかる重い計算や、待ってしまう処理（パスワードのハッシュなど）を、非同期の仕事を回すスレッドとは別のスレッドで動かす関数。ほかの仕事を止めないために使います。	第5巻 UNIT 02
`.join_next()`	`JoinSet` に入れた仕事のうち、終わった順に 1 つずつ結果を受け取るメソッド。全部受け取ると `None` を返します。	第5巻 UNIT 03
`JoinSet::new()`	空の `JoinSet`（いくつもの非同期の仕事をまとめて持つ入れ物）を作る関数。	第5巻 UNIT 03
`JoinSet`	いくつもの非同期の仕事をまとめて持ち、終わった順に結果を受け取れる入れ物（tokio の部品）。	第5巻 UNIT 03
`.join_all()`	`JoinSet` の仕事が全部終わるのを待って、結果を `Vec` で返すメソッド。	第5巻 UNIT 03
`.recv()`	チャンネル（仕事どうしで値を渡す管）から、次の 1 つを受け取るメソッド。届くまで待ちます。送る口が全部無くなると `None` を返します。	第5巻 UNIT 04
`.try_send()`	チャンネルに値を送るメソッド。待たずにすぐ試し、箱がいっぱいなら `Full`、受け取る側がいなければ `Closed` のエラーを返します。	第5巻 UNIT 04
`TrySendError`	`try_send` が失敗した時のエラーの型。`Full`（いっぱい）と `Closed`（受け取り係がいない）の 2 種類があります。	第5巻 UNIT 04
`Full`	`TrySendError::Full` は、「チャンネルの箱がいっぱいで、今は送れない」というエラーの種類です。	第5巻 UNIT 04
`Closed`	`TrySendError::Closed` は、「受け取り係がもういないので、送れない」というエラーの種類です。	第5巻 UNIT 04
`mpsc::channel()`	「送る口がいくつあってもよく、受ける口は 1 つ」のチャンネル（仕事どうしで値を渡す管）を作る関数。`mpsc::channel(16)` の 16 は、ためておける数です。`(送る口, 受ける口)` の組が返ります。	第5巻 UNIT 04
`oneshot::channel()`	1 回だけ値を送れるチャンネルを作る関数。「頼んだ仕事の答えを 1 つだけ返してもらう」時に使います。	第5巻 UNIT 04
`Sender`	チャンネルの「送る口」の型。`.send(値).await` で送ります。`.clone()` で送る口を増やせます（`mpsc` の場合）。	第5巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`Receiver`	チャンネルの「受ける口」の型。`.recv().await` で受け取ります。	第5巻 UNIT 04
`.subscribe()`	`broadcast`（1 つの送り手から、全員に配る）チャンネルで、受け取る口を新しく増やすメソッド。	第5巻 UNIT 04
`.changed()`	`watch` チャンネルの受け取り口で、値が変わるまで待つメソッド。送り手がいなくなると `Err` です。	第5巻 UNIT 04
`watch::channel()`	「いちばん新しい値だけ」を全員に見せるチャンネルを作る関数。設定の値（ログの細かさなど）を、動いている仕事に伝える時に使います。	第5巻 UNIT 04
`RecvError`	チャンネルから受け取る時のエラーの型。`broadcast` では、受け取るのが遅すぎて取りこぼした時の `Lagged` などがあります。	第5巻 UNIT 04
`Lagged`	`RecvError::Lagged(数)` は、「受け取るのが遅れて、その数だけ値を取りこぼした」というエラーの種類です。	第5巻 UNIT 04
`.closed()`	チャンネルの受ける口が全部無くなるまで待つメソッド（送る口に使う）。「全員の片づけが終わった」ことを知る時に使います。	第5巻 UNIT 04
`.acquire_owned()`	`Semaphore`（同時に使える数を決めた札の入れ物）から、札を 1 枚もらうメソッド。札が無ければ、空くまで待ちます。もらった札は、片づけられると自動で返されます。	第5巻 UNIT 05
`Semaphore::new()`	同時に動いてよい数（札の数）を決めて `Semaphore` を作る関数。	第5巻 UNIT 05
`Semaphore`	「同時にいくつまで動いてよいか」を、札の数で決める仕組み。同時に外へ出す要求を 3 つまでに絞る、などに使います。	第5巻 UNIT 05
`#[tokio::main(flavor)]`	`#[tokio::main(flavor = "current_thread")]` は、非同期の仕事を回す係（tokio）を、スレッド 1 本だけで動かす指定です。何も書かなければ、CPU の数に合わせて何本も使います。	第5巻 UNIT 05
`tokio::time::timeout()`	非同期の仕事に時間の制限を付ける関数。`timeout(Duration::from_secs(1), 仕事).await` は、1 秒で終わらなければ `Err` を返します。	第5巻 UNIT 05
`.cancelled()`	`CancellationToken`（止めての合図）が出されるまで待つメソッド。`select!` と組み合わせて、「合図が来たら仕事をやめる」と書きます。	第5巻 UNIT 06
`.tick()`	`interval`（決まった間隔で鳴る時計）の、次の「鳴る時」まで待つメソッド。`loop { ticker.tick().await; … }` で、決まった間隔で同じ仕事をくり返します。	第5巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`.child_token()`	`CancellationToken` から、子の合図を作るメソッド。親が止まれば子も止まりますが、子だけを止めることもできます。	第5巻 UNIT 06
`.close()`	`TaskTracker` に「もう新しい仕事は足さない」と知らせるメソッド。このあと `.wait().await` で、全部が終わるのを待ちます。	第5巻 UNIT 06
`.cancel()`	`CancellationToken` で、「止めて」の合図を出すメソッド。合図を受けた仕事は、`.cancelled()` の待ちから抜けて、片づけを始めます。	第5巻 UNIT 06
`CancellationToken::new()`	「止めて」の合図を伝える `CancellationToken` を作る関数。	第5巻 UNIT 06
`TaskTracker::new()`	動かした仕事を覚えて、全部終わるまで待てる `TaskTracker` を作る関数。	第5巻 UNIT 06
`tokio::signal::ctrl_c()`	`Ctrl` ＋ `C` が押されるまで待つ関数。サーバーを行儀よく止める合図に使います。	第5巻 UNIT 06
`CancellationToken`	「止めて」の合図を、いくつもの仕事に一斉に伝えるための物（`tokio_util` の部品）。`.cancel()` で合図を出し、仕事の側は `.cancelled().await` で受け取ります。サーバーを行儀よく止める時に使います。	第5巻 UNIT 06
`TaskTracker`	動かした仕事を覚えておき、「全部終わるまで待つ」ことができる入れ物（`tokio_util` の部品）。`tracker.spawn(…)` で動かし、`tracker.close()` → `tracker.wait().await` で待ちます。	第5巻 UNIT 06
`tokio::signal::unix::signal()`	Linux などで、OS からの合図（シグナル）を受け取る関数。`SignalKind::terminate()`（「終わって」の合図）を受け取って、サーバーを片づけてから終わる時に使います（Fly.io などで動かす時。第12巻）。	第5巻 UNIT 06
`tokio::signal::unix::SignalKind::terminate()`	Linux などの「終わって」（SIGTERM）の合図の種類。Fly.io などがサーバーを止める時に送ってきます。	第5巻 UNIT 06
`SignalKind`	受け取る OS の合図（シグナル）の種類。`SignalKind::terminate()` は「終わって」（SIGTERM）の合図です。	第5巻 UNIT 06
`AsyncReadExt`	tokio で、接続やファイルから非同期に読むメソッド（`read`・`read_to_end` など）を使えるようにする機能（トレイト）。`use tokio::io::AsyncReadExt;` が要ります。	第5巻 UNIT 08
`AsyncWriteExt`	tokio で、非同期に書くメソッド（`write_all` など）を使えるようにする機能（トレイト）。`use tokio::io::AsyncWriteExt;` が要ります。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`#[async_trait]`	トレイトの中に `async fn` を書き、しかもそれを `dyn`（中身の型を後で決める）で使えるようにする印（`async_trait` の部品）。	第7巻 UNIT 09

ネットワーク

コード	意味と使い方	初めて出てくる所
`TcpStream::connect()`	相手（`"example.com:80"` のように「名前:ポート番号」）に TCP でつなぐ関数。つながると `TcpStream` が返り、それに書く・読むことで、相手とやり取りします。つながらないこともあるので、結果は `Result` です。	第0巻 UNIT 05
`TcpStream`	ネットワークの 1 本の接続を表す型。`TcpStream::connect("127.0.0.1:8080")` でつなぎ、`write_all` で送り、`read_to_string` で受け取ります。HTTP などの通信は、この接続の上で文字をやり取りしています。	第0巻 UNIT 05
`Connection`	ここでは Rust の型ではなく、HTTP の見出し（ヘッダー）の名前です。`Connection: close` は「答えを送り終えたら、接続を閉じて」という頼みです。同じ行の `User-Agent:` も見出しの名前で、「どんなプログラムからの頼みか」を伝えます。（第5巻からは、データベースとの接続を表す `libsql::Connection` という型も出てきます。）	第0巻 UNIT 05
`.incoming()`	サーバーが、やって来る接続を 1 つずつ受け取れる形にするメソッド。`for stream in listener.incoming() { … }` で、接続が来るたびに中が 1 回動きます。	第0巻 UNIT 06
`TcpListener::bind()`	決まったポート番号で、接続を待ち受け始める関数。`TcpListener::bind("127.0.0.1:8080")` は「このパソコンの 8080 番で待つ」。もうほかのプログラムが使っている番号だと、エラーになります。	第0巻 UNIT 06
`TcpListener`	決まったポート番号で接続を待つ、サーバー側の型。`TcpListener::bind(…)` で作り、`.incoming()` で来た接続を受け取ります。	第0巻 UNIT 06
`.accept()`	サーバーが待ち受けている所に来た接続を、1 つ受け取るメソッド（`listener.accept().await`）。来るまで待ちます。	第5巻 UNIT 02
`.local_addr()`	待ち受けている（つないでいる）自分の側の住所（IP アドレスとポート番号）を返すメソッド。番号 `0` で待ち受けると、OS が空いている番号を選ぶので、それを知るのに使います。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`SocketAddr`	IP アドレスとポート番号の組（例: `127.0.0.1:8080`）を表す型。	第5巻 UNIT 08
`std::net::TcpListener::bind()`	決めた番号で、接続を待ち受け始める関数（`TcpListener::bind` と同じ）。番号を `0` にすると、OS が空いている番号を選びます。	第5巻 UNIT 08
`tokio::net::TcpListener::bind()`	非同期で接続を待ち受け始める関数（tokio 版）。`.await` で待ちます。Web サーバーは、これで決めた番号を待ち受けます。	第9巻 UNIT 10

HTTP

コード	意味と使い方	初めて出てくる所
`.body()`	リクエストやレスポンスの本文を取り出す（決める）メソッド。テストで `Request::builder()….body(Body::from("…"))` のように本文を入れます。	第5巻 UNIT 08
`Body`	リクエストやレスポンスの本文を表す型。Topcoat では `topcoat::router::Body` として使い、ミドルウェア（`layer`）で本文を受け取って次へ渡す時などに出てきます。	第5巻 UNIT 09
`StatusCode`	HTTP の状態コード（`200 OK`・`404 Not Found` など）を表す型。`StatusCode::OK`・`StatusCode::NOT_FOUND` のように、名前で書けます。	第5巻 UNIT 09
`CREATED`	`StatusCode::CREATED` は、HTTP の答えの番号 201（「作りました」）。新しいデータを作った時に返します。	第6巻 UNIT 06
`.method()`	`Request::builder().method(Method::POST)` のように、作るリクエストの HTTP のメソッドを決めるメソッド。	第6巻 UNIT 07
`.uri()`	`Request::builder().uri("/notes")` のように、作るリクエストの URL を決めるメソッド。	第6巻 UNIT 07
`.into_body()`	HTTP の答え（レスポンス）から、本文の部分だけを取り出すメソッド。テストで、返ってきた本文を確かめる時に使います。	第6巻 UNIT 07
`Body::empty()`	中身が空っぽの本文を作る関数。本文の要らないリクエスト（GET など）をテストで作る時に、`.body(Body::empty())` と書きます。	第6巻 UNIT 07
`Method`	HTTP のメソッド（`GET`・`POST`・`PUT`・`DELETE` など）を表す型。	第6巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`OK`	`StatusCode::OK` は、HTTP の答えの番号 200（成功）です。	第7巻 UNIT 10
`.headers()`	答え（レスポンス）の見出し（ヘッダー）の一覧を返すメソッド。`.headers().get("set-cookie")` のように、名前で 1 つ取り出せます。	第8巻 UNIT 01
`.get_all()`	HTTP のヘッダーで、同じ名前の物（`Set-Cookie` など）を全部取り出すメソッド。`get` だと最初の 1 つだけです。	第8巻 UNIT 01
`Body::from()`	文字列やバイトの並びから、本文（`Body`）を作る関数。`Body::from("title=memo&body=")` は、フォームの値を本文にしたものです。	第8巻 UNIT 01
`HOST`	`header::HOST` は、HTTP の `Host` ヘッダー（どのサイト宛ての頼みか）の名前。	第8巻 UNIT 01
`COOKIE`	`header::COOKIE` は、HTTP の `Cookie` ヘッダー（ブラウザーが覚えているクッキーを送る所）の名前。	第8巻 UNIT 01
`Origin`	`Origin` ヘッダーは、フォームなどを送ったページが、どのサイトの物かを表します。ほかのサイトから勝手に送らせる攻撃（CSRF）を見分けるのに使います（第8巻 UNIT 06）。	第8巻 UNIT 01
`UNAUTHORIZED`	`StatusCode::UNAUTHORIZED` は、HTTP の答えの番号 401（「ログインしていない」）です。	第8巻 UNIT 01
`CONFLICT`	`StatusCode::CONFLICT` は、HTTP の答えの番号 409（「ぶつかった」）。同じメールアドレスで登録しようとした時などに返します。	第8巻 UNIT 03
`.is_redirection()`	HTTP の答えの番号が 300〜399（別のページへ移って）なら `true` を返すメソッド。	第8巻 UNIT 03
`FORBIDDEN`	`StatusCode::FORBIDDEN` は、HTTP の答えの番号 403（「その操作は許されていない」）です。	第8巻 UNIT 10
`ALLOW`	`header::ALLOW` は、HTTP の `Allow` ヘッダー（「この URL で使えるメソッドはこれ」）の名前。405（使えないメソッド）の答えに付きます。	第9巻 UNIT 02
`LOCATION`	`header::LOCATION` は、HTTP の `Location` ヘッダー（「ここへ移って」という行き先）の名前。303 などの答えに付きます。	第9巻 UNIT 08
`HEAD`	`Method::HEAD` は、本文を受け取らずに、見出し（ヘッダー）だけをもらう HTTP のメソッド。サーバーが動いているかを軽く確かめる時に使います。	第9巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`ORIGIN`	`header::ORIGIN` は、HTTP の `Origin` ヘッダー（送ってきたページのサイト）の名前。	第9巻 UNIT 08
`.canonical_reason()`	HTTP の答えの番号の、決まった説明の言葉を返すメソッド（`404` なら `"Not Found"`）。	第13巻 UNIT 07

HTTP クライアント

コード	意味と使い方	初めて出てくる所
`curl.exe -sv`	`-sv` は `-s` と `-v` を合わせた物です。`-v`（verbose）は、送った要求（`>` の行）と、返ってきた見出し（`<` の行）を全部表示します。通信の中身を調べる時に使います。	第0巻 UNIT 05
`curl.exe -o`	`-o ファイル名` は、返ってきた中身を、画面ではなくファイルに保存します。`-o NUL` と書くと、中身を捨てます（Windows の `NUL` は「どこにも書かない」という特別な名前です）。	第0巻 UNIT 05
`curl.exe -s`	`-s`（silent）は、進み具合やエラーの表示を出さない指定です。返ってきた中身だけを見たい時に付けます。	第0巻 UNIT 06
`curl.exe -i`	`-i` は、返ってきた中身の前に、見出し（`HTTP/1.1 200 OK` の行や、`content-type` などのヘッダー）も表示する指定です。	第0巻 UNIT 06
`curl.exe -X`	`-X POST` は、要求の種類（メソッド）を指定します。付けない時は GET（`-d` を付けた時は POST）です。	第0巻 UNIT 06
`curl.exe -w`	`-w '%{http_code}'` は、終わった後に表示する物を決めます。`%{http_code}` は状態コード（`200` など）です。`-o NUL` と組み合わせて、状態コードだけを見る時に使います。	第1巻 UNIT 09
`curl.exe -d`	`-d "title=牛乳"` は、送る中身（本文）を指定します。付けると、要求は POST になり、`Content-Type` は `application/x-www-form-urlencoded`（フォームの形）になります。	第1巻 UNIT 09
`.connect_timeout()`	`reqwest` の組み立て係で、「相手につなぐまで」の時間の制限を決めるメソッド。	第5巻 UNIT 08
`.timeout()`	`reqwest` の組み立て係で、「頼んでから答えが全部届くまで」の時間の制限を決めるメソッド。	第5巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.is_success()`	HTTP の答えの番号（ステータスコード）が 200〜299（成功）なら `true` を返すメソッド。	第5巻 UNIT 08
`.error_for_status()`	`reqwest` の答えのうち、番号が 400〜599（失敗）の物を `Err` に変えるメソッド。`?` と組み合わせると、失敗の答えを見逃さずに済みます。	第5巻 UNIT 08
`reqwest::Client::builder()`	ほかのサーバーへ HTTP で頼みに行くための「クライアント」（`reqwest` の部品）を、設定しながら作り始める関数。`.timeout(…).build()` のようにつなげます。	第5巻 UNIT 08
`Client`	`reqwest::Client` は、HTTP で頼みに行く係。中で接続をためて使い回すので、1 つ作ってアプリ全体で共有するのがよい使い方です。	第5巻 UNIT 08
`.json()`	`reqwest` の答え（レスポンス）の本文を、JSON として読んで、決めた型に変えるメソッド。`response.json::<Weather>().await`。	第5巻 UNIT 08
`.is_connect()`	`reqwest` のエラーが「つなぐこと自体に失敗した」ものなら `true` を返すメソッド。	第5巻 UNIT 08
`.is_timeout()`	`reqwest` のエラーが「時間切れ」なら `true` を返すメソッド。	第5巻 UNIT 08
`.is_server_error()`	HTTP の答えの番号が 500〜599（サーバー側の失敗）なら `true` を返すメソッド。もう一度試す価値がある失敗かを見分けるのに使います。	第5巻 UNIT 08
`.as_u16()`	HTTP の答えの番号（ステータスコード）を、`200` や `429` のような数（`u16`）で返すメソッド。	第5巻 UNIT 08
`.post()`	`reqwest` の `Client` で、POST（データを送る）のリクエストを作り始めるメソッド。`client.post(url).body("…").send().await` のようにつなげて送ります。	第5巻 UNIT 08
`reqwest::Client::new()`	ほかのサーバーへ HTTP で頼みに行く係（クライアント）を、おすすめの設定で作る関数。1 つ作って使い回します。	第5巻 UNIT 08
`curl.exe -I`	`-I`（大文字）は、中身を取らずに、見出しだけを取る（HEAD という種類の要求を送る）指定です。	第6巻 UNIT 02
`curl.exe --compressed`	`--compressed` は、「圧縮して送ってよい」とサーバーに伝え、受け取った物をほどいてから表示する指定です。	第6巻 UNIT 02
`curl.exe -A`	`-A "RustNote-test/1.0"` は、「どの道具から来た要求か」を表す見出し（`User-Agent`）を指定します。	第6巻 UNIT 04
`curl.exe -H`	`-H "Content-Type: application/json"` は、要求に見出し（ヘッダー）を 1 つ足します。何度も書けます。	第6巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`curl.exe --data-binary`	`--data-binary` は、送る中身を、改行なども変えずにそのまま送る指定です。`--data-binary "@note.json"` のように `@` を付けると、ファイルの中身を送ります。	第6巻 UNIT 06

クッキー

コード	意味と使い方	初めて出てくる所
`.value()`	クッキーから、値の部分（`名前=値` の `値`）を取り出すメソッド。`cookie.value().to_string()`。	第6巻 UNIT 10
`Cookie::build()`	クッキー（ブラウザーに覚えておいてもらう小さな値）を、設定しながら作り始める関数。`Cookie::build(("theme", "dark")).path("/").build()` のように、名前と値、使う範囲などを決めます。	第6巻 UNIT 10
`Cookie`	クッキー 1 つ（名前と値と、使う範囲・期限などの設定）を表す型。	第6巻 UNIT 10
`Cookies`	今のリクエストのクッキーを読み書きする入れ物（クッキーの瓶）。`jar.add(…)` で足し、`jar.get("名前")` で読みます。	第6巻 UNIT 10
`cookie!`	Topcoat のマクロ。クッキーを、名前・値・決まり（`HttpOnly`・`SameSite`・`MaxAge` など）を並べて作ります（第10巻 UNIT 09）。	第10巻 UNIT 09
`HttpOnly`	クッキーの決まりの 1 つ。付けると、ページの JavaScript からそのクッキーを読めなくなり、盗まれにくくなります。	第10巻 UNIT 09
`SameSite`	クッキーの決まりの 1 つ。ほかのサイトから来た頼みに、このクッキーを付けるかを決めます。`Lax` は、ほかのサイトからのリンクでの移動には付けるが、フォームの送信などには付けない設定です。	第10巻 UNIT 09
`Lax`	`SameSite = Lax` は、ほかのサイトからのリンクで来た時（ふつうの移動）にはクッキーを送り、ほかのサイトからのフォームの送信には送らない設定です。	第10巻 UNIT 09
`MaxAge`	クッキーを、ブラウザーが覚えておく長さを決める決まり。過ぎると消えます。	第10巻 UNIT 09

認証

コード	意味と使い方	初めて出てくる所
`.hash_password()`	`argon2` の部品で、パスワードからハッシュ（元に戻せない、ソルト付きの文字列）を作るメソッド。`Argon2::default().hash_password(パスワードのバイト, &salt)`。データベースには、パスワードではなくこのハッシュを保存します。	第5巻 UNIT 05
`Argon2::default()`	パスワードを安全に保存するための計算（Argon2id）を、おすすめの設定で用意する書き方（`argon2` の部品）。この計算は、わざと重く作られていて、パスワードを総当たりで当てるのを難しくします（第8巻）。	第5巻 UNIT 05
`Argon2`	パスワードをハッシュにする方法の 1 つ（Argon2）を使うための型。`Argon2::default()` で、おすすめの設定のものを作ります。わざと時間とメモリを使うので、総当たりで破られにくくなります。	第5巻 UNIT 05
`.verify_password()`	`argon2` の部品で、入力されたパスワードが、保存してあるハッシュと合うかを確かめるメソッド。合えば `Ok(())`、合わなければ `Err` です。	第5巻 UNIT 07
`PasswordHash::new()`	保存してあるハッシュの文字列を読み取って、確かめに使える形にする関数。形が違えばエラーです。	第5巻 UNIT 07
`PasswordHasher`	パスワードからハッシュ（保存用の値）を作る機能（トレイト。`password_hash` の部品）。`argon2.hash_password(…)` を使うには、これを `use` します。	第5巻 UNIT 07
`PasswordVerifier`	入力されたパスワードが、保存してあるハッシュと合うかを確かめる機能（トレイト）。`argon2.verify_password(…)` を使うには、これを `use` します。	第5巻 UNIT 07
`PasswordHash`	保存してあるハッシュの文字列（`$argon2id$v=19$…`）を読み取った物の型。`PasswordHash::new(&保存した文字列)` で作り、確かめに使います。	第5巻 UNIT 07
`SaltString::generate()`	パスワードのハッシュに混ぜる「塩」（ソルト：毎回でたらめに作る値）を作る関数。同じパスワードでも、人ごとに違うハッシュになり、まとめて解かれにくくなります。	第8巻 UNIT 02
`SaltString`	ソルト（塩）を文字列の形で持つ型。	第8巻 UNIT 02
`OsRng`	OS が用意する、予想できないでたらめな数を作る係。パスワードやトークンなど、安全のための値は、必ずこのような係で作ります。	第8巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`.encode()`	`base64` の部品で、バイトの並びを、URL に入れても安全な文字（Base64）に変えるメソッド。`URL_SAFE_NO_PAD.encode(バイト)`。	第8巻 UNIT 09
`.random()`	`rand` の部品で、でたらめな値を作るメソッド（`rand::rng().random()`）。トークンなど、予想されてはいけない値に使います。	第8巻 UNIT 09
`Sha256::digest()`	データから、SHA-256 というハッシュ（32 バイトの指紋）を計算する関数（`sha2` の部品）。	第8巻 UNIT 09
`rand::rng()`	でたらめな数を作る係を取り出す関数（`rand` の部品）。`rand::rng().random()` で、でたらめな値を作ります。	第8巻 UNIT 09
`Engine`	`base64` の部品の、エンコード（バイト → 英数字の文字列）の機能（トレイト）。`URL_SAFE_NO_PAD.encode(バイト)` を使うには、これを `use` します。	第8巻 UNIT 09
`RngExt`	`rand` の部品の、`.random()` などのメソッドを使えるようにする機能（トレイト）。	第8巻 UNIT 09
`Digest`	ハッシュの計算（`digest`・`update`・`finalize`）を使えるようにする機能（トレイト。`sha2` などの部品）。	第8巻 UNIT 09
`Sha256`	SHA-256 というハッシュの計算をする型。データの「指紋」を作るのに使います（パスワードの保存には、わざと重い Argon2 を使います）。	第8巻 UNIT 09
`.finalize()`	ハッシュや MAC の計算を締めくくって、結果を取り出すメソッド。	第8巻 UNIT 09
`.into_bytes()`	計算の結果を、バイトの並びとして取り出すメソッド。	第8巻 UNIT 09
`Hmac`	HMAC（秘密の鍵を混ぜたハッシュ）を計算する型（`hmac` の部品）。鍵を知っている人だけが同じ値を作れるので、「改ざんされていないか」の確かめに使います。	第8巻 UNIT 09
`KeyInit`	鍵から計算の係を作る（`new_from_slice`）機能（トレイト）。	第8巻 UNIT 09
`Mac`	HMAC の計算（`update`・`finalize`・`verify_slice`）を使えるようにする機能（トレイト）。	第8巻 UNIT 09
`Sha1`	SHA-1 というハッシュの型。古い方法ですが、2 段階認証（TOTP）の決まりで今も使われています。	第8巻 UNIT 09

ログ

コード	意味と使い方	初めて出てくる所
`info!`	ログ（動作の記録）を「お知らせ」の重さで書くマクロ（`tracing` の部品）。`info!("起動しました");`。ほかに `debug!`（細かい調査用）・`warn!`（注意）・`error!`（エラー）があります。	第5巻 UNIT 09
`warn!`	ログを「注意」の重さで書くマクロ。`warn!(%path, ?elapsed, "応答が遅めです")` の `%` は `Display` で、`?` は `Debug` で、その値を記録に付けます。	第5巻 UNIT 09
`error!`	ログを「エラー」の重さで書くマクロ。失敗した時に、原因を追うための情報を残します。	第5巻 UNIT 09
`debug!`	ログを「細かい調査用」の重さで書くマクロ。ふだんは表示されず、ログの細かさを `debug` にした時だけ出ます。	第5巻 UNIT 09
`.init()`	ログの設定を、プログラム全体で使えるように登録するメソッド。`tracing_subscriber::fmt().init();` を `main` の最初に 1 回だけ呼びます。	第5巻 UNIT 09
`%値`	`tracing` のログの欄に `user_id = %id` と書くと、値を人が読む形（Display。`{}` と同じ）で記録します。`%id` のように、欄の名前を省くと、変数の名前がそのまま欄の名前になります。	第5巻 UNIT 09
`?値`	`tracing` のログの欄に `error = ?e` と書くと、値をプログラマー向けの形（Debug。`{:?}` と同じ）で記録します。`Display` の無い型も記録できます。	第5巻 UNIT 09
`#[instrument]`	`tracing` の印（属性）。付けた関数が呼ばれるたびに、その関数の名前と引数を「スパン」（ひとまとまりの処理の記録）として残します。ログを、どの処理の中で起きたかでまとめて見られます。	第5巻 UNIT 09
`.with_span_events()`	スパン（ひとまとまりの処理）の始まりや終わりにも、ログを出すように決めるメソッド。`FmtSpan::CLOSE` で、終わった時に、かかった時間も出ます。	第5巻 UNIT 09
`.with_timer()`	ログの行の頭に付ける時刻の形を決めるメソッド。	第5巻 UNIT 09
`FmtSpan`	スパンのどの時点でログを出すかの指定。`FmtSpan::CLOSE` は「終わった時」です。	第5巻 UNIT 09
`CLOSE`	`FmtSpan::CLOSE` は、「スパン（ひとまとまりの処理）が終わった時に、かかった時間と一緒にログを出す」指定です。	第5巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`#[instrument(skip)]`	`#[instrument(skip(db, password))]` の `skip` は、「この引数は、ログに記録しない」という指定です。パスワードのような秘密の値や、大きくて記録できない値（データベースの接続など）を外します。	第5巻 UNIT 09
`#[instrument(fields)]`	`#[instrument(fields(user_id = %id))]` の `fields` は、スパン（処理の区間の記録）に、引数のほかに記録したい欄を足す指定です。	第5巻 UNIT 09
`.with_env_filter()`	どのログを表示するか（細かさ）を、決まりで絞るメソッド。	第5巻 UNIT 09
`EnvFilter::try_from_default_env()`	環境変数 `RUST_LOG`（例: `RUST_LOG=debug`）から、表示するログの細かさを読み取る関数。設定が無ければ `Err` なので、`unwrap_or_else(|_| EnvFilter::new("info"))` で既定を決めます。	第5巻 UNIT 09
`EnvFilter::new()`	表示するログの細かさの決まりを、文字（`"info"` など）から作る関数。	第5巻 UNIT 09
`EnvFilter`	「どのログを表示するか」の決まり（`info` 以上だけ・この部品だけ `debug` など）の型。	第5巻 UNIT 09
`info_span!`	「ここからここまでが 1 つの処理」というスパンを作るマクロ。`info_span!("request", request_id, path)` のように、処理の目印になる値を付けておくと、その中のログに自動で付きます。	第5巻 UNIT 09
`.instrument()`	async の仕事に、スパンを付けるメソッド。その仕事の中で書いたログが、このスパンの中の物として記録されます。	第5巻 UNIT 09
`.without_time()`	ログの行に時刻を付けないようにするメソッド（結果を見比べやすくするため）。	第5巻 UNIT 09
`Instrument`	`.instrument()` を使えるようにする機能（トレイト）。`use tracing::Instrument;` が要ります。	第5巻 UNIT 09
`.in_current_span()`	async の仕事に、「今のスパン」を付けるメソッド。`tokio::spawn` で別の仕事にした時も、ログがどの処理の物か分かるようになります。	第5巻 UNIT 09
`.with_writer()`	ログを書く先（標準出力・ファイルなど）を決めるメソッド。	第11巻 UNIT 07

テスト

コード	意味と使い方	初めて出てくる所
`assert_eq!`	「2 つの値が同じか」を確かめるマクロ。`assert_eq!(実際の値, 期待する値);`。違う時はテストが失敗し、`left`（実際）と `right`（期待）の値を表示します。	第0巻 UNIT 10
`assert!`	「条件が `true` か」を確かめるマクロ。`assert!(is_valid(&title));`。`false` ならテストが失敗します。	第0巻 UNIT 10
`#[cfg]`	「この条件の時だけ、この部分を翻訳する」という印（属性）。`#[cfg(test)]` を付けた部分は、`cargo test` の時だけ翻訳され、ふつうの実行ファイルには入りません。	第0巻 UNIT 10
`#[test]`	「この関数はテストです」という印（属性）。`cargo test` が、この印の付いた関数を全部探して動かします。	第0巻 UNIT 10
`#[cfg(test)]`	`#[cfg(test)]` を付けた所（ふつうは `mod tests { … }`）は、`cargo test` の時だけ翻訳されます。`cargo run` や本番の実行ファイルには入りません。テストのコードは、この中に書きます。	第0巻 UNIT 10
`cfg!`	条件（機能が ON か・どの OS か など）が合っているかを、`true`・`false` で返すマクロ。`if cfg!(feature = "json") { … }` のように、普通の `if` の中で使えます。	第4巻 UNIT 07
`#[cfg_attr]`	「条件が合う時だけ、この印（属性）を付ける」という印。`#[cfg_attr(feature = "json", derive(Serialize))]` は、`json` の機能が ON の時だけ `Serialize` を付けます。	第4巻 UNIT 07
`#[allow]`	「この警告は出さなくてよい」と伝える印（属性）。`#[allow(dead_code)]` は、使っていない関数への警告を止めます。本当に要らない警告だけに使います。	第4巻 UNIT 08
`#[should_panic]`	「このテストは、パニック（止まる）するのが正しい」という印（属性）。`#[should_panic(expected = "文の一部")]` で、止まる時のメッセージも確かめられます。	第4巻 UNIT 10
`.unwrap_err()`	`Result` が `Err` の時に、そのエラーを取り出すメソッド。`Ok` だった時は止まります。テストで「ちゃんと失敗するか」を確かめる時に使います。	第4巻 UNIT 10
`assert_ne!`	「2 つの値が違うか」を確かめるマクロ。同じだとテストが失敗します。	第8巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`#[track_caller]`	付けた関数の中で止まった（パニックした）時に、その関数の中ではなく「呼んだ所」の行の番号を表示する印（属性）。テスト用の手伝いの関数に付けると、失敗の場所が分かりやすくなります。	第11巻 UNIT 03
`.select()`	`scraper` の部品で、HTML の中から、CSS と同じ書き方（`Selector`）に合うタグを探すメソッド。テストで、返ってきた HTML の中身を確かめる時に使います。	第11巻 UNIT 03
`Selector`	`scraper` の部品の、「どのタグを探すか」（CSS と同じ書き方）を表す型。`Selector::parse("a.nav")` で作ります。	第11巻 UNIT 03
`proptest!`	`proptest` の部品のマクロ。決まった値ではなく、たくさんのでたらめな入力を自動で作って、「どんな入力でも、この性質が成り立つか」を確かめるテスト（プロパティテスト）を書きます（第11巻）。	第11巻 UNIT 04
`prop_assert_eq!`	`proptest!` の中で使う `assert_eq!`。違った時に、失敗する入力を小さく絞り込んで教えてくれます。	第11巻 UNIT 04
`.new_tree()`	`proptest` の作り方（`Strategy`）から、でたらめな値を 1 つ作るメソッド（仕組みを確かめる用）。	第11巻 UNIT 04
`.current()`	`proptest` の値の木（`ValueTree`）から、今の値を取り出すメソッド。	第11巻 UNIT 04
`TestRunner::deterministic()`	毎回同じでたらめな値の並びになる `proptest` の実行係を作る関数。結果を見比べたい時に使います。	第11巻 UNIT 04
`ValueTree`	`proptest` が作った値と、その値を小さくしていく道筋を持つ物の型。`.current()` で今の値を取り出します。	第11巻 UNIT 04
`TestRunner`	`proptest` の、テストを何回も動かす係の型。	第11巻 UNIT 04
`prop_oneof!`	いくつかの作り方のうち、どれか 1 つをでたらめに選んで値を作るマクロ（`proptest`）。	第11巻 UNIT 04
`prop_assert!`	`proptest!` の中で使う `assert!`。	第11巻 UNIT 04
`.prop_map()`	`proptest` で作った値を、関数で変換して、別の形の値にするメソッド（`any::<u64>().prop_map(|n| n.to_string())` で「数字だけの文字列」）。	第11巻 UNIT 04
`TestRunner::new()`	設定（`Config::with_cases(300)` など）を決めて、`proptest` の実行係を作る関数。	第11巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`proptest::collection::vec()`	`proptest` で、「中身をでたらめに作った `Vec` 」を作る作り方。`vec(any::<u8>(), 0..40)` は、0〜39 個のでたらめなバイトの並び。	第11巻 UNIT 04
`Strategy`	`proptest` の「でたらめな値の作り方」を表す型（トレイト）。`impl Strategy<Value = String>` は「文字列を作る作り方」。	第11巻 UNIT 04
`cargo fuzz init --target`	`cargo +nightly fuzz init --target parsers` の `--target` は、fuzz（でたらめな入力で試すテスト）の、試す対象の名前を決めます。	第11巻 UNIT 04
`fuzz_target!`	ファズテスト（でたらめな入力を大量に与えて、止まらないかを確かめる）の中身を書くマクロ（ `libfuzzer-sys` の部品）。	第11巻 UNIT 04
`cargo fuzz run --`	`cargo +nightly fuzz run parsers -- -max_total_time=60` の `--` は、「ここから後ろは、fuzz を動かす係（libFuzzer）に渡す」という区切りです。	第11巻 UNIT 04
`cargo fuzz run -- -max_total_time`	`-max_total_time=60` は、fuzz を 60 秒で止める指定です。付けないと、問題が見つかるまでずっと続けます。	第11巻 UNIT 04
`#[ignore]`	「ふだんは動かさないテスト」にする印（属性）。時間がかかるテストに付けておき、`cargo test -- --ignored` の時だけ動かします。	第11巻 UNIT 06
`cargo mutants --list`	`cargo mutants --list` は、コードをどう書きかえて試すか（変異）の一覧だけを表示し、テストは動かしません。	第11巻 UNIT 06
`cargo mutants --file`	`--file src/auth.rs` は、指定したファイルの中だけを書きかえて試す指定です。	第11巻 UNIT 06
`cargo mutants --timeout`	`--timeout 300` は、1 つの変異のテストを、最長 300 秒で打ち切る指定です。書きかえのせいで終わらなくなったテストを止めます。	第11巻 UNIT 06
`cargo mutants --`	`cargo mutants … -- --test http` の `--` は、「ここから後ろは、`cargo test` に渡す」という区切りです。	第11巻 UNIT 06
`cargo mutants -- -- test`	`-- --test http` は、変異のたびに動かすテストを、`tests\http.rs` だけにする指定です。	第11巻 UNIT 06

性能

コード	意味と使い方	初めて出てくる所
`criterion_group!`	`criterion`（速さを正確に測る部品）で、測る関数をまとめるマクロ。	第11巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`criterion_main!`	`criterion` の測る仕組みの入り口（ `main` ）を作るマクロ。	第11巻 UNIT 08
`.benchmark_group()`	`criterion` で、関係する測り方をまとめるグループを作るメソッド。	第11巻 UNIT 08
`.bench_function()`	`criterion` で、1 つの処理の速さを何回も測るメソッド。	第11巻 UNIT 08
`Criterion`	`criterion` の、速さを測る係の型。	第11巻 UNIT 08
`#[global_allocator]`	プログラム全体のメモリの貸し出し係（アロケーター）を、自分の物に置きかえる印。使ったメモリの量を数える時などに使います。	第11巻 UNIT 08
`.size()`	`Layout` （貸し出すメモリの形）の大きさ（バイト）を返すメソッド。	第11巻 UNIT 08
`GlobalAlloc`	メモリの貸し出し係が持つ機能（トレイト）。	第11巻 UNIT 08
`Layout`	貸し出すメモリの大きさと並びの決まりを表す型。	第11巻 UNIT 08
`System`	OS のふつうのメモリの貸し出し係。自分の係から、本当の貸し出しをこれに任せます。	第11巻 UNIT 08
`oha`	Web サーバーに、たくさんの頼みを一度に送って、速さを測る道具。`oha -n 5000 -c 10 URL` は、5000 回を、同時に 10 本ずつ送ります。	第11巻 UNIT 09
`oha --no-tui`	`--no-tui` は、途中の動く画面を出さず、終わった後に結果だけを文字で表示する指定です。	第11巻 UNIT 09
`oha -n`	`-n 5000` は、送る要求の合計の数です。	第11巻 UNIT 09
`oha -c`	`-c 10` は、同時に送る数（同時に使う人の数のようなもの）です。	第11巻 UNIT 09
`oha -m`	`-m POST` は、要求の種類（メソッド）を指定します。	第11巻 UNIT 09
`oha -T`	`-T application/x-www-form-urlencoded` は、送る中身の種類（ `Content-Type` ）を指定します。	第11巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`oha -d`	`-d "email=…&password=…"` は、送る中身（本文）を指定します。	第11巻 UNIT 09
`oha -H`	`-H "Cookie: …"` は、要求に見出し（ヘッダー）を足します。	第11巻 UNIT 09

データベース

コード	意味と使い方	初めて出てくる所
`params!`	`libsql::params![a, b]` は、SQL の `?1`・`?2` の場所に入れる値を並べるマクロ。値は SQL の文字列につながず、必ずこの形で渡します（SQL インジェクションを防ぐ。第7巻）。	第5巻 UNIT 01
`.connect()`	libSQL の `Database` から、SQL を送るための接続（`Connection`）を取るメソッド。`let conn = db.connect()?;`。	第5巻 UNIT 01
`.execute_batch()`	`;` で区切った、いくつもの SQL の文を、まとめて実行するメソッド（libsql）。表を作る時などに使います。値を入れる文には使わず、`execute` と `params!` を使います。	第5巻 UNIT 06
`.execute()`	値を入れたり変えたりする SQL の文を、1 つ実行するメソッド（libsql）。`conn.execute("UPDATE … WHERE id = ?1", params![id]).await?` のように、値は `params!` で渡します。変わった行の数が返ります。	第5巻 UNIT 10
`Builder::new_local()`	libsql で、手元のファイル（SQLite の形）のデータベースを開く準備をする関数。`.build().await?` で開いた物ができます（第7巻）。	第5巻 UNIT 10
`Database`	libSQL の、データベース（ファイルやサーバー）を表す型。`Builder::new_local("data.db").build().await?` で作り、`.connect()` で接続を取ります。	第5巻 UNIT 10
`.prepare()`	SQL の文を、実行する前に用意（解釈）しておくメソッド（libsql）。同じ文を何度も使う時に速くなり、`?` の数や列の名前も調べられます。	第7巻 この巻で身に付けること
`.columns()`	用意した SQL の文が返す列の一覧を返すメソッド。空なら、行を返さない文（`INSERT` など）です。	第7巻 この巻で身に付けること
`.column_count()`	結果の列の数を返すメソッド。	第7巻 この巻で身に付けること

コード	意味と使い方	初めて出てくる所
`.column_name()`	結果の、何番目かの列の名前を返すメソッド。	第7巻 この巻で身に付けること
`.get_value()`	結果の 1 行から、何番目かの列の値を、型を決めずに（`Value` として）取り出すメソッド。	第7巻 この巻で身に付けること
`Value`	libsql の、型を決めずにデータベースの値を持つ型。`Null`・`Integer`・`Real`・`Text`・`Blob` のどれかです。	第7巻 この巻で身に付けること
`Null`	`Value::Null` は、データベースの値が NULL（値が無い）であることを表す種類です。	第7巻 この巻で身に付けること
`Integer`	`Value::Integer(n)` は、データベースの値が整数であることを表す種類です。	第7巻 この巻で身に付けること
`Real`	`Value::Real(x)` は、データベースの値が小数であることを表す種類です。	第7巻 この巻で身に付けること
`Text`	`Value::Text(s)` は、データベースの値が文字列であることを表す種類です。	第7巻 この巻で身に付けること
`Blob`	`Value::Blob(bytes)` は、データベースの値がバイトの並び（画像などの生のデータ）であることを表す種類です。	第7巻 この巻で身に付けること
`.transaction()`	トランザクション（いくつもの変更を「全部成功か、全部取り消し」にするまとまり）を始めるメソッド。`tx.commit()` で確定、`tx.rollback()`（または片づけ）で取り消しです（第7巻 UNIT 05）。	第7巻 UNIT 04
`.is_autocommit()`	今、トランザクションの外（1 文ずつ自動で確定する状態）なら `true` を返すメソッド。	第7巻 UNIT 05
`.transaction_with_behavior()`	トランザクションの始め方を決めて始めるメソッド。`TransactionBehavior::Immediate` は、始めた時点で書き込みの順番を取ります。	第7巻 UNIT 05
`.busy_timeout()`	ほかの接続が書き込み中で待たされた時に、何秒まで待つかを決めるメソッド。決めないと、すぐに「使用中」のエラーになります。	第7巻 UNIT 05
`TransactionBehavior`	トランザクションの始め方の種類。`Deferred`（必要になってから）・`Immediate`（すぐに書き込みの順番を取る）・`Exclusive`（ほかを全部止める）があります。	第7巻 UNIT 05
`Immediate`	`TransactionBehavior::Immediate` は、トランザクションを始めた時点で、書き込みの順番を取る始め方です。あとで「使用中」で失敗するのを減らせます。	第7巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`.rollback()`	トランザクションの変更を取り消すメソッド。	第7巻 UNIT 06
`.last_insert_rowid()`	いちばん最後に `INSERT` した行の番号（rowid）を返すメソッド。今作ったノートの番号を知る時に使います。	第7巻 UNIT 07
`named_params!`	`libsql::named_params! { ":owner": 1 }` は、SQL の `:owner` のような名前付きの場所に入れる値を並べるマクロ。番号（ `?1` ）の代わりに名前で渡せます。	第7巻 UNIT 07
`.parameter_count()`	用意した SQL の文の、値を入れる場所（ `?` ）の数を返すメソッド。	第7巻 UNIT 07
`SqliteFailure`	`libsql::Error::SqliteFailure(番号, 文)` は、SQLite が返したエラーの種類です。番号で、どんな失敗か（2067 は UNIQUE 違反＝同じ値が既にある）が分かります。	第7巻 UNIT 07
`.sync()`	組み込みレプリカ（手元の写し）を、Turso のサーバーの最新に合わせるメソッド（第7巻 UNIT 08）。	第7巻 UNIT 08
`.sync_interval()`	組み込みレプリカを、決めた間隔で自動で同期（サーバーに合わせる）するように設定するメソッド。	第7巻 UNIT 08
`Builder::new_remote()`	Turso のサーバーにあるデータベースへ、ネット越しにつなぐ準備をする関数。URL と、合言葉（トークン）を渡します。	第7巻 UNIT 08
`Builder::new_remote_replica()`	手元のファイルに写し（レプリカ）を持ちながら、Turso のサーバーと同期するデータベースを開く準備をする関数。読むのは手元から、書くのはサーバーへ、となります（第7巻 UNIT 08）。	第7巻 UNIT 08
`Row`	libSQL で、SQL の結果の 1 行を表す型。`row.get::<i64>(0)` のように、列の番号で値を取り出します。	第7巻 UNIT 09
`IntoParams`	「SQL に渡す値の並びにできる」機能（トレイト。libsql）。関数の引数を `impl IntoParams` にすると、`params![…]` でも `()` でも受け取れます。	第7巻 UNIT 09
`Rows`	`query` が返す、結果の行の並びの型（libsql）。`while let Some(row) = rows.next().await? { … }` で 1 行ずつ取り出します。	第7巻 UNIT 09

SQL

コード	意味と使い方	初めて出てくる所
`CREATE`	`CREATE TABLE 表の名前 ( … )` で、データベースに新しい表を作る SQL の命令。かっこの中に、列の名前と型を並べます。	第0巻 UNIT 07
`TABLE`	SQL で「表」を表す言葉。`CREATE TABLE`（表を作る）・`DROP TABLE`（表を消す）のように使います。	第0巻 UNIT 07
`INTEGER`	SQL の列の型の 1 つで、整数を入れる列。	第0巻 UNIT 07
`PRIMARY`	`PRIMARY KEY`（主キー）の一部。主キーは、表の中で 1 行を必ず見分けられる列で、同じ値が 2 回入りません。ふつうは `id` 列にします。	第0巻 UNIT 07
`KEY`	`PRIMARY KEY`（主キー）・`FOREIGN KEY`（よその表とつなぐ列）などの「キー」。	第0巻 UNIT 07
`TEXT`	SQL の列の型の 1 つで、文字を入れる列。	第0巻 UNIT 07
`NOT`	SQL で「〜ではない」。`NOT NULL` は「この列は空（NULL）を許さない」という決まりです。	第0巻 UNIT 07
`NULL`	SQL で「値が無い」ことを表す特別な値。0 や空の文字列とは違います。	第0巻 UNIT 07
`UNIQUE`	「この列の値は、表の中で重ならない」という決まり。`email TEXT NOT NULL UNIQUE` は、同じメールアドレスを 2 回登録できなくします。	第0巻 UNIT 07
`REFERENCES`	「この列の値は、あの表の、あの列に必ずある値」という決まり（外部キー）。`user_id INTEGER REFERENCES users(id)` で、存在しないユーザーの番号を入れられなくなります。	第0巻 UNIT 07
`CHECK`	「この列に入れてよい値の条件」を決める決まり。`CHECK (length(title) BETWEEN 1 AND 80)` は、題名を 1〜80 文字に限ります。合わない値を入れようとすると、データベースが断ります。	第0巻 UNIT 07
`BETWEEN`	「〜から〜の間」。`CHECK (score BETWEEN 0 AND 100)` は「0 以上 100 以下だけ許す」という決まりです。	第0巻 UNIT 07
`AND`	SQL で「かつ」。`WHERE a = 1 AND b = 2` は両方に合う行だけ。`BETWEEN 0 AND 100` の一部としても使います。	第0巻 UNIT 07
`DEFAULT`	列に値を入れなかった時に使う、決めておいた値（既定値）。`done INTEGER NOT NULL DEFAULT 0` は「指定が無ければ 0」。	第0巻 UNIT 07

コード	意味と使い方	初めて出てくる所
`INSERT`	`INSERT INTO 表 (列, …) VALUES (値, …)` で、表に新しい行を足す SQL の命令。	第0巻 UNIT 07
`INTO`	`INSERT INTO 表` の一部で、「この表へ」入れる、という意味。	第0巻 UNIT 07
`VALUES`	`INSERT INTO 表 (列, …) VALUES (値, …)` の、入れる値を書く所。	第0巻 UNIT 07
`SELECT`	表から行を取り出す SQL の命令。`SELECT 列, … FROM 表 WHERE 条件 ORDER BY 列` の形でよく使います。`SELECT *` は全部の列です。	第0巻 UNIT 07
`FROM`	`SELECT … FROM 表` の一部で、「どの表から」取り出すかを書きます。	第0巻 UNIT 07
`WHERE`	`SELECT … FROM 表 WHERE 条件` の、条件を書く所。`WHERE user_id = 1` は「持ち主が 1 番の行だけ」。`UPDATE`・`DELETE` で `WHERE` を忘れると、全部の行が変わる（消える）ので注意します。	第0巻 UNIT 07
`length()`	SQL の `length(title)` は、文字の数を数える関数です（SQLite では文字の数、バイトの数ではありません）。`CHECK (length(title) BETWEEN 1 AND 80)` は「題は 1〜80 文字」という決まりです。	第0巻 UNIT 07
`'文字'`	SQL では、文字の値を `'…'`（シングルクォート）で囲みます（`'2026-11-05'`・`'山田'`）。`"…"`（ダブルクォート）は表や列の名前に使う物なので、文字の値には使いません。中に `'` を入れる時は `''` と 2 つ書きます。	第0巻 UNIT 07
`;`	SQL の `;` は、1 つの命令（文）の終わりの印です。psql では、`;` を打つまで命令が送られません（`;` を忘れると、続きを待つ `practice-#` の表示になります）。	第0巻 UNIT 07
`?1`	SQL の `?1`・`?2` は、「ここに 1 番目・2 番目の値を入れる」という、あとから値を渡す場所（プレースホルダー）です。値は SQL の文字とは別に渡すので、入力に `'` などが入っていても、SQL の命令として読まれません（SQL インジェクションを防ぎます）。入力の値を、SQL の文字に直接つないではいけません。	第5巻 UNIT 01
`LIMIT`	`LIMIT 件数` で、取り出す行の数の上限を決める書き方。`ORDER BY … LIMIT 20` は「並べて、先頭の 20 件だけ」。	第5巻 UNIT 03
`PRAGMA`	SQLite（libSQL）の設定を見る・変える命令。`PRAGMA foreign_keys = ON;` は、表どうしのつながりの決まり（外部キー）を確かめるようにします。	第5巻 UNIT 06

コード	意味と使い方	初めて出てくる所
`ON`	SQL の「〜の時」「〜で」。`ON DELETE CASCADE` は「消された時は」、`JOIN … ON a.id = b.user_id` は「この条件で横につなぐ」、`CREATE INDEX … ON 表 (列)` は「この表の、この列に」という意味です。	第5巻 UNIT 06
`IF`	`CREATE TABLE IF NOT EXISTS` の一部で、「もし〜なら」。「無ければ作る、あれば何もしない」という意味になります。	第5巻 UNIT 10
`EXISTS`	SQL で「ある」。`IF NOT EXISTS` は「無ければ」という意味です。	第5巻 UNIT 10
`IN`	SQL で「このどれか」。`WHERE status IN ('queued', 'running')` は、状態がどちらかの行だけです。`CHECK (pinned IN (0, 1))` は、0 か 1 しか入れられない決まりです。	第5巻 UNIT 10
`RETURNING`	`INSERT … RETURNING id` で、入れた（変えた）行の値（作られた番号など）を、その場で返してもらう書き方。	第5巻 UNIT 10
`UPDATE`	`UPDATE 表 SET 列 = 値 WHERE 条件` で、行の中身を書きかえる SQL の命令。`WHERE` を忘れると、全部の行が変わります。	第5巻 UNIT 10
`SET`	`UPDATE` の中で、「どの列を、どの値にするか」を書く所。	第5巻 UNIT 10
`OR`	SQL で「または」。`WHERE a = 1 OR b = 2` は、どちらかに合う行です。`AND` と混ぜる時は、`( )` で囲んで順番をはっきりさせます。	第5巻 UNIT 10
`ORDER`	`ORDER BY 列` で、取り出した行を並べる書き方。`DESC` を付けると大きい順（新しい順）です。	第5巻 UNIT 10
`BY`	`ORDER BY`（並べる）・`GROUP BY`（まとめる）の一部で、「〜で」という意味です。	第5巻 UNIT 10
`CASE`	SQL の場合分け。`CASE WHEN 条件 THEN 値 ELSE 別の値 END` で、条件に合う時と合わない時で、違う値を使えます。	第5巻 UNIT 10
`WHEN`	`CASE` の中の「もし〜なら」の部分。`WHEN attempts >= max_attempts THEN 'dead'`。	第5巻 UNIT 10
`THEN`	`CASE WHEN 条件 THEN 値` の「その時は、この値」の部分。	第5巻 UNIT 10
`ELSE`	`CASE` の中の「どの条件にも合わなかった時は、この値」の部分。	第5巻 UNIT 10
`END`	`CASE … END` の終わりの印。	第5巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`COUNT`	行の数を数える SQL の関数。`SELECT COUNT(*) FROM notes` は、ノートの数です。`COUNT(列)` は、その列が NULL でない行の数です。	第5巻 UNIT 10
`GROUP`	`GROUP BY 列` で、同じ値の行をまとめて、まとめごとに数える・足す時の書き方。`SELECT user_id, COUNT(*) FROM notes GROUP BY user_id` は、人ごとのノートの数です。	第5巻 UNIT 10
`COUNT(*)`	SQL の `COUNT(*)` は、行の数を数えます。`GROUP BY status` と組み合わせると、`status` ごとの件数になります。	第5巻 UNIT 10
`IGNORE`	SQLite の `INSERT OR IGNORE` は、「入れようとした行が、決まり（`UNIQUE` など）に合わなかったら、エラーにせず、何もしない」という書き方です。同じ物を 2 回入れても、1 回分だけ残ります。	第5巻 UNIT 10
`DELETE`	`DELETE FROM 表 WHERE 条件` で、表の行を消す SQL の命令。`WHERE` を忘れると、全部の行が消えます。`ON DELETE CASCADE` の中では「消された時」という意味です。	第7巻 この巻で身に付けること
`EXPLAIN`	`EXPLAIN QUERY PLAN SELECT …` で、データベースがその SQL をどう実行するつもりか（索引を使うか、全部の行を見るか）を表示します（第7巻 UNIT 04）。	第7巻 この巻で身に付けること
`QUERY`	`EXPLAIN QUERY PLAN` の一部。	第7巻 この巻で身に付けること
`PLAN`	`EXPLAIN QUERY PLAN` の一部で、「実行の計画」という意味です。	第7巻 この巻で身に付けること
`AS`	SQL で、列や表に別の名前を付ける書き方。`SELECT COUNT(*) AS notes` は、結果の列の名前を `notes` にします。`FROM users AS u` は、表を `u` と短く呼べるようにします。	第7巻 この巻で身に付けること
`--`	SQL では、`--` から行の終わりまでがコメントです。	第7巻 この巻で身に付けること
`SELECT *`	SQL の `SELECT *` の `*` は、「全部の列」という意味です。試しに中身を見る時は便利ですが、プログラムでは、使う列の名前を並べて書くほうが安全です（あとで列が増えても、読む物が変わりません）。	第7巻 UNIT 01
`datetime()`	SQLite の `datetime('now')` は、今の日時（UTC）を `2026-11-05 00:03:00` の形の文字で返します。`DEFAULT (datetime('now'))` で、行を作った日時を自動で入れられます。	第7巻 UNIT 01
`CAST`	SQL で、値の型を変える書き方。`CAST('あいう' AS BLOB)` は、文字列をバイトの並びとして扱います。	第7巻 UNIT 01

コード	意味と使い方	初めて出てくる所
`BLOB`	SQL の型の 1 つで、バイトの並び（画像などの生のデータ）を入れる型。	第7巻 UNIT 01
`STRICT`	SQLite（libSQL）で、表の後ろに付けると、列の型と違う値を断るようにする決まり。付けないと、整数の列に文字が入ってしまうことがあります。	第7巻 UNIT 01
`typeof()`	SQLite の `typeof(値)` は、値の種類（`integer`・`text`・`real`・`blob`・`null`）を返します。	第7巻 UNIT 01
`AUTOINCREMENT`	`INTEGER PRIMARY KEY AUTOINCREMENT` で、番号を自動で振り、消した番号を二度と使わないようにする決まり。	第7巻 UNIT 01
`CASCADE`	`ON DELETE CASCADE` は、「つながった元の行（ユーザー）が消されたら、こちらの行（そのユーザーのノート）も一緒に消す」という決まりです。	第7巻 UNIT 01
`INDEX`	`CREATE INDEX 名前 ON 表 (列)` で、表に索引（本の索引のような、速く探すための仕組み）を作る SQL の命令。よく条件や並べかえに使う列に作ります（第7巻 UNIT 04）。	第7巻 UNIT 01
`DESC`	`ORDER BY 列 DESC` で、大きい順（新しい順）に並べます（小さい順は `ASC`）。	第7巻 UNIT 01
`UNION`	2 つの `SELECT` の結果を、縦につなげる書き方。`UNION ALL` は、重なりも消さずにそのままつなげます。	第7巻 UNIT 01
`ALL`	`UNION ALL` は、2 つの結果を、重なりも消さずにそのままつなげる書き方です。	第7巻 UNIT 01
`OFF`	`PRAGMA … = OFF` で、その設定を切ります（`ON` で入れます）。	第7巻 UNIT 01
`OFFSET`	`LIMIT 件数 OFFSET 飛ばす数` で、「先頭から何件飛ばしてから取り出すか」を決める書き方。ページ送り（2 ページ目なら 20 件飛ばす）に使います。	第7巻 UNIT 02
`LIKE`	SQL のあいまい検索。`WHERE title LIKE '%買い物%'` は、`買い物` を含む行です（`%` は「何文字でも」、`_` は「どれか 1 文字」）。	第7巻 UNIT 02
`ESCAPE`	`LIKE … ESCAPE '\'` で、「`\` の次の文字は、特別な意味の無いただの文字」と決める書き方。検索の言葉に `%` や `_` が入っていても、正しく探せます（第10巻 UNIT 07）。	第7巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`%`	SQL の `LIKE '%買い物%'` の `%` は、「何文字でも、何でも合う」という印です（ワイルドカード）。`'買い物%'` なら「買い物で始まる」。`_` は「何でも 1 文字」です。`%` や `_` そのものを探したい時は、`ESCAPE` を使います。	第7巻 UNIT 02
`JOIN`	2 つの表を、条件で横につないで取り出す書き方。`FROM notes JOIN users ON users.id = notes.user_id` は、ノートと、その持ち主を 1 行に並べます。	第7巻 UNIT 02
`LEFT`	`LEFT JOIN` は、左の表の行は、相手が無くても全部残す JOIN。「ノートが 0 件の人も一覧に出す」時に使います。	第7巻 UNIT 02
`表.列`	SQL の `n.title`・`users.email` のように `表.列` と書くと、「どの表の列か」をはっきりさせられます。いくつもの表をつなぐ（`JOIN`）時は、同じ名前の列（`id` など）があるので、こう書きます。`notes AS n` のように、表に短い別名を付けて使います。	第7巻 UNIT 02
`IS`	SQL で NULL かどうかを調べる時の書き方。`WHERE x IS NULL`（無い）・`WHERE x IS NOT NULL`（ある）。NULL は `= NULL` では比べられません。	第7巻 UNIT 02
`ASC`	`ORDER BY 列 ASC` で、小さい順（古い順）に並べます。書かなくても小さい順になります（大きい順は `DESC`）。	第7巻 UNIT 02
`WITH`	`WITH 名前 AS (SELECT …)` で、その文の中だけで使う「一時的な表」に名前を付ける書き方。長い SQL を読みやすく分けられます。	第7巻 UNIT 04
`RECURSIVE`	`WITH RECURSIVE` は、自分自身を使って行を増やしていく一時的な表を作る書き方。1〜1000 の数を作る、などに使えます。	第7巻 UNIT 04
`||`	SQL の `||` は、文字と文字をつなぐ記号です。`'user' || 42 || '@example.com'` → `user42@example.com`。	第7巻 UNIT 04
`lower()`	SQL の `lower(email)` は、英字を小文字にします。大文字・小文字を区別せずに比べる時に使います。	第7巻 UNIT 04
`DROP`	`DROP TABLE 表`・`DROP INDEX 索引` で、表や索引を消す SQL の命令。中のデータも消えるので注意します。	第7巻 UNIT 04
`BEGIN`	トランザクションを始める SQL の命令。ここから `COMMIT`（確定）か `ROLLBACK`（取り消し）までが、1 つのまとまりになります。	第7巻 UNIT 05
`ROLLBACK`	トランザクションの中でした変更を、全部取り消す SQL の命令。	第7巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`COMMIT`	トランザクションの中でした変更を、確定する SQL の命令。確定するまでは、ほかの人からは見えません。	第7巻 UNIT 05
`ALTER`	`ALTER TABLE 表 …` で、もうある表の形を変える SQL の命令（列を足す・名前を変える など）。	第7巻 UNIT 06
`ADD`	`ALTER TABLE 表 ADD COLUMN 列 型` で、表に列を足します。	第7巻 UNIT 06
`COLUMN`	SQL で「列」。`ADD COLUMN`（列を足す）・`RENAME COLUMN`（列の名前を変える）のように使います。	第7巻 UNIT 06
`RENAME`	`ALTER TABLE 表 RENAME TO 新しい名前` で、表の名前を変えます。	第7巻 UNIT 06
`TO`	`RENAME TO 新しい名前` の「〜へ」の部分です。	第7巻 UNIT 06
`printf()`	SQLite の `printf('%10001s', '')` は、書式に従って文字を作る関数です。`%10001s` は「10001 文字の幅（足りない分は空白）」なので、ここでは空白が 10001 個の文字を作ります。	第7巻 UNIT 06
`:名前`	SQL の `:owner` のように `:` で始まる名前は、名前で値を渡す場所（名前付きのプレースホルダー）です。	第7巻 UNIT 07
`COALESCE`	`COALESCE(値1, 値2)` は、最初の NULL でない値を返す SQL の関数。`COALESCE(SUM(done), 0)` は「合計が NULL（行が無い）なら 0」。	第7巻 UNIT 09
`SUM`	列の値の合計を出す SQL の関数。	第7巻 UNIT 09
`VACUUM`	SQLite の `VACUUM` は、データベースのファイルを詰め直して、消した行の分の空きを無くす命令です。`VACUUM INTO 'copy.db'` は、開いたまま、別のファイルに写しを書き出します（バックアップに使えます）。	第7巻 UNIT 10
`unixepoch()`	SQLite の `unixepoch()` は、今の日時を、1970 年 1 月 1 日からの秒数（UNIX 時間）で返します。期限が過ぎたかを、数の比べっこで調べられます。	第8巻 UNIT 01
`count()`	SQL の `count(*)` は、行の数を数える関数です（`COUNT(*)` と同じで、大文字・小文字はどちらでもかまいません）。	第8巻 UNIT 01
`substr()`	SQL の `substr(文字, 1, 8)` は、文字の 1 文字目から 8 文字分を取り出します。	第8巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`instr()`	SQLite の `instr(email, '@')` は、文字の中で、`@` が何文字目にあるかを返します（無ければ 0）。	第8巻 UNIT 05
`DISTINCT`	SQL の `SELECT DISTINCT` は、同じ値の行を 1 つにまとめて（重なりを消して）返します。	第10巻 UNIT 01
`WAL`	SQLite の `PRAGMA journal_mode = WAL` の `WAL` は、書き込みを、まず別のファイル（`-wal`）に書いておく方式です。読む人と書く人が、互いに待たされにくくなります。	第13巻 UNIT 02

JSON

コード	意味と使い方	初めて出てくる所
`serde_json::to_string()`	Rust の値を、JSON の文字列（1 行）に変える関数。値の型に `Serialize` が付いている必要があります。失敗することもあるので `Result` が返ります。	第4巻 UNIT 07
`Serialize`	「JSON などにできる」機能（トレイト。`serde` の部品）。`#[derive(Serialize)]` を付けた型は、`serde_json::to_string` で JSON にできます。	第4巻 UNIT 07
`serde_json::to_string_pretty()`	Rust の値を、人が読みやすい字下げ付きの JSON の文字列に変える関数。	第4巻 UNIT 08
`Deserialize`	「JSON などから作れる」機能（トレイト。`serde` の部品）。`#[derive(Deserialize)]` を付けた型は、`serde_json::from_str` で JSON から作れます。逆向き（値 → JSON）は `Serialize` です。	第4巻 UNIT 08
`#[serde]`	`serde` の印（属性）。JSON にする時の細かい決まりを付けます。`#[serde(rename_all = "camelCase")]`（名前の書き方を変える）・`#[serde(default)]`（無い時は初期値）・`#[serde(skip_serializing_if = "Option::is_none")]`（空なら書かない）などがあります。	第4巻 UNIT 08
`#[serde(deny_unknown_fields)]`	`serde` の印。JSON に、構造体に無い名前の項目が入っていたら、読み込みをエラーにします。項目の名前の書き間違い（`titel` など）を見逃さないために付けます。	第4巻 UNIT 08
`#[serde(default)]`	`serde` の印。JSON にその項目が無い時、エラーにせず、型の初期値（`Default`。数なら 0、文字列なら空）を入れます。	第4巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`#[serde(rename_all)]`	`#[serde(rename_all = "camelCase")]` は、項目の名前を、JSON では別の書き方にする印です。`camelCase` なら、Rust の `width_mm` が JSON では `widthMm` になります。`snake_case`・`kebab-case`・`UPPERCASE` などもあります。	第4巻 UNIT 08
`#[serde(try_from)]`	`#[serde(try_from = "String")]` は、「JSON の値を、いったん `String` として読み、それから `TryFrom` でこの型に変える」という印です。変える時の確かめ（空でないか、など）が失敗したら、読み込みのエラーになります。	第4巻 UNIT 08
`json!`	`serde_json::json!({ "ok": true })` は、JSON の形をそのまま書いて、JSON の値を作るマクロです。	第14巻 B. サンプル集

Topcoat

コード	意味と使い方	初めて出てくる所
`topcoat`	Topcoat の道具のコマンド（`cargo install topcoat-cli` で入る）。`topcoat dev`（コードを直すと自動で作り直して読み直す開発用の起動）・`topcoat asset bundle`（CSS などの束を作る）などがあります（第6巻）。	第1巻 UNIT 08
`topcoat --version`	`topcoat --version` は、topcoat CLI（Topcoat の道具）の版を表示します。アプリで使う Topcoat の版と同じかを確かめます。	第1巻 UNIT 08
`topcoat --help`	`topcoat --help` は、topcoat CLI のコマンドとオプションの一覧を表示します。	第1巻 UNIT 08
`topcoat new --minimal`	`topcoat new 名前 --minimal` の `--minimal` は、いちばん小さなひな形（1 ページだけ）でプロジェクトを作る指定です。	第1巻 UNIT 08
`topcoat new --no-interactive`	`--no-interactive` は、質問をせずに、決まりの答えで作る指定です。	第1巻 UNIT 08
`module_router!`	Topcoat のマクロ。Rust のモジュール（ファイル）の並びを、そのまま URL の並びにしたルーター（URL ごとに、どの関数に答えさせるかの名簿）を作ります（第6巻 UNIT 03）。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`view!`	Topcoat のマクロ。中に HTML をほぼそのままの形で書いて、画面（`View`）を作ります。文字は `"…"` で囲み、Rust の値は `{値}` で埋め込みます。埋め込んだ文字の `<` などは、自動で安全な形に置きかえられます（第6巻 UNIT 05）。	第1巻 UNIT 08
`asset!`	Topcoat のマクロ。CSS や画像などのファイル（アセット）を「このファイルを配る」と宣言し、ブラウザーから読める URL を作ります。`asset!("styles.css")` を、`<link>` の `href` などに入れて使います（第6巻 UNIT 08）。	第1巻 UNIT 08
`#[layout]`	Topcoat の印（属性）。この関数を「どのページにも共通の外枠（HTML の骨組み・メニュー）」にします。外枠の中の、ページが入る場所を `slot` で受け取ります（第6巻 UNIT 03）。	第1巻 UNIT 08
`#[page]`	Topcoat の印（属性）。この関数を、1 つの URL に答える「ページ」にします。`#[page("/")]` なら `/` に、`#[page(POST)]` なら送信（POST）に答えます（第6巻 UNIT 02〜03）。	第1巻 UNIT 08
`.assets()`	ルーターに、配るアセット（CSS など）の束を登録するメソッド。`.assets(AssetBundle::load().unwrap())`（第6巻 UNIT 08）。	第1巻 UNIT 08
`AssetBundle::load()`	`topcoat asset bundle` で作った CSS などの束（アセット）を読み込む関数。`.assets(AssetBundle::load().unwrap())` のように、ルーターに渡して配れるようにします（第6巻 UNIT 08）。	第1巻 UNIT 08
`topcoat::dev::script()`	`topcoat dev` で動かしている時だけ、「コードを直したら、ブラウザーを自動で読み直す」ための小さなスクリプトを、ページに入れる関数。`cargo run` や本番では何も出しません（第6巻 UNIT 08）。	第1巻 UNIT 08
`AssetBundle`	Topcoat の型。`topcoat asset bundle` で作った「アセットの束」（名前にハッシュが付いたファイルと目録）を、`AssetBundle::load()` で読み込んで、配れるようにします（第6巻 UNIT 08）。	第1巻 UNIT 08
`RouterBuilderAssetExt`	Topcoat のトレイト（機能のまとまり）。`use` しておくと、ルーターを組み立てる時に、アセット（CSS など）を配る設定のメソッドが使えるようになります。	第1巻 UNIT 08
`Router`	URL とメソッド（GET・POST）ごとに、どの関数に答えさせるかを決めた名簿（ルーター）。Topcoat のサーバーは、来た頼み（リクエスト）をこの名簿で振り分けます（第6巻 UNIT 03）。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`Slot`	外枠（`#[layout]`）の関数が受け取る、「中に入るページの場所」を表す型。外枠の `view!` の中で `(slot)` と書いた所に、各ページの中身が入ります（第6巻）。	第1巻 UNIT 08
`View`	Topcoat の「画面（HTML になる物）」の型（トレイト）。ページの関数は `Result<impl View>`（HTML になる何かを返す。失敗することもある）を返します。	第1巻 UNIT 08
`topcoat asset bundle --release`	`topcoat asset bundle --release` は、CSS などのアセットの束を、本番用（`target\release\assets`）に作ります。`cargo build --release` と組にして使います。	第1巻 UNIT 09
`topcoat::start()`	組み立てたルーターを受け取って、Web サーバーを動かし始める関数。`topcoat::start(router).await` は、止めるまでリクエストを待ち続けます。	第5巻 UNIT 01
`href!`	Topcoat のマクロ。ページの関数から、その URL を作ります。`href!(id::show, id::Id(note.id))` は「このノートの詳細ページの URL」。URL を文字で書かないので、打ちまちがいがコンパイルの時に見つかります（第6巻 UNIT 04）。	第5巻 UNIT 08
`.resolve()`	`href!` で作った URL の部品を、今のリクエスト（`cx`）に合わせて、本物の URL の文字列にするメソッド。	第5巻 UNIT 08
`POST`	`#[page(POST)]` の `POST` は、「フォームの送信（POST）に答えるページ」にする指定。何も書かなければ、ふつうの表示（GET）に答えます（第6巻 UNIT 06）。	第5巻 UNIT 08
`Cx`	Topcoat の「今のリクエストの情報（URL・クッキー・セッションなど）が入った箱」。ページの関数の引数 `cx: &Cx` で受け取り、`href!(…).resolve(cx)` などで使います（第6巻 UNIT 04）。	第5巻 UNIT 08
`Form`	Topcoat の、送られてきたフォームの中身を受け取る型。`Form(input): Form<NoteForm>` と書くと、フォームの項目が `NoteForm` の形に読み取られて `input` に入ります（第6巻 UNIT 06）。	第5巻 UNIT 08
`Id`	URL の一部（`/notes/3` の `3` など）を、型として受け取るための入れ物。この本の RustNote では、`id::Id(番号)` の形で、URL の中のノートの番号を表します。	第5巻 UNIT 08
`#[layer]`	Topcoat の印（属性）。全部（またはある範囲）のページの前後に、決まった処理（アクセスのログ・ログインの確かめなど）をはさむ「層」を作ります（第6巻 UNIT 03・第5巻）。	第5巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`Next`	`#[layer]` の関数で受け取る「次の処理」。`next.run(…)` を呼ぶと、その先（ページ）が動いて、答えが返ってきます。前後に自分の処理をはさめます。	第5巻 UNIT 09
`SeeOther`	「303 See Other」（別のページへ移って）という答えを表す型。フォームを送ったあと、一覧や詳細のページへ移す時（PRG）に使います（第6巻 UNIT 06）。	第5巻 UNIT 09
`.discover()`	`Router::builder().discover()` で、`#[page]` などの付いた関数を全部自動で集めて、ルーターに登録するメソッド（第6巻 UNIT 02）。	第6巻 UNIT 02
`Router::builder()`	Topcoat のルーター（URL ごとの名簿）を組み立て始める関数。`Router::builder().discover().build()` は、`#[page]` を付けた関数を全部自動で集めて（`discover`）、ルーターを作ります（第6巻 UNIT 02）。	第6巻 UNIT 02
`RouterBuilderDiscoverExt`	`.discover()`（`#[page]` の付いた関数を自動で集める）を使えるようにする機能（トレイト）。`use` しておく必要があります。	第6巻 UNIT 02
`class!`	Topcoat のマクロ。HTML の `class` の値を、条件つきで組み立てます。`class!("nav", "active" if on_about)` は、`on_about` が `true` の時だけ `active` を足して `"nav active"` にします。	第6巻 UNIT 03
`#[route]`	Topcoat の印（属性）。画面（HTML）ではなく、JSON やただの文字などを返す「ルート」にします。`#[route(GET)]` のように、メソッドを必ず書きます。人が見る画面は `#[page]`、プログラムが読む答えは `#[route]` と使い分けます。	第6巻 UNIT 03
`.is_current()`	`href!(…)` で作ったリンク先が、「今開いているページ」なら `true` を返すメソッド。メニューの今のページに印（`aria-current`）を付ける時に使います。	第6巻 UNIT 03
`.cookies()`	ルーターに、クッキーを使う仕組みを登録するメソッド（`module_router!().cookies()`）。セッションを使う時も、先にこれを登録します。	第6巻 UNIT 03
`.sessions()`	ルーターに、セッション（ログインの状態を覚える仕組み）を登録するメソッド。`.sessions(SessionConfig::default())`。先に `.cookies()` も登録します（第8巻）。	第6巻 UNIT 03
`.app_context()`	アプリ全体で共有する物（データベースの接続など）をルーターに登録するメソッド。ページの関数からは、`cx` を通して取り出せます（第6巻 UNIT 07）。	第6巻 UNIT 03

コード	意味と使い方	初めて出てくる所
`SessionConfig::default()`	Topcoat のセッション（ログインの状態などを覚える仕組み）を、おすすめの設定で用意する書き方。`.sessions(SessionConfig::default())` でルーターに登録します（第6巻 UNIT 10）。	第6巻 UNIT 03
`AssetConfig`	Topcoat のアセット（CSS・画像など）の配り方の設定の型。本番は束（`AssetBundle`）から、テストは `AssetConfig::hosted_at(…)` のように、場面に合わせて渡します。	第6巻 UNIT 03
`SessionConfig`	Topcoat のセッションの設定（クッキーの名前・期限など）の型。	第6巻 UNIT 03
`module_param!`	Topcoat のマクロ。モジュールの区切りを「URL の変わる部品」にします。`mod id` の中に `module_param!(id: u64, error = not_found);` と書くと、URL が `/notes/{id}` になり、数として読めない時は 404 になります（第6巻 UNIT 04）。	第6巻 UNIT 04
`#[query_params]`	Topcoat の印（属性）。`?q=rust&page=2` のようなクエリ（URL の `?` の後ろ）を読むための構造体に付けます。項目は `Option<T>` にしておき、無い・空の時は `None` になります（第6巻 UNIT 04）。	第6巻 UNIT 04
`Name`	`module_param!(name: String)` で作られる、URL の部品の型（名前の頭を大文字にした形）。`path_param::<Name>(cx)` で、その部品の値を読みます。	第6巻 UNIT 04
`#[query_params(error)]`	`#[query_params(error = bad_request)]` の `error = …` は、クエリ（URL の `?page=abc` など）を読めなかった時に返す答えを決める指定です。`bad_request` なら 400、`not_found` なら 404 を返します。	第6巻 UNIT 04
`#[component]`	Topcoat の印（属性）。画面の一部を作る、何度も使える部品（コンポーネント）にします。`view!` の中で `panel(title: "一覧", …)` のように、名前付きの引数で呼べます（第6巻 UNIT 05）。	第6巻 UNIT 05
`#[default]`	コンポーネントの引数に付ける印。`#[default] child: Child<'_>` は、「名前を付けずにタグの間に書いた中身が、この引数に入る」という意味です。（`#[derive(Default)]` の中で、どの種類を初期値にするかの印にも使われます。）	第6巻 UNIT 05
`#[key]`	`view!` の中の `for` に付ける印。くり返しの 1 つずつを、何で見分けるか（ノートの番号など）を決めます。画面を部分的に描き直す時に、正しく対応させるために使います。	第6巻 UNIT 05
`Child`	コンポーネントの「中身」（タグの間に書いた物）を受け取る型。部品の中で `{child}` と書いた所に置かれます。	第6巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`.push_string()`	Topcoat の画面を作る時に、文字を書き足すメソッド。入れた文字は、置く場所に合わせて自動でエスケープ（`<` などを安全な形に置きかえ）されます。	第6巻 UNIT 05
`NodeViewParts`	`view!` の `( )` に入れられる物が持っている機能（トレイト）。文字・数・`bool` などは最初から持っています。自分の型（`NoteId` など）を入れたい時は、これを実装します。	第6巻 UNIT 05
`PartsWriter`	`NodeViewParts` を実装する時に受け取る「書き込み先」。`push_string` などで、画面の中身を書きます。	第6巻 UNIT 05
`Json`	本文を JSON で返すための型（`topcoat::router::content::Json`）。`Json(Health { … })` のように包んで返すと、JSON の文字に変えて送ってくれます。	第6巻 UNIT 06
`#[memoize]`	Topcoat の印（属性）。付けた関数の結果を、**1 つの要求（リクエスト）の間だけ**覚えておきます。レイアウト・ページ・部品のどこから呼んでも、データベースを 1 回しか読まずに済みます（第6巻 UNIT 07）。	第6巻 UNIT 07
`#[memoize(as_ref)]`	`#[memoize(as_ref)]` の `as_ref` は、覚えておいた結果を、複製せずに「借りる形」で返す指定です。`Option<User>` を返す関数なら、呼んだ側には `Option<&User>` が返ります。付けない `#[memoize]` は `&Option<User>` を返します。	第6巻 UNIT 07
`Asset`	`asset!` で宣言した、配るファイル（CSS・画像など）1 つを表す型。ページの中で URL として使えます。	第6巻 UNIT 08
`topcoat asset bundle --bin`	`topcoat asset bundle --bin u08_assets` の `--bin` は、`src\bin` の中のどのプログラムのアセットを束ねるかを指定します。	第6巻 UNIT 08
`topcoat dev --bin`	`topcoat dev --bin u08_dev` は、開発用に動かすコマンドです。ファイルを保存するたびに、作り直して動かし直します。`--bin` は、どのプログラムかを選びます。	第6巻 UNIT 08
`#[procedure]`	Topcoat の印（属性）。ブラウザーの中のボタンなどから、サーバーの関数を呼べるようにします（中では `POST /api/…` という通信になります。第6巻 UNIT 09）。	第6巻 UNIT 09
`.runtime()`	ルーターに、ブラウザーの中で画面を変える仕組み（runtime）を登録するメソッド（第6巻 UNIT 09）。	第6巻 UNIT 09
`topcoat::runtime::script()`	ブラウザーの中で画面を変える仕組み（runtime）を使うためのスクリプトを、ページに入れる関数（第6巻 UNIT 09）。	第6巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`RouterBuilderRuntimeExt`	ルーターに `.runtime()`（ブラウザーの中で画面を変える仕組み）を登録できるようにする機能（トレイト）。	第6巻 UNIT 09
`@click=`	`view!` の中の `@click=$(\|_e\| …)` は、「押された時に、ブラウザーの中で動かす処理」を付ける書き方です。サーバーには送らずに、ブラウザーの中だけで動きます（`$( )` の中は JavaScript になります）。	第6巻 UNIT 09
`:hidden=`	`view!` の中の `:hidden=$(…)` は、「ブラウザーの中で、値が変わるたびに付け直す属性」の書き方です（結び付けの属性）。`$( )` の中が `true` の時は `hidden`（隠す）が付き、`false` の時は外れます。	第6巻 UNIT 09
`$( … )`	`view!` の中の `$( … )` は、「ブラウザの中でも動く式」です。サーバーが最初の画面を作る時に計算し、さらに同じ式を JavaScript にしてページに入れます。signal の値が変わるたびに、ブラウザがその式を計算し直して、画面を書きかえます。ふつうの `( … )` は、サーバーで 1 回計算するだけです。	第6巻 UNIT 09
`not_found!`	Topcoat のマクロ。その場で「404 Not Found（見つかりません）」の答えにします。	第6巻 UNIT 10
`.description()`	`BadRequestError` などに入れた、利用者向けの説明の文を取り出すメソッド。	第6巻 UNIT 10
`.ok_or_not_found()`	Topcoat のメソッド。`Option` が `None`（無い）の時に、404 Not Found のエラーにします。`… .ok_or_not_found()?` と書くと、無い時はそこで 404 を返します（`RouterErrorExt` を `use` して使う）。	第6巻 UNIT 10
`BadRequestError`	Topcoat の「400 Bad Request（頼み方がおかしい）」のエラーの型。`bad_request("検索の言葉を入れてください")` で作ります。文は利用者に見えるので、直し方が分かる短い文にします。	第6巻 UNIT 10
`NotFoundError`	Topcoat の「404 Not Found（見つかりません）」のエラーの型。	第6巻 UNIT 10
`RouterErrorExt`	`.ok_or_not_found()` などを使えるようにする機能（トレイト）。	第6巻 UNIT 10
`session::token_hash()`	Topcoat の関数。今のリクエストのセッションの印（トークン）の、ハッシュを返します。データベースの `sessions` 表に、ハッシュで記録しておき、ログイン中かを調べるのに使います（第8巻 UNIT 05）。	第6巻 UNIT 10
`session::stop()`	Topcoat のセッションを終わらせる（ログアウトする）関数。消したセッションの番号を返すので、データベースからも消せます（第8巻）。	第6巻 UNIT 10
`RouterBuilderCookieExt`	ルーターに `.cookies()`（クッキーを使う仕組み）を登録できるようにする機能（トレイト）。	第6巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`RouterBuilderSessionExt`	ルーターに `.sessions(…)`（セッションの仕組み）を登録できるようにする機能（トレイト）。セッションはクッキーを使うので、先に `.cookies()` も登録します。	第6巻 UNIT 10
`TokenHash`	セッションの印（トークン）のハッシュの型。トークンそのものではなくハッシュを保存しておくと、データベースが盗まれても、そのままではログインに使えません。	第6巻 UNIT 10
`RemoteAddr`	相手（ブラウザー側）の IP アドレスを表す型。ログインの回数の制限などで、どこからの頼みかを見分けるのに使います。	第8巻 UNIT 10
`Token::new()`	セッションの印（トークン）を、決めた中身から作る関数（テストで、わざとにせのトークンを作る時に使います）。	第8巻 UNIT 10
`Token`	`topcoat::session::Token` は、セッションの印（でたらめな長い文字列）の型。ブラウザーのクッキーに入れ、サーバーには、そのハッシュだけを保存します。	第8巻 UNIT 10
`AssetBundle::load_dir()`	指定したフォルダから、アセットの束（`topcoat asset bundle` で作った物）を読み込む関数（第12巻）。	第9巻 UNIT 10
`TokenHash::new()`	セッションの印のハッシュを、決めた中身から作る関数（テスト用）。	第11巻 UNIT 02
`AssetConfig::hosted_at()`	アセット（CSS など）を、決めた URL の場所から配る設定を作る関数。テストで、束を作らずに CSS の URL だけを決める時に使います。	第13巻 UNIT 10
`Manifest`	アセットの目録（元の名前と、ハッシュの付いた本当の名前の対応表）の型。	第13巻 UNIT 10
`ManifestEntry`	アセットの目録の 1 行（1 つのファイルの情報）の型。	第13巻 UNIT 10

RustNote で作る物

コード	意味と使い方	初めて出てくる所
`Action`	RustNote のノートに対する操作（公開する・アーカイブするなど）を表す enum。この巻の後の方（「もう一歩」の所）で作ります。	第4巻 UNIT 10
`NoteStatus`	RustNote のノートの状態（下書き・公開・アーカイブ）を表す enum。この巻の後の方（「もう一歩」の所）で作ります。	第4巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`TransitionError`	RustNote で、ノートの状態を、決まりにない形で変えようとした時のエラーを表す型。この巻の後の方で作ります。	第4巻 UNIT 10
`Progress`	練習問題で作る、進み具合を表す enum。この巻の解答と解説で作ります。	第4巻 UNIT 10
`Db::open()`	RustNote の `Db` を、設定（`config`）に書かれた場所で開く関数。全体のコードは第13巻に載っています。	第5巻 UNIT 01
`db::migrate()`	RustNote の、テーブルを作る・変える SQL（マイグレーション）を順に流す関数。くわしくは第7巻、全体のコードは第13巻に載っています。	第5巻 UNIT 01
`Db`	RustNote の、データベース（libSQL）の入れ物の型。`Db::open(…)` で開き、`db.connect()` で接続を取ります。全体のコードは第13巻（完全コード）に載っています。	第5巻 UNIT 01
`UserRepository`	RustNote で、利用者（ユーザー）をデータベースに入れる・探す仕事をまとめたトレイト（決まりごと）。`Arc<dyn UserRepository + Send + Sync>` は「この決まりを守る物なら何でも入る、スレッドをまたいで共有できる箱」です。全体のコードは第13巻に載っています。	第5巻 UNIT 02
`SessionRepository`	RustNote で、ログインの記録（セッション）を入れる・探す・消す仕事をまとめたトレイト。全体のコードは第13巻に載っています。	第5巻 UNIT 02
`Password`	パスワードを入れる専用の型（`struct Password(String)`）。ふつうの `String` と区別することで、うっかりログに出すなどの間違いを防ぎます。この巻の後の方で作ります。	第5巻 UNIT 07
`flash::set()`	RustNote で、次の画面に 1 回だけ出すお知らせ（フラッシュ。「ノートを作りました」など）を決める関数。第10巻で作ります。	第5巻 UNIT 08
`NoteForm`	ノートの作成・編集のフォームから送られてくる値（題名と本文）を受け取る struct。`Form(input): Form<NoteForm>` で受け取ります。第10巻で作ります。	第5巻 UNIT 08
`Flash`	RustNote の、画面に 1 回だけ出すお知らせの種類を表す enum。`Flash::NoteCreated`（作りました）・`Flash::LoggedOut`（ログアウトしました）などがあります。全体のコードは第13巻に載っています。	第5巻 UNIT 08
`NoteCreated`	`Flash` の値の 1 つで、「ノートを作りました」のお知らせ。	第5巻 UNIT 08
`.delete_expired()`	`SessionRepository` のメソッドで、期限が切れたセッションをまとめて消します。全体のコードは第13巻に載っています。	第5巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`routes::build_router()`	RustNote の画面（ページ）を全部登録したルーターを組み立てる関数。第6巻の後の方と第10巻で作ります。	第6巻 UNIT 02
`config::listen_address()`	RustNote の、待ち受ける場所（`127.0.0.1:3000` など）を、環境変数から決める関数。全体のコードは第13巻に載っています。	第6巻 UNIT 02
`NoteQuery::new()`	`NoteQuery` を作る関数。ページ番号が無い・0 の時は 1 にするなど、値を整えてから作ります。	第6巻 UNIT 04
`NoteQuery`	ノートの一覧の、検索の言葉とページ番号をまとめた struct。`NoteQuery::new(検索語, ページ)` で作ります。全体のコードは第13巻に載っています。	第6巻 UNIT 04
`LoggedOut`	`Flash` の値の 1 つで、「ログアウトしました」のお知らせ。	第6巻 UNIT 06
`Health`	RustNote の `/health`（動いているかを確かめる URL）が返す中身の struct。第9巻で作ります。	第6巻 UNIT 06
`Db::open_local()`	RustNote の `Db` を、手元のファイル（`data/rustnote.db` など）で開く関数。全体のコードは第13巻に載っています。	第7巻 UNIT 08
`GuardError`	練習で作る、データベースの見張り（決まりに合わない変更を止める）のエラーの enum。この巻の解答と解説で作ります。	第7巻 UNIT 10
`.sign_up()`	テスト用の `TestApp` のメソッドで、利用者を登録して、ログイン済みのクッキーを返します。全体のコードは第13巻に載っています。	第8巻 UNIT 01
`.post_form()`	テスト用の `TestApp` のメソッドで、フォームの値を POST で送って、答えを受け取ります。	第8巻 UNIT 01
`TestApp`	テストのたびに、空のデータベースとルーターを用意してくれる、テスト用の struct。`tests/common/mod.rs` に置きます。全体のコードは第13巻に載っています。	第8巻 UNIT 01
`.delete()`	`SessionRepository`（セッションの出し入れ）のメソッドで、決めたセッションを消します。	第8巻 UNIT 01
`SignupOutcome`	登録の結果（できた・断った）を表す enum。`SignupOutcome::Rejected(状態コード, フォーム)` は、入力をやり直してもらう時の値です。全体のコードは第13巻に載っています。	第8巻 UNIT 03
`Rejected`	`SignupOutcome` の値の 1 つで、「登録を断った（入力をやり直してもらう）」を表します。	第8巻 UNIT 03

コード	意味と使い方	初めて出てくる所
`.find_by_email()`	`UserRepository` のメソッドで、メールアドレスから利用者を探します。見つからなければ `None` です。	第8巻 UNIT 03
`LoggedIn`	`Flash` の値の 1 つで、「ログインしました」のお知らせ。	第8巻 UNIT 04
`NotePage`	ノートの一覧の 1 ページ分（ノートの並びと、全部の数・ページ番号）をまとめた struct。全体のコードは第13巻に載っています。	第8巻 UNIT 07
`.with_errors()`	`NoteFormState`（フォームの状態）に、項目ごとのエラーの文を足して返すメソッド。第10巻で作ります。	第9巻 UNIT 02
`NoteFormState::from_input()`	入力された題名と本文から、`NoteFormState` を作る関数。第10巻で作ります。	第9巻 UNIT 02
`NoteFormState`	フォームに出す値とエラーの文をまとめた struct。入力にまちがいがあった時、入れた値を残したまま、エラーを出し直すのに使います。第10巻で作ります。	第9巻 UNIT 02
`NoteBody`	ノートの本文を入れる専用の型（`struct NoteBody(String)`）。長さなどの決まりを確かめてからでないと作れないので、まちがった本文が入り込みません。全体のコードは第13巻に載っています。	第9巻 UNIT 03
`LibsqlUserRepository::new()`	`LibsqlUserRepository` を、データベース（`Db`）を受け取って作る関数。	第9巻 UNIT 07
`LibsqlUserRepository`	`UserRepository` の決まりを、libSQL のデータベースで守る（実装する）struct。全体のコードは第13巻に載っています。	第9巻 UNIT 07
`flash::take()`	RustNote で、決めておいたお知らせ（フラッシュ）を取り出す関数。取り出すと消えるので、1 回だけ表示されます。第10巻で作ります。	第10巻 UNIT 01
`NoteInputErrors`	ノートの入力のエラー（題名・本文それぞれの、まちがいの文）をまとめた struct。全体のコードは第13巻に載っています。	第10巻 UNIT 02
`NoteToggled`	`Flash` の値の 1 つで、「完了・未完了を切りかえました」のお知らせ。	第10巻 UNIT 05
`NoteDeleted`	`Flash` の値の 1 つで、「ノートを消しました」のお知らせ。	第10巻 UNIT 06
`.code()`	`Flash` の値を、クッキーに入れる短い文字に変えるメソッド。全体のコードは第13巻に載っています。	第10巻 UNIT 09
`Flash::from_code()`	クッキーに入れておいた短い文字（コード）から、`Flash` の値を作り直す関数。全体のコードは第13巻に載っています。	第10巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.message()`	`Flash` の値から、画面に出す文（「ノートを作りました」など）を返すメソッド。全体のコードは第13巻に載っています。	第10巻 UNIT 09
`LibsqlSessionRepository::new()`	`LibsqlSessionRepository` を、データベース（`Db`）を受け取って作る関数。	第11巻 UNIT 02
`LibsqlSessionRepository`	`SessionRepository`（セッションの出し入れ）の決まりを、libSQL のデータベースで守る（実装する）struct。全体のコードは第13巻に載っています。	第11巻 UNIT 02
`UserRecord`	データベースから読んだ利用者の 1 行（番号・メールアドレス・パスワードのハッシュなど）をまとめた struct。	第13巻 UNIT 05
`LoginRequired`	`Flash` の値の 1 つで、「ログインしてください」のお知らせ。	第13巻 UNIT 06

HTML

コード	意味と使い方	初めて出てくる所
`DOCTYPE`	`<!DOCTYPE html>` は、「この文書は HTML5 です」とブラウザーに伝える、HTML の先頭の決まった 1 行です。	第1巻 UNIT 08
`<!DOCTYPE html>`	`<!DOCTYPE html>` は、「これは今の HTML（HTML5）で書いたページです」という、ページのいちばん最初に書く宣言です。大文字でも小文字（`<!doctype html>`）でもかまいません。	第1巻 UNIT 08
`<html>`	HTML の文書全体を包むタグ。中に `<head>`（ページの情報）と `<body>`（画面に出す中身）を入れます。	第1巻 UNIT 08
`lang=`	`<html lang="ja">` の `lang` は、ページの言葉（`ja` は日本語、`en` は英語）です。読み上げソフトの読み方や、ブラウザーの翻訳の判断に使われます。	第1巻 UNIT 08
`<head>`	`<head>` は、ページの「見えない設定」（文字の符号・題名・CSS の読み込みなど）を書く所です。画面に出る中身は `<body>` に書きます。	第1巻 UNIT 08
`<meta>`	HTML の `<head>` の中に書く、ページの情報のタグ。`<meta charset="utf-8">`（文字コード）・`<meta name="viewport" …>`（スマホでの表示の幅）などがあります。	第1巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`charset=`	`<meta charset="utf-8">` の `charset` は、ページの文字の符号（エンコーディング）です。いつも `utf-8` にしておくと、日本語が文字化けしません。	第1巻 UNIT 08
`name=`	`name` は、入力欄（`<input>`・`<textarea>`・`<select>`）の名前です。フォームを送ると、`title=牛乳` のように、この名前と入れた値の組がサーバーに届きます。`<meta name="viewport" …>` の `name` は、設定の種類の名前です。	第1巻 UNIT 08
`content=`	`<meta name="viewport" content="width=device-width, initial-scale=1">` の `content` は、その設定の値です。この `viewport` の設定は、「スマホでも、画面の幅に合わせて表示する」という意味で、いつも書いておきます。	第1巻 UNIT 08
`<title>`	ブラウザーのタブに出る、ページの題名のタグ。`<head>` の中に書きます。	第1巻 UNIT 08
`<link>`	`<link rel="stylesheet" href="style.css">` は、ページに CSS のファイルをつなぐタグです。閉じるタグ（`</link>`）は要りません。	第1巻 UNIT 08
`rel=`	`<link rel="stylesheet" …>` の `rel` は、つなぐファイルが「何か」です。`stylesheet` は CSS のファイル、`icon` はタブの小さな絵です。	第1巻 UNIT 08
`href=`	`href` は、リンク先や、読み込むファイルの場所（URL）です。`<a href="/notes">一覧</a>` は、押すと `/notes` へ移るリンクです。	第1巻 UNIT 08
`<body>`	`<body>` は、画面に出る中身（見出し・文・ボタンなど）を書く所です。	第1巻 UNIT 08
`<main>`	`<main>` は、ページの「主な中身」を囲むタグです。メニューや足元（フッター）以外の、そのページの本題を入れます。読み上げソフトが、本題に飛べるようになります。	第1巻 UNIT 08
`<h1>`	いちばん大きな見出しのタグ。	第1巻 UNIT 08
`<p>`	段落のタグ。	第1巻 UNIT 08
`<code>`	`<code>` は、中の文字が「プログラムのコード」だと示すタグです。ふつう、等幅の文字で表示されます。	第1巻 UNIT 08
`<a>`	リンクのタグ。`<a href="URL">文字</a>` で、押すとその URL へ移ります。	第6巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`<nav>`	`<nav>` は、ほかのページへのリンク（メニュー）をまとめるタグです。	第6巻 UNIT 03
`aria-*=`	`aria-label="…"`・`aria-current="page"` のように `aria-` で始まる属性は、読み上げソフトなどの手助けの道具に、見た目では分からないことを伝える印です。`aria-label` は名前、`aria-current="page"` は「今開いているページ」、`aria-invalid="true"` は「入力に間違いがある」です。	第6巻 UNIT 03
`class=`	`class` は、タグに付ける「グループの名前」です。CSS で `.note { … }` と書くと、`class="note"` の物全部に、同じ見た目が付きます。`class="note done"` のように、空白で区切っていくつも付けられます。	第6巻 UNIT 03
`<header>`	`<header>` は、ページや区切りの「頭の部分」（サイトの名前・メニューなど）を囲むタグです。	第6巻 UNIT 03
`<div>`	`<div>` は、意味を持たない、ただの「囲み」のタグです。CSS で見た目を付けるために、いくつかの物をまとめる時に使います。	第6巻 UNIT 03
`<li>`	`<li>` は、並び（`<ul>`・`<ol>`）の中の 1 つを表すタグです。	第6巻 UNIT 05
`data-*=`	`data-tag="…"` のように `data-` で始まる属性は、自分で決めた名前で、タグに値を付けておける入れ物です。CSS や JavaScript から読めます。	第6巻 UNIT 05
`<section>`	`<section>` は、ページの中の、ひとまとまりの区切り（見出しを持つ部分）を囲むタグです。	第6巻 UNIT 05
`<h2>`	`<h2>` は、2 番目の大きさの見出しです。`<h1>` がページの題、`<h2>` がその下の区切りの見出し、`<h3>` がさらに下です。見た目の大きさではなく、文の組み立てで選びます。	第6巻 UNIT 05
`<ul>`	`<ul>` は、順番の無い並び（・で始まる箇条書き）のタグです。中に `<li>` を並べます。順番のある並びは `<ol>` です。	第6巻 UNIT 05
`<input>`	1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。	第6巻 UNIT 05
`type=`	`<input type="…">` の `type` は、入力欄の種類です。`text`（文字）・`email`・`password`（伏せ字）・`number`・`date`・`checkbox`（チェック）・`file`（ファイル）・`search` などがあります。`<button type="submit">` は、押すとフォームを送るボタンです。	第6巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`checked=`	`checked` は、チェックボックスが最初からチェックされた状態にする印です。	第6巻 UNIT 05
`disabled=`	`disabled` は、入力欄やボタンを使えなく（押せなく）する印です。送信中に、2 回押されるのを防ぐ時などに使います。	第6巻 UNIT 05
`title=`	`title` は、マウスを乗せた時に出る、短い説明です。	第6巻 UNIT 05
`<form>`	入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。	第6巻 UNIT 06
`method=`	`<form method="post">` の `method` は、フォームの送り方です。`post` はデータを送って何かを変える時（登録・削除）、`get` は検索のように、URL の `?q=…` に入れて送る時に使います。	第6巻 UNIT 06
`action=`	`<form action="/notes">` の `action` は、フォームの送り先の URL です。書かなければ、今のページに送ります。	第6巻 UNIT 06
`value=`	`value` は、入力欄の最初の値や、選び肢（ `<option>` ）を選んだ時に送られる値です。	第6巻 UNIT 06
`<button>`	ボタンのタグ。`<form>` の中の `type="submit"` のボタンを押すと、フォームが送られます。	第6巻 UNIT 06
`<span>`	`<span>` は、文の中の一部を囲む、意味を持たないタグです。一部の文字だけに色を付ける時などに使います（ `<div>` とちがい、改行しません）。	第6巻 UNIT 07
`id=`	`id` は、ページの中で 1 つだけの名前です。`<label for="title">` で欄と結び付けたり、`#title` のリンクでその場所へ飛んだりできます。同じ `id` を 2 つ付けてはいけません。	第8巻 UNIT 06
`placeholder=`	`placeholder` は、入力欄が空の時に、うすく見せる例の文字です。入力の説明の代わりにはならないので、`<label>` も付けます。	第9巻 UNIT 02
`<label>`	入力欄の名前を表すタグ。`<label for="title">件名</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります。	第9巻 UNIT 02
`for=`	`<label for="q">` の `for` は、この説明の文字が、どの入力欄（ `id="q"` ）のための物かを結び付けます。文字を押しても、その欄に入力できるようになり、読み上げソフトも欄の名前を読めます。	第9巻 UNIT 02

コード	意味と使い方	初めて出てくる所
`role=`	`role="status"` は、その部分の役わりを、読み上げソフトに伝える印です。`status` は「お知らせ」で、変わった時に読み上げられます。`role="alert"` は、すぐ知らせる大事なお知らせです。	第10巻 UNIT 09
`required=`	`required` は、「この欄は必ず入れる」という印です。空のまま送ろうとすると、ブラウザーが止めて知らせます（サーバー側でも、必ず確かめます）。	第13巻 UNIT 07
`maxlength=`	`maxlength="100"` は、入力できる文字の数の上限です。それより長くは打てません（サーバー側でも、必ず確かめます）。	第13巻 UNIT 07
`<textarea>`	`<textarea>` は、何行も書ける入力欄です。`rows="6"` で、見える行の数を決めます。中の初めの文字は、タグの間に書きます。	第13巻 UNIT 07
`rows=`	`<textarea rows="6">` の `rows` は、見える行の数です。	第13巻 UNIT 07
`autocomplete=`	`autocomplete="email"` は、ブラウザーに、「この欄にはメールアドレスを入れる」と伝え、覚えている値を出してもらう印です。`username`・`current-password`・`new-password` などがあります。	第13巻 UNIT 07
`<details>`	`<details>` は、押すと開いたり閉じたりする部分を作るタグです。中の `<summary>` が、いつも見える見出しになります。`open` を付けると、最初から開きます。	第13巻 UNIT 08
`open=`	`<details open>` の `open` は、最初から開いた状態にする印です。	第13巻 UNIT 08
`<summary>`	`<summary>` は、`<details>` の中に書く、いつも見える見出しです。押すと、残りの中身が開きます。	第13巻 UNIT 08
`<article>`	`<article>` は、それだけで意味がまとまった中身（ノート 1 件・記事 1 つ）を囲むタグです。	第13巻 UNIT 08

CSS

コード	意味と使い方	初めて出てくる所
`font-family:`	CSS で、文字の書体（フォント）を決める項目。`font-family: sans-serif;` は「飾りの無い書体」。	第6巻 UNIT 08
`margin:`	CSS で、箱の外側の余白を決める項目。`margin: 2rem;` は、上下左右に文字 2 つ分くらいの余白です。	第6巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`color:`	CSS で、文字の色を決める項目。`color: #1e40af;` のように、色の番号（16 進数）や名前で書きます。	第6巻 UNIT 08
`/* */`	CSS では、`/*` から `*/` までがコメントです。	第6巻 UNIT 08
`数rem`	CSS の `1.5rem` の `rem` は、「ページの基本の文字の大きさ（ふつう 16px）の何倍か」で表す単位です。利用者が文字を大きくする設定にすると、一緒に大きくなるので、読みやすさのために、文字や余白によく使います。	第6巻 UNIT 08
`#rrggbb`	CSS の `#0f172a` のような `#` と 6 けた（または 3 けた）の 16 進数は、色の書き方です。2 けたずつ、赤・緑・青の強さ（00〜ff）を表します。`#ffffff` は白、`#000000` は黒です。	第6巻 UNIT 08
`color-text:`	この本の CSS で決めた色の変数（`--color-text`：ふつうの文字の色）。`var(--color-text)` で使います。CSS の変数は、名前の頭を `--` にして `:root { … }` の中で決めると、どこからでも使えます。	第9巻 UNIT 09
`color-muted:`	この本の CSS の色の変数（`--color-muted`：控えめな文字の色）。	第9巻 UNIT 09
`color-border:`	この本の CSS の色の変数（`--color-border`：枠線の色）。	第9巻 UNIT 09
`color-bg:`	この本の CSS の色の変数（`--color-bg`：ページの背景の色）。	第9巻 UNIT 09
`color-card:`	この本の CSS の色の変数（`--color-card`：カードの背景の色）。	第9巻 UNIT 09
`color-primary:`	この本の CSS の色の変数（`--color-primary`：ボタンなどの主な色）。	第9巻 UNIT 09
`color-primary-dark:`	この本の CSS の色の変数（`--color-primary-dark`：主な色の濃い版。マウスを乗せた時など）。	第9巻 UNIT 09
`color-success-bg:`	この本の CSS の色の変数（`--color-success-bg`：成功のお知らせの背景）。	第9巻 UNIT 09
`color-success-text:`	この本の CSS の色の変数（`--color-success-text`：成功のお知らせの文字）。	第9巻 UNIT 09
`color-warning-bg:`	この本の CSS の色の変数（`--color-warning-bg`：注意のお知らせの背景）。	第9巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`color-warning-text:`	この本の CSS の色の変数（`--color-warning-text`：注意のお知らせの文字）。	第9巻 UNIT 09
`color-danger:`	この本の CSS の色の変数（`--color-danger`：消すボタン・エラーの色）。	第9巻 UNIT 09
`:root`	CSS の `:root` は、ページ全体（いちばん外側の `<html>`）を選ぶ書き方です。ページのどこからでも使う色などの変数（`--color-text`）を、ここに書きます。	第9巻 UNIT 09
`--名前`	CSS の `--color-text: #0f172a;` のように `--` で始まる名前は、自分で決める変数（カスタムプロパティ）です。`color: var(--color-text);` で使います。色を 1 か所で決めておけば、変える時も 1 か所で済みます。	第9巻 UNIT 09
`box-sizing:`	CSS で、箱の幅に枠線と内側の余白を含めるかを決める項目。`box-sizing: border-box;` にすると、`width` の中に `padding` と `border` が含まれ、大きさの計算が分かりやすくなります。	第13巻 UNIT 09
`background:`	CSS で、背景（色や画像）を決める項目。`background: var(--color-bg);` は、色の変数の色にします。	第13巻 UNIT 09
`line-height:`	CSS で、行の高さ（行と行の間隔）を決める項目。`1.7` は「文字の大きさの 1.7 倍」。	第13巻 UNIT 09
`display:`	CSS で、箱の並べ方を決める項目。`display: flex;` は「中身を横一列に並べる」、`display: grid;` は「ます目に並べる」、`display: none;` は「表示しない」。	第13巻 UNIT 09
`align-items:`	`display: flex` の中身を、上下（交わる向き）のどこにそろえるかを決める項目。`center` で真ん中にそろえます。	第13巻 UNIT 09
`justify-content:`	`display: flex` の中身を、並ぶ向きにどう配るかを決める項目。`space-between` は「両端に寄せて、間を均等に空ける」。	第13巻 UNIT 09
`gap:`	`flex` や `grid` の中身どうしの間の幅を決める項目。	第13巻 UNIT 09
`padding:`	CSS で、箱の内側の余白を決める項目。`padding: 12px 24px;` は、上下 12px・左右 24px。	第13巻 UNIT 09
`border-bottom:`	CSS で、箱の下の枠線を決める項目。`1px solid 色` は「太さ 1px の実線」。	第13巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`font-weight:`	CSS で、文字の太さを決める項目。`700` や `bold` で太字、`800` はもっと太い字です。	第13巻 UNIT 09
`font-size:`	CSS で、文字の大きさを決める項目。	第13巻 UNIT 09
`text-decoration:`	CSS で、文字の下線などを決める項目。`none` で、リンクの下線を消せます。	第13巻 UNIT 09
`max-width:`	CSS で、箱の幅の上限を決める項目。広い画面でも、読みやすい幅で止められます。	第13巻 UNIT 09
`border:`	CSS で、箱の枠線（太さ・線の種類・色）をまとめて決める項目。`border: 1px solid #e2e8f0;` 。	第13巻 UNIT 09
`border-radius:`	CSS で、箱の角の丸みを決める項目。	第13巻 UNIT 09
`cursor:`	CSS で、マウスを乗せた時の矢印の形を決める項目。`pointer` は、押せる物の「指の形」です。	第13巻 UNIT 09
`button:`	`button:hover` は、「マウスを乗せている時のボタン」を指す CSS の書き方（ `:hover` は乗せている時の状態）。	第13巻 UNIT 09
`border-color:`	CSS で、枠線の色だけを決める項目。	第13巻 UNIT 09
`link-button:`	`.link-button:hover` は、この本の CSS で決めた「リンクのように見えるボタン」（ `link-button` という class）の、マウスを乗せた時の見た目です。	第13巻 UNIT 09
`margin-top:`	CSS で、箱の上の外側の余白を決める項目。	第13巻 UNIT 09
`width:`	CSS で、箱の幅を決める項目。	第13巻 UNIT 09
`font:`	CSS で、文字の大きさ・書体などをまとめて決める項目。`font: inherit;` は「親と同じにする」（ボタンや入力欄の文字を、ページの文字とそろえる）。	第13巻 UNIT 09
`margin-bottom:`	CSS で、箱の下の外側の余白を決める項目。	第13巻 UNIT 09
`list-style:`	CSS で、並び（ `ul` ・ `ol` ）の点や番号の形を決める項目。`none` で消せます。	第13巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`flex-wrap:`	`display: flex` の中身が入りきらない時に、次の行へ折り返すかを決める項目。`wrap` で折り返します。	第13巻 UNIT 09
`margin-left:`	CSS で、箱の左の外側の余白を決める項目。`auto` にすると、その箱を右へ寄せられます。	第13巻 UNIT 09
`white-space:`	CSS で、空白や改行をどう表示するかを決める項目。`pre-wrap` は「改行と空白はそのまま、はみ出す所は折り返す」。	第13巻 UNIT 09
`overflow-wrap:`	CSS で、長い単語（URL など）が箱からはみ出す時に、途中で折り返すかを決める項目。`anywhere` で、どこでも折り返せます。	第13巻 UNIT 09
`text-align:`	CSS で、文字の左右のそろえ方を決める項目。`center` で中央。	第13巻 UNIT 09
`*`	CSS の `*` は、「全部のタグ」を選ぶ書き方です。`* { box-sizing: border-box; }` は、全部のタグの幅の数え方をそろえます。	第13巻 UNIT 09
`var()`	CSS の `var(--color-text)` は、`--` で始まる変数の値を使う書き方です。	第13巻 UNIT 09
`.クラス名`	CSS の `.note { … }` のように `.` で始まる名前は、「`class="note"` の付いたタグ全部」を選ぶ書き方（クラスセレクター）です。`{ }` の中の見た目が、その全部に付きます。	第13巻 UNIT 09
`数px`	CSS の `16px` の `px` は、画面の点（ピクセル）の数で表す大きさの単位です。	第13巻 UNIT 09
`A B`	CSS の `.note-list li` のように、空白で区切って並べると、「`.note-list` の**中にある** `li` 」を選びます（子孫セレクター）。	第13巻 UNIT 09
`A, B`	CSS の `button, .btn { … }` のように `,` で区切ると、「`button` と `.btn` の両方」に、同じ見た目を付けます。	第13巻 UNIT 09
`:hover`	CSS の `button:hover` の `:hover` は、「マウスが乗っている間だけ」の見た目です。`:focus` は「キーボードで選ばれている間」、`:disabled` は「使えない時」です。	第13巻 UNIT 09
`数%`	CSS の `100%` は、「入れ物（親）の大きさの何パーセントか」で表す単位です。	第13巻 UNIT 09

JavaScript・TypeScript

コード	意味と使い方	初めて出てくる所
`import`	ほかのファイルや部品から、関数などを読み込む書き方。`import { rmSync } from "node:fs";` は、Node.js のファイルの道具から `rmSync` を使えるようにします。	第8巻 UNIT 08
`from`	`import { 名前 } from "場所";` の一部で、「どこから読み込むか」を書きます。	第8巻 UNIT 08
`const`	JavaScript・TypeScript で、変えない値に名前を付ける書き方。`const db = …;` 。あとで別の値を入れようとするとエラーになります。	第8巻 UNIT 08
`new`	クラスから、新しい物を作る書き方。`new DatabaseSync("ba.sqlite")`・`new Date()`・`new Error("…")` のように使います。	第8巻 UNIT 08
`await`	JavaScript・TypeScript で、時間のかかる処理（`Promise`）が終わるまで待って、結果を受け取る書き方。`async` の関数の中で使います。	第8巻 UNIT 08
`let`	JavaScript・TypeScript で、あとで変える値に名前を付ける書き方。	第8巻 UNIT 08
`async`	JavaScript・TypeScript で、中で `await` を使える関数にする印。結果は `Promise`（あとで届く約束）で返ります。	第8巻 UNIT 08
`function`	関数を作る書き方。`async function post(path, body) { … }` 。	第8巻 UNIT 08
`try`	失敗するかもしれない処理を `try { … } catch (e) { … }` で囲み、失敗した時の処理を `catch` に書く書き方。	第8巻 UNIT 08
`catch`	`try` の中で失敗（エラー）が起きた時に動く所。`catch (e)` の `e` に、エラーが入ります。	第8巻 UNIT 08
`return`	JavaScript・TypeScript で、関数の結果を返して、関数を終える命令。	第8巻 UNIT 08
`for`	くり返しの書き方。`for (const x of 配列) { … }` は、配列の中身を 1 つずつ `x` に入れてくり返します。	第8巻 UNIT 08
`of`	`for (const x of 配列)` の一部で、「配列の中身を 1 つずつ」という意味です。	第8巻 UNIT 08
`.runMigrations()`	Better Auth の、データベースの表を作る・変える手順（マイグレーション）を実行するメソッド（第8巻 UNIT 08 の比べ）。	第8巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.now()`	`performance.now()` は、ページ（プログラム）が始まってからの時間をミリ秒で返します。`Date.now()` は、1970 年 1 月 1 日からのミリ秒です。どちらも、かかった時間を計るのに使えます。	第8巻 UNIT 08
`.handler()`	Better Auth の、HTTP のリクエストを受け取って答えを返す関数（ `auth.handler(new Request(…))` ）。サーバーを立てずに試す時に、直接呼んでいます。	第8巻 UNIT 08
`.stringify()`	`JSON.stringify(値)` は、JavaScript の値を JSON の文字列にする関数です。	第8巻 UNIT 08
`.round()`	`Math.round(x)` は、小数を四捨五入して整数にする関数です。	第8巻 UNIT 08
`.parse()`	`JSON.parse(文字列)` は、JSON の文字列を、JavaScript のオブジェクトや配列に変えます。逆は `JSON.stringify(値)` です。	第8巻 UNIT 08
`.slice()`	文字列や配列の、一部分を切り出すメソッド。`text.slice(0, 40)` は、最初の 40 文字です。	第8巻 UNIT 08
`.get()`	名前（キー）で 1 つ取り出すメソッド。`response.headers.get("set-cookie")`（見出しを 1 つ）・`map.get(キー)`（`Map` から）・`stmt.get()`（SQLite の結果の 1 行）のように使います。無ければ `undefined` や `null` です。	第8巻 UNIT 08
`.split()`	JavaScript・TypeScript で、文字列を区切りで分けて配列にするメソッド。`"a,b".split(",")` は `["a", "b"]`。正規表現（ `/,/` ）でも区切れます。	第8巻 UNIT 08
`.join()`	JavaScript・TypeScript で、配列の中身を区切り文字でつないで 1 つの文字列にするメソッド。`["a", "b"].join(";")` は `"a;b"`。	第8巻 UNIT 08
`.log()`	`console.log(…)` は、渡した値を画面（ターミナルやブラウザーの開発者ツール）に表示します。	第8巻 UNIT 08
`.push()`	JavaScript・TypeScript で、配列の後ろに足すメソッド（元の配列が変わる）。	第8巻 UNIT 08
`.add()`	`Set`（同じ物を 2 回入れない入れ物）に 1 つ足すメソッド。すでにあれば何もしません。	第8巻 UNIT 08
`.prepare()`	Node.js の SQLite（ `node:sqlite` ）で、SQL の文を用意するメソッド。`.get()`・`.all()` で結果を受け取ります。	第8巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`.startsWith()`	文字列が、その文字で始まっていれば `true` を返すメソッド（Rust の `starts_with`）。	第8巻 UNIT 08
`??`	`a ?? b` は、`a` が `null` か `undefined`（値が無い）なら `b` を使う、という書き方です。	第8巻 UNIT 08
`=>`	アロー関数（短く書ける関数）の矢印。`(path) => `…${path}`` は「`path` を受け取って文字列を返す関数」。	第8巻 UNIT 08
`...`	スプレッド構文。`[...a, x]` は配列 `a` の中身を広げて新しい配列を作り、`{ ...obj, k: v }` はオブジェクトの中身を写して一部だけ変えた新しいオブジェクトを作ります。	第8巻 UNIT 08
`//`	JavaScript・TypeScript では、`//` から行の終わりまでがコメントです。	第8巻 UNIT 08
`${…}`	`` `…` ``（バッククォートで囲んだ文字。テンプレート文字列）の中の `${ }` には、値や式を入れられます。`` `エラー ${res.status}` `` は、`res.status` が 404 なら `エラー 404` になります。改行も、そのまま書けます。	第8巻 UNIT 08
`/…/`	JavaScript の `/…/` は、正規表現（文字の形を表す書き方）をそのまま書く形です。`/\d+/` は「数字が 1 つ以上」。`.match(/…/)`・`.test(…)`・`.split(/…/)` などで使います。記号ごとの意味は、この一覧の「正規表現」の表にあります。	第8巻 UNIT 08
`条件 ? A : B`	`条件 ? A : B` は、「条件が合えば A、合わなければ B」になる式です（三項演算子）。`user.isAdmin ? "管理者" : "一般"`。`if` とちがい、値を返すので、そのまま変数や表示に使えます。	第8巻 UNIT 08
`const [ … ] =`	`const [a, b] = 並び;` は、並びの 0 番目を `a` に、1 番目を `b` に取り出す書き方です（分割代入）。React の `const [count, setCount] = useState(0);` もこれで、「今の値」と「値を変える関数」の 2 つを受け取っています。	第8巻 UNIT 08
`++`	`n++` は、`n` を 1 増やします（`n += 1` と同じ）。`nextIp++` のように式の中で使うと、「今の値を使ってから、1 増やす」になります。	第8巻 UNIT 08
`Request`	`Request` は、HTTP の要求（どこへ・何を）を表す、Web の決まりの型です。`new Request(URL, { method: "POST", … })` で作ります。	第8巻 UNIT 08
`Math`	`Math` は、計算の関数が集まった物です。`Math.round(3.6)` は四捨五入で `4`、`Math.max(1, 5)` は大きい方の `5` です。	第8巻 UNIT 08

コード	意味と使い方	初めて出てくる所
`Set`	`new Set(並び)` は、同じ値を 1 つにまとめた集まりを作ります。`new Set(chosen).size !== chosen.length` は、「重なりを消すと数が減る」＝「同じ人が 2 回選ばれている」の確かめです。	第8巻 UNIT 08
`Date`	`new Date(文字)` は、日時の値を作ります。`.toLocaleString("ja-JP")` で、日本の形の日時の文字になります。	第8巻 UNIT 08
`betterAuth()`	`betterAuth({ … })` は、Better Auth（JavaScript のログインの部品）の設定を受け取って、ログインの仕組みを作る関数です。	第8巻 UNIT 08
`getMigrations()`	Better Auth の `getMigrations(設定)` は、ログインに要る表を、データベースに作るための準備をする関数です。	第8巻 UNIT 08
`decodeURIComponent()`	`decodeURIComponent("%E5%B1%B1")` は、URL やクッキーの中で `%XX` の形にした文字を、元の文字（山）に戻します。反対は `encodeURIComponent()` です。	第8巻 UNIT 08
`.headers`	`fetch` の答えの `.headers` は、返ってきた見出し（ヘッダー）です。`.headers.get("set-cookie")` で 1 つ読めます。	第8巻 UNIT 08
`.value`	`.value` は、入力欄に今入っている文字や、クッキーなどの値です。	第8巻 UNIT 08
`({ 名前 })`	関数の引数の `({ status, label })` は、渡されたオブジェクトから、`status` と `label` を取り出して受け取る書き方です。React の部品（コンポーネント）が props を受け取る時によく使います。	第8巻 UNIT 08
`.createElement()`	`document.createElement('iframe')` は、新しい HTML のタグ（要素）を作る、ブラウザーの関数です。	第10巻 UNIT 10
`.appendChild()`	作ったタグを、ページの中の決めた所（親）の最後に入れるメソッド。	第10巻 UNIT 10
`.querySelectorAll()`	CSS と同じ書き方（`.flash`・`button` など）で、ページの中の合うタグを全部探すメソッド。	第10巻 UNIT 10
`.filter()`	配列から、条件に合う物だけを集めた新しい配列を作るメソッド。	第10巻 UNIT 10
`.map()`	配列の中身を 1 つずつ変換した、新しい配列を作るメソッド。	第10巻 UNIT 10
`.toLowerCase()`	英字を小文字にした文字列を返すメソッド（Rust の `to_lowercase`）。	第10巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`.pop()`	JavaScript・TypeScript で、配列の最後の 1 つを取り出すメソッド（元の配列から消える）。	第10巻 UNIT 10
`.remove()`	画面の要素（タグ）を、ページから取り除くメソッド。`f.remove()`。	第10巻 UNIT 10
`.padEnd()`	文字列の後ろを空白で埋めて、決めた長さにするメソッド。表のように並べて表示する時に使います。	第10巻 UNIT 10
`&&`	`a && b` は、「a と b の両方が当てはまる」（かつ）です。左が当てはまらなければ、右は調べません。	第10巻 UNIT 10
`async () =>`	`async () => { … }` は、中で `await`（終わるまで待つ）を使える、名前の無い関数（アロー関数）です。	第10巻 UNIT 10
`Promise`	`Promise` は、「あとで終わる処理」（サーバーの答えを待つ、時間を待つ）を表す物です。`await` を付けると、終わるまで待って、結果を受け取れます。`new Promise((resolve) => …)` で、自分で作ることもできます。	第10巻 UNIT 10
`.documentElement`	`document.documentElement` は、ページのいちばん外側の `<html>` のタグです。	第10巻 UNIT 10
`.addEventListener()`	クリック・入力などの出来事（イベント）が起きた時に動く関数を登録するメソッド。`document.addEventListener('focusin', (e) => { … })`。	第10巻 UNIT 10
`.trim()`	JavaScript・TypeScript で、文字列の前後の空白を取り除いたものを返すメソッド。	第10巻 UNIT 10
`?.`	`a?.b` は、`a` が `null` や `undefined` なら、そこで止めて `undefined` にする書き方（オプショナルチェーン）。無いかもしれない物を、エラーにせずにたどれます。	第10巻 UNIT 10
`getComputedStyle()`	`getComputedStyle(el)` は、タグに最終的に付いている見た目（CSS の値）を読みます。	第10巻 UNIT 10
`.target`	イベント（`e`）の `.target` は、イベントが起きたタグ（入力欄など）です。`e.target.value` で、入力欄に今入っている文字を読めます。	第10巻 UNIT 10
`.textContent`	`.textContent` は、タグの中の文字です。入れた文字は、HTML としてではなく、ただの文字として表示されるので、安全です（`innerHTML` に入れると、タグとして動いてしまいます）。	第10巻 UNIT 10

コード	意味と使い方	初めて出てくる所
`.querySelector()`	CSS と同じ書き方で、ページの中の合うタグを 1 つ（最初の物）だけ探すメソッド。見つからなければ `null` です。	第10巻 UNIT 10
`.href`	`location.href` は、今のページの URL の全部です。	第10巻 UNIT 10
`if`	JavaScript・TypeScript の条件。`if (条件) { … } else { … }`。条件は `( )` で囲みます。	第10巻 UNIT 10
`null`	JavaScript で「値が無い」ことを表す値。	第10巻 UNIT 10
`.getBoundingClientRect()`	タグが、画面のどこに、どの大きさで表示されているか（位置・幅・高さ）を返すメソッド。はみ出しや重なりを調べる時に使います。	第10巻 UNIT 10
`export`	「このファイルの外から使ってよい」という印。`export default 値` は、そのファイルのいちばんの物を 1 つ外に出します。	第11巻 UNIT 05
`default`	`export default` は、そのファイルから「いちばんの物」を 1 つ外に出す書き方。読み込む側は `import 名前 from "./ファイル"` と、好きな名前で受け取れます。	第11巻 UNIT 05
`defineConfig()`	`defineConfig({ … })` は、設定ファイル（Vite の `vite.config` や Playwright の `playwright.config`）で、設定を書くための関数です。書き間違いを、エディターが教えてくれるようになります。	第11巻 UNIT 05
`.repeat()`	JavaScript・TypeScript で、文字列をくり返してつないだ新しい文字列を作るメソッド。`"あ".repeat(100)`。	第11巻 UNIT 05
`.replace()`	JavaScript・TypeScript で、文字列の一部を置きかえた新しい文字列を返すメソッド。正規表現で探す所を決め、`"$1"` で、かっこで囲んだ部分を使えます。	第11巻 UNIT 05
`import()`	`import("…")` は、プログラムの途中で、部品（モジュール）を読み込む書き方です。TypeScript の型の所に書く `import("@playwright/test").Page` は、「その部品の `Page` という型」を指します。	第11巻 UNIT 05
`.pathname`	URL の `.pathname` は、`/notes/3` のような、場所の部分です（`?` から後ろを除きます）。	第11巻 UNIT 05
`1_000`	数の中の `_` は、読みやすくするための区切りです。`86_400_000` は `86400000` と同じです。	第11巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`.length`	`.length` は、並びの数や、文字の数です。文字の場合は、UTF-16 という数え方なので、絵文字などは 2 と数えることがあります（`[...文字].length` なら、見た目の文字の数に近くなります）。	第11巻 UNIT 05
`as`	TypeScript で「この値は、この型として扱う」と伝える書き方（`値 as 型`）。	第11巻 UNIT 05
`.getAttribute()`	タグの属性（`placeholder`・`name` など）の値を取り出すメソッド。無ければ `null` です。	第11巻 UNIT 05
`===`	JavaScript で「型も値も同じか」を比べる書き方。`==` は型を勝手にそろえて比べるので、ふつうは `===` を使います（違うかは `!==`）。	第11巻 UNIT 05
`||`	`a || b` は、「a か b のどちらか」（または）です。`el.innerText || el.getAttribute("name") || ""` のように、「左が空なら右を使う」という使い方もします（`0` や `""` も「空」とみなされるので、`null`・`undefined` だけを見たい時は `??` を使います）。	第11巻 UNIT 05
`!`	`!res.ok` のように前に付けた `!` は、「〜ではない」です。`if (!res.ok)` は「うまくいかなかったら」。	第11巻 UNIT 05
`() =>`	`() => { … }` は、名前の無い関数（アロー関数）の書き方です。`( )` の中が引数、`=>` の右が中身です。`(e) => setKeyword(e.target.value)` のように、1 つの式なら `{ }` を省けます。ほかの関数に「あとで呼んでもらう処理」を渡す時によく使います。	第11巻 UNIT 05
`.innerText`	`.innerText` は、タグの中の、画面に見えている文字です。	第11巻 UNIT 05
`.headers`	`fetch` の答えの `.headers` は、返ってきた見出し（ヘッダー）です。`.headers.get("set-cookie")` で 1 つ読めます。	第12巻 UNIT 09
`else`	JavaScript・TypeScript で、`if` の条件に合わなかった時の所。`else if (条件)` で、条件をつなげられます。	第13巻 UNIT 09
`.getPropertyValue()`	`getComputedStyle(要素).getPropertyValue("white-space")` のように、CSS の項目の、実際に効いている値を取り出すメソッド。	第13巻 UNIT 09
`.close()`	Playwright で、ブラウザーを閉じるメソッド（`await browser.close()`）。	第13巻 UNIT 09

TypeScript

コード	意味と使い方	初めて出てくる所
`string`	`string` は、文字の型です。	第11巻 UNIT 05
`HTMLElement`	`HTMLElement` は、ページの中のタグ（要素）1 つの型です。`as HTMLElement` で、「これはタグです」と TypeScript に伝えます。	第11巻 UNIT 05

Playwright

コード	意味と使い方	初めて出てくる所
`.goto()`	Playwright で、ブラウザーを指定した URL へ移動させるメソッド。`await page.goto("/signup")`。	第11巻 UNIT 05
`.getByLabel()`	ラベル（`<label>`）の文字で、入力欄を探すメソッド。利用者が見るのと同じ文字で探すので、テストが読みやすく、壊れにくくなります。	第11巻 UNIT 05
`.fill()`	入力欄に文字を入れるメソッド。`await page.getByLabel("メールアドレス").fill("alice@example.com")`。	第11巻 UNIT 05
`.getByRole()`	役割（ボタン・見出し・リンクなど）と名前で、画面の部品を探すメソッド。`page.getByRole("button", { name: "登録する" })`。	第11巻 UNIT 05
`.click()`	Playwright で、見つけたボタンやリンクをクリックするメソッド。`await page.getByRole("button", { name: "登録する" }).click()`。	第11巻 UNIT 05
`.toHaveURL()`	「今のページの URL が、これになっているか」を確かめるメソッド（`expect(page).toHaveURL("/notes")`）。少し待ってくれます。	第11巻 UNIT 05
`.getByText()`	表示されている文字で、部品を探すメソッド。	第11巻 UNIT 05
`.toBeVisible()`	「その部品が画面に見えているか」を確かめるメソッド。	第11巻 UNIT 05
`.toHaveText()`	「その部品の文字が、これと同じか」を確かめるメソッド。	第11巻 UNIT 05
`test()`	Playwright の `test("名前", async ({ page }) => { … })` は、テストを 1 つ書く関数です。`page` は、本物のブラウザーのタブです。	第11巻 UNIT 05

コード	意味と使い方	初めて出てくる所
`expect()`	Playwright の `expect(値)` は、「こうなっているはず」を確かめる関数です。`await expect(page).toHaveURL(/…/)` は、URL がその形になるまで待って確かめます。	第11巻 UNIT 05
`.beforeEach()`	`test.beforeEach(async ({ page }) => { … })` は、それぞれのテストの前に、毎回動かす準備（ログインなど）を書く所です。	第11巻 UNIT 05
`.pressSequentially()`	キーボードで 1 文字ずつ打つように、入力欄に文字を入れるメソッド。`maxlength` などで止まるかを確かめる時に使います。	第11巻 UNIT 05
`.inputValue()`	入力欄に今入っている文字を取り出すメソッド。	第11巻 UNIT 05
`.analyze()`	`@axe-core/playwright` の部品で、ページのアクセシビリティ（目の見えにくい人なども使えるか）の問題を調べるメソッド。	第11巻 UNIT 05
`AxeBuilder`	`AxeBuilder` は、axe（ページの使いやすさの問題を、自動で見つける道具）を、Playwright のページで動かすための部品です。`new AxeBuilder({ page }).analyze()` で調べます。	第11巻 UNIT 05
`.setTimeout()`	`test.setTimeout(5_000)` は、そのテストの制限時間（ミリ秒）を決める関数です。	第11巻 UNIT 05
`.status()`	Playwright の答え（レスポンス）から、HTTP の状態コード（200・303 など）を返すメソッド。	第11巻 UNIT 05
`.textContent()`	部品の中の文字を取り出すメソッド。	第11巻 UNIT 05
`.toBe()`	`expect(値).toBe(期待する値)` で、「同じか」を確かめるメソッド。	第11巻 UNIT 05
`.press()`	キーを 1 つ押すメソッド。`await page.keyboard.press("Tab")` は Tab キー（次の部品へ移る）。	第11巻 UNIT 05
`.evaluate()`	ブラウザーの中で、JavaScript を動かして結果を受け取るメソッド。	第11巻 UNIT 05
`.launch()`	Playwright で、ブラウザーを起動するメソッド。`chromium.launch({ channel: 'chrome' })` は、入っている Chrome を使います。	第13巻 UNIT 09
`.newContext()`	起動したブラウザーの中に、まっさらな「利用者ひとり分」の環境（クッキーなどが別々）を作るメソッド。画面の大きさも決められます。	第13巻 UNIT 09
`.newPage()`	新しいタブ（ページ）を開くメソッド。	第13巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.setViewportSize()`	画面の大きさ（幅・高さ）を変えるメソッド。スマホの幅で崩れないかを確かめる時に使います。	第13巻 UNIT 09

Python

コード	意味と使い方	初めて出てくる所
`import`	ほかのモジュール（部品）を読み込む書き方。`import re` で正規表現の道具、`from pathlib import Path` で `Path` だけを使えるようにします。	第9巻 UNIT 09
`def`	Python で関数を作る書き方。`def luminance(hex_color):` の次の行から、字下げした所が関数の中身です。	第9巻 UNIT 09
`for`	Python のくり返し。`for x in 並び:` の次の行から、字下げした所が中身です。`[式 for x in 並び]` の形（内包表記）で、新しいリストを 1 行で作ることもできます。	第9巻 UNIT 09
`in`	`for x in 並び`（並びから 1 つずつ）と、`値 in 並び`（含まれているか）の 2 つの使い方があります。	第9巻 UNIT 09
`if`	Python の条件。`if 条件:` の次の行から、字下げした所が中身です。`a if 条件 else b` の形で、1 行の中で値を選ぶこともできます。	第9巻 UNIT 09
`else`	`if` の条件に合わなかった時の所。`if … else …` の 1 行の形でも使います。	第9巻 UNIT 09
`return`	Python の関数の結果を返して、関数を終える命令。	第9巻 UNIT 09
`.read()`	ファイルの中身を全部読んで、文字列で返すメソッド。`open(パス, encoding="utf-8").read()`。	第9巻 UNIT 09
`.findall()`	`re.findall(パターン, 文字列)` は、正規表現（文字の形の決まり）に合う所を、全部リストにして返す関数です。	第9巻 UNIT 09
`open()`	ファイルを開く関数。`open(パス, encoding="utf-8")` で読む、`open(パス, "w", encoding="utf-8")` で書く形で開きます。`with open(…) as f:` と書くと、使い終わったら自動で閉じます。	第9巻 UNIT 09
`dict()`	Python の辞書（キー → 値の表）を作る関数。`dict([("a", 1), ("b", 2)])` のように、組の並びから作れます。	第9巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`int()`	Python で、文字列などを整数にする関数。`int("ff", 16)` は、16 進数の `ff` を 255 にします。	第9巻 UNIT 09
`max()`	いちばん大きい物を返す関数。`max(3, 9, 4)` は 9、`max([5, 1, 8])` は 8。いちばん小さい物は `min()` です。	第9巻 UNIT 09
`min()`	Python で、いちばん小さい値を返す関数。`min(1, 5, 3)` は 1。	第9巻 UNIT 09
`print()`	渡した値を画面に表示する関数。`print("a", 1)` のように、いくつも渡すと空白でつないで表示し、最後に改行します。	第9巻 UNIT 09
`#`	Python では、`#` から行の終わりまでがコメントです。	第9巻 UNIT 09
`"""…"""`	`"""…"""`（`"` を 3 つ）で囲んだ文字は、何行にもまたがって書けます。関数・クラス・ファイルのいちばん最初に書いた物は、その説明書（docstring）になり、`help()` やエディターで読めます。	第9巻 UNIT 09
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。正規表現の `r"\d{6}"` のように、`\` の多い文字をそのまま書けます。	第9巻 UNIT 09
`f"…{値}…"`	`f"…"`（頭に `f` を付けた文字）の中の `{ }` には、変数や式を書けて、その値が入ります（f 文字列）。`f"環境変数 {name} がありません"`。`{ }` の中の `:` の後ろには、`{amount:,}` のように書式も書けます。	第9巻 UNIT 09
`{値:<幅}`	f 文字列の `{where:<16}` の `<` と数は、「16 文字分の場所に、左寄せで入れる」指定です。足りない分は右に空白が入り、後ろの列がそろいます。`>` なら右寄せ、`^` なら中央です。	第9巻 UNIT 09
`{値:幅}`	f 文字列の `{r:5.2f}` の `5` のように、`:` の後ろの数は幅（何文字分の場所を取るか）です。数は右寄せ、文字は左寄せになります。	第9巻 UNIT 09
`{値:.桁f}`	f 文字列の `{ratio:.2f}` の `.2f` は、「小数点の下を 2 けたにした小数」で表示する指定です（`3.14159` → `3.14`）。	第9巻 UNIT 09
`[式 for 名前 in 並び]`	`[f"http://{host}" for host in ALLOWED_HOSTS]` は、「並びの 1 つずつから、新しい値を作って、並び（リスト）にする」書き方です（リスト内包表記）。後ろに `if 条件` を付けると、条件に合う物だけになります。	第9巻 UNIT 09
`名前: 型 =`	`app_name: str = "依頼・承認アプリ"` は、変数に型の説明を付けて、値を入れる書き方です。	第9巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`**`	Python の `2 ** 10` は、2 の 10 乗（`1024`）です。	第9巻 UNIT 09
`A if 条件 else B`	`a if 条件 else b` は、「条件が合えば a、合わなければ b」になる式です。`ext = name.rsplit(".", 1)[-1] if "." in name else ""`。	第9巻 UNIT 09
`名前=値`	`open(path, encoding="utf-8")` の `encoding="utf-8"` のように、`名前=値` で渡す引数は「キーワード引数」です。順番に関係なく、名前で渡せます。	第9巻 UNIT 09
`from`	`from パッケージ import 名前` の一部で、「どこから読み込むか」を書きます。`from pathlib import Path`。	第12巻 UNIT 04
`None`	Python の「値が無い」ことを表す値。調べる時は `x is None` と書きます。	第12巻 UNIT 04
`global`	関数の中で、外（ファイル全体）の変数を書きかえる時に使う宣言。	第12巻 UNIT 04
`not`	Python で「〜ではない」。`if not ok:` は「`ok` が成り立たなければ」。	第12巻 UNIT 04
`is`	「同じ物か」を調べる書き方。`x is None`・`x is not None` のように、`None` と比べる時に使います。	第12巻 UNIT 04
`and`	Python で「かつ」。両方が成り立つ時だけ `True` になります。	第12巻 UNIT 04
`True`	Python の「正しい（はい）」の値。反対は `False`。	第12巻 UNIT 04
`or`	「または」。`a or b` は、`a` が空（`None`・`""`・0 など）なら `b` を使います。	第12巻 UNIT 04
`try`	Python で、失敗するかもしれない処理を `try:` の中に書き、失敗した時の処理を `except:` に書く書き方。	第12巻 UNIT 04
`with`	使い終わったら自動で片づける（ファイルや接続を閉じる）書き方。`with open(…) as f:` の中で使った物は、抜ける時に閉じられます。	第12巻 UNIT 04
`as`	Python で、別の名前を付ける書き方。`except Exception as error:`（起きたエラーを `error` で受け取る）・`import numpy as np`・`with open(…) as f:` のように使います。	第12巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`except`	`try:` の中でエラーが起きた時に動く所。`except Exception as error:` で、起きたエラーを `error` という名前で受け取ります。	第12巻 UNIT 04
`.loads()`	文字列から読み取るメソッド。`tomllib.loads(文字列)` は TOML、`json.loads(文字列)` は JSON を、Python の辞書にします。	第12巻 UNIT 04
`.read_text()`	`Path` のファイルの中身を、全部文字列として読むメソッド。`encoding="utf-8"` を付けて読みます。	第12巻 UNIT 04
`.get()`	辞書（`dict`）から、キーで値を取り出すメソッド。`d.get("port", 8080)` は、キーが無ければ 2 つ目の値（無ければ `None`）を返すので、`d["port"]` と違ってエラーになりません。	第12巻 UNIT 04
`.search()`	`re.search(パターン, 文字列)` は、文字列の中から正規表現に合う所を探し、見つかれば「合った物」（`Match`）を、無ければ `None` を返します。`m.group(1)` で、かっこで囲んだ部分を取り出せます。	第12巻 UNIT 04
`.group()`	正規表現で見つかった所から、かっこ `( )` の中の部分を取り出すメソッド。`m.group(1)` は 1 つ目のかっこの中です。	第12巻 UNIT 04
`.urlopen()`	`urllib.request.urlopen(URL)` は、URL にアクセスして、返事を受け取る関数です。	第12巻 UNIT 04
`.exit()`	`sys.exit(番号)` は、Python のプログラムを、その終了コードで終える関数です。	第12巻 UNIT 04
`len()`	長さ（中身の数）を返す関数。`len("abc")` は 3、`len([1, 2])` は 2、`len(sys.argv)` は「渡された引数の数＋1」です。	第12巻 UNIT 04
`str()`	値を文字列に変える関数。`str(8080)` は `"8080"`。数と文字列は `==` で比べても等しくならないので、そろえる時に使います。	第12巻 UNIT 04
`getattr()`	物から、名前で属性（中の値）を取り出す関数。`getattr(error, "code", 既定値)` は、`code` が無ければ既定値を返します。	第12巻 UNIT 04
`type()`	値の型を返す関数。`type(error).__name__` は、エラーの種類の名前です。	第12巻 UNIT 04
`bool()`	値を `True`・`False` にする関数。空の文字列や `None` は `False`、中身があれば `True` です。	第12巻 UNIT 04
`{値!r}`	f 文字列の `{no!r}` の `!r` は、値を `repr()` の形（プログラマー向けの形）で表示する指定です。文字なら `'012345'` のように `'` が付くので、空白や空の文字が見分けやすくなります。	第12巻 UNIT 04

コード	意味と使い方	初めて出てくる所
`+=`	`total += n` は `total = total + n` の短い形です。	第12巻 UNIT 04
`==`	`a == b` は、「a と b が等しいか」です。`=`（1 つ）は「値を入れる」です。	第12巻 UNIT 04
`is not`	`x is not None` は、「x が None ではない」です。`None` かどうかは、`==` ではなく `is` で調べます。	第12巻 UNIT 04
`__name__`	`__name__` は、今のファイル（モジュール）の名前が入った、特別な変数です。ファイルを直接動かした時は `"__main__"` になります。`type(error).__name__` は、エラーの種類の名前です。	第12巻 UNIT 04
`Path`	`Path`（`pathlib`）は、ファイルやフォルダの場所を表す型です。`Path("memo.txt").exists()`・`.read_text()`・`/` でのつなぎ（`BASE_DIR / "media"`）ができます。	第12巻 UNIT 04
`Exception`	`Exception` は、Python のふつうのエラー全部の、元になる型です。`class TransitionError(Exception):` で、自分のエラーの種類を作れます。`except Exception as e:` は、ほとんど全部のエラーを受け止めます。	第12巻 UNIT 04
`elif`	Python の「そうでなくて、もし〜なら」（`else if` の短い形）。	第12巻 UNIT 09
`class`	クラス（データと、それを扱う関数をまとめた設計図）を作る書き方。`class Handler(BaseHTTPRequestHandler):` は、元のクラスを受け継いで作ります。	第12巻 UNIT 09
`pass`	「何もしない」という Python の命令。中身が要らないけれど、形の上で何か書く必要がある所に置きます。	第12巻 UNIT 09
`while`	Python で、条件が `True` の間くり返す書き方。`while True:` は、`break` で抜けるまでずっとくり返します。	第12巻 UNIT 09
`break`	Python のくり返しを、途中で抜ける命令。	第12巻 UNIT 09
`.Lock()`	`threading.Lock()` は、いくつものスレッドが同時に同じ物を使わないようにする鍵を作る関数です。	第12巻 UNIT 09
`.join()`	Python で、区切り文字の方に付けて、リストの中身をつなぐメソッド。`" ".join(["a", "b"])` は `"a b"`（TypeScript とは書く順番が逆なので注意）。	第12巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.split()`	Python で、文字列を分けてリストにするメソッド。`"a b c".split()` のように何も渡さないと、空白（いくつ続いても）で分けます。	第12巻 UNIT 09
`.append()`	Python で、リストの後ろに 1 つ足すメソッド。	第12巻 UNIT 09
`.lower()`	英字を小文字にした文字列を返すメソッド。	第12巻 UNIT 09
`.readline()`	1 行だけ読むメソッド。	第12巻 UNIT 09
`.strip()`	前後の空白や改行を取り除いたものを返すメソッド（Rust の `trim`）。	第12巻 UNIT 09
`.HTTPConnection()`	`http.client.HTTPConnection(ホスト, ポート)` は、HTTP で相手のサーバーにつなぐ物を作る関数です。	第12巻 UNIT 09
`.items()`	辞書の「キーと値」の組を 1 つずつ取り出せる形にするメソッド。`for k, v in d.items():` で、両方を同時に使えます。	第12巻 UNIT 09
`.request()`	`http.client` の接続で、HTTP の頼み（メソッド・パス・本文・ヘッダー）を送るメソッド。	第12巻 UNIT 09
`.getresponse()`	送った頼みに対する、相手の返事を受け取るメソッド。	第12巻 UNIT 09
`.write()`	ファイルに文字を書くメソッド。`f.write(line + "\n")`（改行は自分で付けます）。	第12巻 UNIT 09
`.send_response()`	HTTP のサーバーとして、答えの番号（200 など）を返し始めるメソッド。	第12巻 UNIT 09
`.getheaders()`	受け取った返事のヘッダーを、`(名前, 値)` の組の並びで返すメソッド。	第12巻 UNIT 09
`.send_header()`	HTTP のサーバーとして、ヘッダーを 1 つ返すメソッド。	第12巻 UNIT 09
`.end_headers()`	ヘッダーの終わりを知らせるメソッド。このあとに本文を書きます。	第12巻 UNIT 09
`.close()`	接続やファイルを閉じるメソッド。	第12巻 UNIT 09

コード	意味と使い方	初めて出てくる所
`.serve_forever()`	HTTP のサーバーを、止めるまでずっと動かし続けるメソッド。	第12巻 UNIT 09
`urlsplit()`	URL を、部品（`scheme`・`hostname`・`port`・`path` など）に分ける関数。	第12巻 UNIT 09
`b"…"`	`b"…"` は、文字ではなく「バイトの並び」（`bytes`）を作る書き方です。`b"\xff\xd8\xff"` は、JPEG の画像のファイルの、最初の 3 バイトです。	第12巻 UNIT 09
`*args`	`def f(*args):` の `*args` は、「いくつでも引数を受け取り、タプル（並び）にまとめる」書き方です。	第12巻 UNIT 09
`{キー: 値 for …}`	`{u.pk: u for u in users}` は、「1 つずつから、キーと値の組を作って、辞書にする」書き方です（辞書内包表記）。	第12巻 UNIT 09
`[開始:終わり]`	`text[1:4]` は、1 番目から 4 番目の手前まで（1, 2, 3）を取り出します（スライス）。`[:3]` は最初から、`[3:]` は 3 番目から最後まで、`[::-1]` は逆さまです。	第12巻 UNIT 09
`-> 型`	`def apply(self, action: str) -> None:` の `-> None` は、「この関数が返す物の型」の説明です（`None` は「何も返さない」）。	第12巻 UNIT 09
`名前: 型`	`def apply(self, action: str)` の `: str` は、「この引数は文字」という型の説明（型ヒント）です。Python は動かす時には確かめませんが、エディターや mypy が間違いを教えてくれます。	第12巻 UNIT 09
`self`	クラスの中の関数（メソッド）の最初の引数 `self` は、「その関数を呼んだ、その物自身」です。`self.amount` で、自分の項目を読み書きします。呼ぶ時は、`self` は書きません（`r.apply("submit")`）。	第12巻 UNIT 09
`not in`	`key not in TRANSITIONS` は、「`key` が `TRANSITIONS` の中に無い」です。`in` の反対です。	第12巻 UNIT 09
`BaseHTTPRequestHandler`	`BaseHTTPRequestHandler`（`http.server`）は、Python で小さな HTTP のサーバーを作る時の、要求を受ける係の元になる型です。`do_GET`・`do_POST` を書くと、その要求に答えます。	第12巻 UNIT 09
`ThreadingHTTPServer`	`ThreadingHTTPServer`（`http.server`）は、要求を同時にいくつもさばける、Python の小さな HTTP のサーバーです。	第12巻 UNIT 09
`ValueError`	`ValueError` は、「型は合っているが、値がおかしい」時のエラーです。`int("abc")` で起きます。	第12巻 UNIT 09
`.path`	`self.path` は、届いた HTTP の要求の場所（`/notes?page=2` のような URL の部分）です。	第12巻 UNIT 09